УЧЕБНОЕ ПОСОБИЕ · 4 КУРС

Kotlin | Jetpack Compose | Material 3 | Room | Retrofit | Hilt | Glide

Мобильная разработка

на Kotlin

Сборник практических работ — 14 практик, 72 часа

Учебно-методическое пособие по современной Android-разработке: декларативный UI на Jetpack Compose, Material 3, локальная база данных Room, сетевой слой Retrofit, внедрение зависимостей Hilt, корутины и Flow, работа с изображениями через Coil, интероп с View, основы рефлексии и тестирование. Все практики построены вокруг одного сквозного проекта NotesApp.

14	72	8+
ПРАКТИК	ЧАСА	ТЕХНОЛОГИЙ

Оглавление

Практика 1. Введение в Kotlin и Android Studio 5

Цель и задачи . . . 5
Теоретическая часть . . . 5
Примеры кода . . . 15
Задание . . . 16
Контрольные вопросы . . . 17
Чек-лист сдачи работы . . . 18
Что сдавать . . . 18

Практика 2. Основы Jetpack Compose 19

Цель и задачи . . . 19
Теоретическая часть . . . 19
Примеры кода . . . 28
Задание . . . 30
Контрольные вопросы . . . 31
Чек-лист сдачи работы . . . 32
Что сдавать . . . 32

Практика 3. Material 3 и MaterialTheme 33

Цель и задачи . . . 33
Теоретическая часть . . . 33
Примеры кода . . . 40
Задание . . . 42
Контрольные вопросы . . . 44
Чек-лист сдачи работы . . . 44
Что сдавать . . . 44

Практика 4. Layouts и навигация в Compose 46

Цель и задачи . . . 46
Теоретическая часть . . . 46
Примеры кода . . . 53
Задание . . . 56
Контрольные вопросы . . . 57
Чек-лист сдачи работы . . . 57

Что сдавать . . . 58

Практика 5. Coroutines и Flow: основы 59

Цель и задачи . . . 59
Теоретическая часть . . . 59
Примеры кода . . . 66
Задание . . . 68
Контрольные вопросы . . . 70
Чек-лист сдачи работы . . . 70
Что сдавать . . . 71

Практика 6. State management и ViewModel 72

Цель и задачи . . . 72
Теоретическая часть . . . 72
Примеры кода . . . 75
Задание . . . 76
Контрольные вопросы . . . 77
Чек-лист сдачи работы . . . 78
Что сдавать . . . 78

Практика 7. Room: локальная база данных 79

Цель и задачи . . . 79
Теоретическая часть . . . 79
Примеры кода . . . 83
Задание . . . 85
Контрольные вопросы . . . 86
Чек-лист сдачи работы . . . 87
Что сдавать . . . 87

Практика 8. Retrofit: работа с сетью 88

Цель и задачи . . . 88
Теоретическая часть . . . 88
Примеры кода . . . 92
Задание . . . 93
Контрольные вопросы . . . 95
Чек-лист сдачи работы . . . 96
Что сдавать . . . 96

Практика 9. Dagger Hilt: внедрение зависимостей 97

Цель и задачи . . . 97
Теоретическая часть . . . 97
Примеры кода . . . 101
Задание . . . 103
Контрольные вопросы . . . 104
Чек-лист сдачи работы . . . 105
Что сдавать . . . 105

Практика 10. Glide: загрузка изображений 106

Цель и задачи . . . 106
Теоретическая часть . . . 106
Примеры кода . . . 114
Задание . . . 116
Контрольные вопросы . . . 118
Чек-лист сдачи работы . . . 118
Что сдавать . . . 119

Практика 11. ViewFactory и интероп с View 120

Цель и задачи . . . 120
Теоретическая часть . . . 120
Примеры кода . . . 124
Задание . . . 125
Контрольные вопросы . . . 127
Чек-лист сдачи работы . . . 127
Что сдавать . . . 128

Практика 12. Рефлексия в Kotlin (ознакомление) 129

Цель и задачи . . . 129
Теоретическая часть . . . 129
Примеры кода . . . 133
Задание . . . 133
Контрольные вопросы . . . 135
Чек-лист сдачи работы . . . 135
Что сдавать . . . 136

Практика 13. Тестирование Android-приложений 137

Цель и задачи . . . 137
Теоретическая часть . . . 137
Примеры кода . . . 146
Задание . . . 148
Контрольные вопросы . . . 154
Чек-лист сдачи работы . . . 155
Что сдавать . . . 156

Практика 14. Финальный проект: сборка приложения 157

Цель и задачи . . . 157
Теоретическая часть . . . 157
Задание . . . 166
Контрольные вопросы (расширенные) . . . 167
Критерии оценки финального проекта . . . 168
Чек-лист сдачи работы . . . 168
Что сдавать . . . 169

Практика 1. Введение в Kotlin и Android Studio

Трудоёмкость: 6 академических часов. **Форма работы:** индивидуальная, в IDE Android Studio. **Форма отчёта:** исходный код проекта + устная защита.

Цель и задачи

Цель работы — познакомиться с языком программирования Kotlin как основным инструментом разработки под Android, освоить структуру Android-проекта в IDE Android Studio и подготовить рабочее место для всех последующих практикумов. Студенты уже владеют основами программирования на Java или другом ООП-языке, поэтому фокус сделан на отличиях Kotlin от Java и на архитектурных решениях современного Android-проекта (Gradle Kotlin DSL, модули, манифест, ресурсы).

По итогам работы студент должен уметь:

1. создать новый Android-проект с шаблоном Empty Activity (Compose) и объяснить назначение каждого элемента в структуре каталогов;

2. прочитать и модифицировать файл **build.gradle.kts** на уровне модуля (добавить зависимость, изменить minSdk);

3. описать ключевые отличия Kotlin от Java: *null-safety*, *data class*, *sealed class*, функции расширения, *scope functions*, инфиксные операторы;

4. запустить приложение на эмуляторе и на физическом устройстве, прочитать лог в **Logcat**.

Теоретическая часть

1.2.1. Краткая история Kotlin

Kotlin — статически типизированный язык программирования, разработанный компанией JetBrains и официально представленный в 2011 году. В 2017 году Google объявил Kotlin первоклассным языком разработки под Android, а с 2019 года все новые примеры и библиотеки AndroidX публикуются именно на Kotlin. На сегодняшний день Kotlin — обязательный язык для современного Android-разработчика: Jetpack Compose, Coroutines, Room, Hilt и другие ключевые библиотеки спроектированы с учётом его возможностей.

Главные преимущества Kotlin по сравнению с Java для Android: лаконичный синтаксис (на 30–40% меньше строк в типичном файле), встроенная *null-safety* на уровне типа, расширение классов без наследования, удобные неизменяемые коллекции, умное приведение типов (*smart cast*), поддержка корутин как нативной конструкции языка, а также полная interoperability с Java — можно вызывать Java-библиотеки и наоборот без накладных расходов.

1.2.2. Null-safety и типобезопасность

В Java ссылка на любой объект может быть null, и попытка вызвать метод по такой ссылке приводит к NullPointerException (NPE) уже в рантайме. В Kotlin тип по умолчанию не допускает null: переменная типа String не может содержать null, а String? — может. Компилятор проверяет все обращения к

nullable-переменным и требует явной проверки или оператора безопасного вызова ?.. Это исключает целый класс ошибок ещё на этапе компиляции.

```kotlin
// Kotlin — null-safety на уровне типа
var a: String = "hello" // не может быть null
var b: String? = null // может быть null

// a.length // OK — a не может быть null
// b.length // ошибка компиляции
b?.length // безопасный вызов: вернёт Int? (null если b == null)
b?.length ?: -1 // elvis-оператор: дефолтное значение
b!!.length // assert not-null: бросит NPE если b == null (избегать)
```

1.2.3. Классы данных (data class)

Подавляющее большинство классов в Android-приложении — это контейнеры данных (модели из сети, сущности БД, UI-состояния). В Java для них пишут boilerplate: геттеры, сеттеры, equals, hashCode, toString, copy. Ключевое слово data class генерирует все эти методы автоматически на основе свойств, перечисленных в первичном конструкторе. Главное ограничение: data class должен иметь хотя бы один val/var параметр в первичном конструкторе и не может быть унаследован (можно наследоваться от sealed class).

```kotlin
data class Note(
val id: Long,
val title: String,
val content: String,
val createdAt: Long,
val type: NoteType,
)

// Автоматически: equals, hashCode, toString, copy, component1..N
val n1 = Note(1, "Купить молоко", "...", System.currentTimeMillis(), NoteType.TEXT)
val n2 = n1.copy(title = "Купить хлеб") // изменяем только title
val (id, title) = n1 // деструктуризация
```

1.2.4. Sealed class — закрытая иерархия

sealed class задаёт иерархию, в которой все прямые наследники должны быть объявлены в том же файле (с Kotlin 1.5 — в том же модуле). Это позволяет компилятору знать полный список наследников и проверять полноту when-выражения: если перечислены все ветки, ветка else не нужна, и при добавлении нового наследника компилятор выдаст ошибку во всех when, где ветвление стало неполным. Sealed class активно используется в архитектурных паттернах Android: *UiState*, *NetworkResult*, *ViewEffect*.

```kotlin
sealed class NoteType {
data object Text : NoteType()
data object Image : NoteType()
data object Audio : NoteType()
data class Link(val url: String) : NoteType()
}

fun iconFor(t: NoteType): Int = when (t) {
is NoteType.Text -> R.drawable.ic_text
is NoteType.Image -> R.drawable.ic_image
is NoteType.Audio -> R.drawable.ic_audio
is NoteType.Link -> R.drawable.ic_link // ветка else не нужна
}
```

1.2.5. Функции расширения (extension functions)

Функция расширения позволяет добавить метод к существующему классу без наследования и без изменения исходного класса. Под капотом это статический вызов с первым параметром — получателем; расширения **не** модифицируют класс. В Android расширения используются повсеместно: View.visible(), Context.dp(px), Flow.collectAsState() и т. д. Чтобы не плодить глобальные функции, расширения принято группировать в файлы по домену (ViewExt.kt, DateExt.kt).

```kotlin
// Расширение для Note
fun Note.summary(maxChars: Int = 50): String =
if (content.length <= maxChars) content
else content.take(maxChars) + "…"

// Расширение для View
fun View.visible() { visibility = View.VISIBLE }
fun View.gone() { visibility = View.GONE }

// Использование
val n = Note(1, "x", "Очень длинный текст заметки", 0L, NoteType.Text)
println(n.summary()) // "Очень длинный текст заметки"
```

1.2.6. Scope functions: let, run, with, apply, also

Пять scope-функций — let, run, with, apply, also — позволяют выполнить блок кода в контексте объекта. Различаются двумя параметрами: чем является объект внутри блока (this или it) и что возвращает блок (сам объект или результат лямбды). Эта таблица — самый быстрый способ запомнить различия; её стоит выучить.

Функция	Объект как	Возвращает	Типичное применение
`let`	`it`	`результат лямбды`	null-checks: obj?.let { ... }
`run`	`this`	`результат лямбды`	вычисления в контексте объекта
`with`	`this`	`результат лямбды`	группировка вызовов над одним объектом
`apply`	`this`	`сам объект`	конфигурация: builder.apply { ... }
`also`	`it`	`сам объект`	side-effects: логирование, валидация

1.2.7. Структура Android-проекта

Современный Android-проект на Kotlin строится на системе сборки Gradle с Kotlin DSL (файлы build.gradle.kts). Корневой build.gradle.kts описывает общие настройки для всех модулей (плагины, репозитории). На уровне модуля (обычно :app) указываются: namespace, compileSdk, minSdk, зависимости и buildFeatures (например, флаг compose = true). Каталог src/main содержит исходный код Kotlin, ресурсы и манифест.

Путь / файл	Назначение
`settings.gradle.kts`	Объявляет список модулей проекта и репозитории Maven
`build.gradle.kts (root)`	Версии плагинов Android, Kotlin, Hilt; общие репозитории
`app/build.gradle.kts`	Конфигурация модуля: SDK, зависимости, buildTypes, flavors
`app/src/main/AndroidManifest.xml`	Описание приложения: package, activity, permissions
`app/src/main/java/.../`	Исходный код Kotlin (организованный по пакетам)
`app/src/main/res/`	Ресурсы: layout, drawable, values (strings, colors, themes)
`app/src/test/`	Локальные unit-тесты (JVM, без устройства)
`app/src/androidTest/`	Инструментальные тесты (требуют эмулятор/устройство)

1.2.8. AndroidManifest — точка входа

Манифест — это XML-описание приложения для системы Android. В нём объявляются: имя пакета (устаревший атрибут, теперь задаётся в build.gradle.kts через namespace), разрешения (<uses-permission>), компоненты — Activity, Service, BroadcastReceiver, ContentProvider. У каждой Activity должен быть указан intent-filter, если она доступна извне. Главная точка входа приложения помечается фильтром android.intent.action.MAIN + категорией android.intent.category.LAUNCHER.

```xml
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

<!-- Разрешения -->
<uses-permission android:name="android.permission.INTERNET" />

<application
android:allowBackup="true"
android:icon="@mipmap/ic_launcher"
android:label="@string/app_name"
android:theme="@style/Theme.NotesApp">

<activity
android:name=".MainActivity"
android:exported="true">
<intent-filter>
<action android:name="android.intent.action.MAIN" />
<category android:name="android.intent.category.LAUNCHER" />
</intent-filter>
</activity>
</application>
</manifest>
```

1.2.9. Жизненный цикл Activity (кратко)

Жизненный цикл Activity — это последовательность обратных вызовов, которые система вызывает при смене состояния экрана. Базовые состояния: onCreate → onStart → onResume → onPause → onStop → onDestroy. В Compose-приложении вы редко пишете эти методы вручную — компонент ComponentActivity и viewModel() берут на себя управление жизненным циклом. Тем не менее, знать основы необходимо: например, onCreate — единственное место, где вызывается setContent { ... }, а onDestroy — последний шанс освободить ресурсы (закрыть файлы, остановить корутины, не привязанные к ViewModel).

1.2.10. Logcat и отладка

Logcat — окно в Android Studio, отображающее лог-поток приложения и системы. Используйте функцию Log.d(TAG, msg) для отладочного вывода вместо println: Logcat фильтрует по TAG и приоритету. Точки останова (breakpoints) работают как в любой IDE: клик в gutter → Run → Debug. Для Compose существует плагин **Layout Inspector**, позволяющий видеть дерево Composable прямо на работающем приложении.

1.2.11. Object- declarations и companion object

В Kotlin нет static-членов в классах — вместо них используются object-декларации. Три варианта:

- **object Name { ... }** — Singleton. Один экземпляр на весь процесс приложения. Используется для утилитарных классов, фабрик, глобального состояния.
- **companion object { ... }** — объект внутри класса. Его члены доступны статически через имя класса: MyClass.MEMBER. Аналог static-членов Java. Может иметь имя и реализовывать интерфейсы.
- **object : Listener { ... }** — анонимный object, реализующий интерфейс или наследующий класс. Используется как замена Java-анонимным классам, особенно когда нужно переопределить несколько методов.

KOTLIN

```
// Singleton
object AppConstants {
const val VERSION = "1.0"
val SUPPORTED_TYPES = listOf("text", "image", "audio")
fun isSupported(type: String) = type in SUPPORTED_TYPES
}

// Companion object
class Note(
val id: Long,
val title: String,
/* ... */
) {
companion object {
const val MAX_TITLE_LENGTH = 120
const val NO_ID = -1L

fun empty() = Note(NO_ID, "", "", 0L, NoteType.Text)
}
}

// Использование
val maxLen = Note.MAX_TITLE_LENGTH
val empty = Note.empty()
```

1.2.12. Delegated properties — by lazy, by observable, by vetoable

Делегированные свойства — свойства, которые делегируют свою реализацию другому объекту через by. Самые полезные встроенные делегаты:

Делегат	Назначение	Пример
`by lazy { ... }`	Ленивая инициализация (первый доступ)	val db by lazy { AppDatabase.getInstance(ctx) }
`by Delegates.observable(init) { }`	Уведомление при изменении	var count by Delegates.observable(0) { _, old, new -> log(old, new) }
`by Delegates.vetoable(init) { }`	Запрет изменения (return false)	var age by Delegates.vetoable(0) { _, _, new -> new >= 0 }
`by remember { mutableStateOf(...) }`	Compose-состояние (переживает recomposition)	var text by remember { mutableStateOf("") }
`by NullableVar`	Кастомный делегат для nullable-поля	—

```kotlin
// lazy: потокобезопасная ленивая инициализация
val database by lazy(LazyThreadSafetyMode.SYNCHRONIZED) {
Room.databaseBuilder(ctx, AppDatabase::class.java, "notes.db").build()
}

// observable: уведомление при изменении
var selectedNoteId by Delegates.observable(-1L) { _, old, new ->
if (old != new) {
Log.d(TAG, "Selected: $new")
// например, перезагрузить детали
}
}

// vetoable: запретить отрицательные значения
var itemsPerPage by Delegates.vetoable(20) { _, _, new -> new in 1..100 }
```

1.2.13. Inline (value) classes и type aliases

Inline class (с Kotlin 1.5 — @JvmInline value class) — это класс-обёртка над одним значением, который компилятор «разворачивает» в само значение в байт-коде. Это даёт типобезопасность на этапе компиляции без накладных расходов в рантайме — объект не создаётся, хранится только внутреннее значение. Идеально для type-safe идентификаторов.

```kotlin
@JvmInline
value class NoteId(val value: Long) {
init {
require(value >= 0) { "NoteId must be non-negative" }
}
}

@JvmInline
value class Email(val value: String) {
init {
require(value.contains("@")) { "Invalid email" }
}
}

fun findNote(id: NoteId): Note?
fun sendTo(email: Email)

// Использование — невозможно перепутать NoteId с UserId
val note = findNote(NoteId(42L))
// sendTo(note.id) // ошибка компиляции — NoteId не Email
sendTo(Email("user@example.com"))
```

Type alias — просто псевдоним для существующего типа. Не создаёт нового типа (как inline class), но улучшает читаемость. Часто используется для сложных generic-типов:

```kotlin
typealias NoteList = List<Note>
typealias NoteFlow = Flow<List<Note>>
typealias EventHandler<T> = (T) -> Unit

// Теперь можно писать:
fun observeNotes(): NoteFlow = repo.observeAll()
fun setOnNoteClickListener(listener: EventHandler<Note>)
```

1.2.14. Extension functions — поглубже

Функции расширения могут иметь несколько синтаксических форм:

- **Обычное расширение** — fun Note.summary() = ...
- **Расширение-свойство** — val Note.isAudio: Boolean get() = type is NoteType.Audio
- **Расширение на nullable-типе** — fun String?.orEmpty(): String
- **Расширение на generic-типе** — fun <T> List<T>.randomOrNull(): T?
- **Инфиксное расширение** — infix fun Int.times(action: () -> Unit), вызывается 5.times { println("hi") }
- **Operator-расширение** — operator fun Note.plus(other: Note): List<Note>, вызывается note1 + note2

KOTLIN

```
// Оператор-расширение
operator fun List<Note>.plus(note: Note): List<Note> = this + note

// Теперь можно писать:
val notes = emptyList<Note>()
val withNew = notes + Note(1, "test", "...", 0L, NoteType.Text)
// эквивалентно notes.plus(Note(...))

// Инфиксное расширение для читаемого DSL
infix fun Note.titled(newTitle: String) = copy(title = newTitle)

val renamed = note titled "Новое название"
```

1.2.15. Gradle Kotlin DSL — build types и product flavors

Современный Android-проект использует Gradle Kotlin DSL — файлы build.gradle.kts. Главное преимущество перед Groovy — статическая типизация, автодополнение в IDE, возможность рефакторить скрипты как обычный Kotlin-код. Стандартная конфигурация модуля :app включает buildTypes (debug, release) и опционально productFlavors (например, free/paid, dev/prod).

KOTLIN

```
android {
defaultConfig {
applicationId = "ru.notesapp"
minSdk = 24
targetSdk = 34
versionCode = 1
versionName = "1.0"
}

buildTypes {
getByName("debug") {
isDebuggable = true
applicationIdSuffix = ".debug" // ru.notesapp.debug
versionNameSuffix = "-debug"
buildConfigField("String", "API_URL", "\"https://dev.example.com/\"")
}
getByName("release") {
isMinifyEnabled = true
isShrinkResources = true
proguardFiles(
getDefaultProguardFile("proguard-android-optimize.txt"),
"proguard-rules.pro",
)
buildConfigField("String", "API_URL", "\"https://api.example.com/\"")
}
}

flavorDimensions += listOf("tier", "env")
productFlavors {
create("free") { dimension = "tier" }
create("paid") { dimension = "tier" }
create("dev") { dimension = "env" }
create("prod") { dimension = "env" }
}
}
```

В результате собирается 8 build variants: freeDevDebug, freeDevRelease, freeProdDebug, ... Каждый вариант имеет свой BuildConfig.API_URL и может иметь свой applicationId. В коде доступ к BuildConfig — генерируемый класс, см. import ru.notesapp.BuildConfig.

1.2.16. Version Catalogs — libs.versions.toml

С Gradle 7+ появилась официальная рекомендация — выносить версии зависимостей в **Version Catalog**: файл gradle/libs.versions.toml. Это даёт единый источник правды для всех модулей и упрощает обновление.

```toml
# gradle/libs.versions.toml
[versions]
kotlin = "1.9.24"
compose-bom = "2024.09.00"
room = "2.6.1"
hilt = "2.51.1"

[libraries]
compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref =
"compose-bom" }
compose-ui = { group = "androidx.compose.ui", name = "ui" }
compose-material3 = { group = "androidx.compose.material3", name = "material3" }
room-runtime = { group = "androidx.room", name = "room-runtime", version.ref = "room" }
hilt-android = { group = "com.google.dagger", name = "hilt-android", version.ref = "hilt" }

[plugins]
android-application = { id = "com.android.application", version = "8.5.0" }
kotlin-android = { id = "org.jetbrains.kotlin.android", version.ref = "kotlin" }

# B build.gradle.kts:
dependencies {
implementation(platform(libs.compose.bom))
implementation(libs.compose.ui)
implementation(libs.compose.material3)
implementation(libs.room.runtime)
implementation(libs.hilt.android)
}
```

1.2.17. Android Studio Profilers — CPU, Memory, Network

Android Studio имеет встроенный профайлер (View → Tool Windows → Profiler). Он показывает в реальном времени четыре метрики:

• **CPU** — график загрузки процессора + flame graph. Видно, какие методы занимают больше всего времени. Запись trace (Record) позволяет детально проанализировать узкие места.

• **Memory** — график использования heap. Видно распределение объектов по типам, утечки памяти. Кнопка «Capture heap dump» делает снимок, по которому можно найти, кто держит ссылку на Activity.

• **Network** — график трафика + список запросов. Можно кликнуть на запрос и увидеть заголовки, тело, ответ.

• **Energy** — оценка энергопотребления (CPU + GPS + Network + Camera). Помогает оптимизировать расход батареи.

Для диагностики ANR (Application Not Responding) и зависаний UI используйте **System Trace** — показывает, что происходило на main thread в каждую миллисекунду. Запись через Debug.startMethodTracing() или через Profiler UI.

1.2.18. Layout Inspector и APK Analyzer

Layout Inspector (Tools → Layout Inspector) — позволяет увидеть дерево View на работающем приложении, выделить любой элемент и посмотреть его размеры, отступы, атрибуты. С Jetpack Compose показывает дерево Composable-функций с их параметрами — незаменимо для отладки layout-а.

APK Analyzer (Build → Analyze APK) — открывает собранный APK как архив, показывает: размер каждого файла, размер по DEX-классам, содержимое AndroidManifest, ресурсы. Полезно для поиска причины большого размера APK и проверки, что R8 действительно удалил неиспользуемый код.

1.2.19. Logcat — продвинутые приёмы

Logcat поддерживает фильтры через специальный синтаксис:

- tag:NotesApp — только сообщения с этим TAG
- level:ERROR — только ERROR и выше (ASSERT)
- package:ru.notesapp — только от этого пакета
- message:"Note saved" — поиск по тексту сообщения
- -tag:GoogleApi — минус означает исключение
- Комбинируются через пробел: tag:NotesApp level:DEBUG

В коде используйте функции по приоритету: Log.v (verbose), Log.d (debug), Log.i (info), Log.w (warning), Log.e (error). В release-сборке verbose и debug обычно отключают через ProGuard или обёртку-фасад.

Примеры кода

Ниже — три законченных примера, которые понадобятся в задании. Изучите их структуру, прежде чем приступать к самостоятельной реализации.

Пример 1.1. Структура проекта и HelloWorld-Activity

KOTLIN

```
// app/src/main/java/ru/notesapp/MainActivity.kt
package ru.notesapp

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.material3.Text
import androidx.compose.material3.MaterialTheme

class MainActivity : ComponentActivity() {
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContent {
MaterialTheme {
Text(text = "Привет, Kotlin Android!")
}
}
}
}
```

Пример 1.2. Sealed class NoteType + data class Note

KOTLIN

```
package ru.notesapp.domain

sealed class NoteType {
data object Text : NoteType()
data object Image : NoteType()
data object Audio : NoteType()
}

data class Note(
val id: Long,
val title: String,
val content: String,
val createdAt: Long,
val type: NoteType,
)
```

Пример 1.3. Extension function Note.summary()

KOTLIN

```
package ru.notesapp.domain

fun Note.summary(maxChars: Int = 50): String =
buildString {
append("[")
append(when (type) {
is NoteType.Text -> "T"
is NoteType.Image -> "I"
is NoteType.Audio -> "A"
})
append("] ")
append(if (content.length <= maxChars) content
else content.take(maxChars) + "…")
}
```

Задание

Создайте новый Android-проект **NotesApp** — он будет развиваться на протяжении всех 14 практикумов и станет финальным проектом. На этой практике достаточно заложить структуру: модель данных и одну функцию-расширение. **Важно:** имена пакетов, классов и функций должны точно совпадать с теми, что указаны ниже — иначе на последующих практиках придётся переделывать.

Шаг 1. Создание проекта

1. В Android Studio: File → New → New Project → Empty Activity (Compose).

2. Name: **NotesApp**, Package name: **ru.notesapp**, Language: Kotlin, Minimum SDK: API 24 (Android 7.0).

3. Build configuration language: Kotlin DSL (build.gradle.kts).

4. Дождитесь окончания Gradle Sync.

Шаг 2. Пакетная структура

Создайте в каталоге app/src/main/java/ru/notesapp/ следующие подпакеты — они станут основой архитектуры:

• ru.notesapp.domain — модели предметной области (Note, NoteType);

• ru.notesapp.data — слой данных (репозитории, БД, сеть);

• ru.notesapp.ui — Composable-функции и экраны;

• ru.notesapp.ui.theme — Material 3 тема;

• ru.notesapp.viewmodel — ViewModels;

• ru.notesapp.util — расширения и утилиты.

Шаг 3. Реализация моделей

В файле domain/Note.kt создайте:

1. sealed class NoteType с тремя наследниками: Text, Image, Audio (используйте data object);

2. data class Note с полями: id: Long, title: String, content: String, createdAt: Long (Unix-миллисекунды), type: NoteType.

Шаг 4. Функция-расширение Note.summary()

В файле domain/NoteExt.kt реализуйте функцию расширения fun Note.summary(maxChars: Int = 50): String, которая возвращает строку вида:

```
TEXT

"[T] первые_maxChars_символов_content…"
// где T/I/A — буква, соответствующая типу заметки
// … добавляется только если content длиннее maxChars
```

Реализация должна использовать when по type и buildString. Запрещается использовать if-else для выбора буквы типа — только when по sealed class.

Шаг 5. Проверка в MainActivity

В MainActivity.onCreate создайте список из 3 разных заметок и выведите их summary() в Logcat с TAG "NotesApp". Запустите приложение и убедитесь, что вывод корректный.

Контрольные вопросы

Ответьте письменно на следующие вопросы. Подготовьте устные ответы для защиты практической работы — преподаватель может задать любой из них или попросить пояснить код вашего решения.

1. Чем Kotlin отличается от Java для Android-разработки? Назовите минимум 4 конкретных преимущества.

2. Что такое null-safety в Kotlin? Чем String отличается от String??

3. Что делает оператор ?.? А оператор ?: (elvis)? Приведите примеры.

4. Почему не рекомендуется использовать оператор !!? В каких случаях его применение оправдано?

5. Какие методы автоматически генерирует data class? Может ли data class быть унаследован?

6. Что такое sealed class и зачем он нужен? Почему в when-выражении по sealed class не нужна ветка else?

7. Чем функция расширения отличается от обычного метода класса? Можно ли «переопределить» функцию расширения?

8. В чём разница между apply и also? Приведите типичный сценарий для каждой.

9. Что делает buildString? Как она реализована под капотом (базовый класс)?

10. Какие файлы обязательно присутствуют в модуле app Android-проекта? Опишите назначение каждого.

11. Зачем нужен AndroidManifest.xml? Какие элементы в нём объявляются?

12. Как отфильтровать вывод Logcat по тегу и приоритету? Какая функция используется для отладочного лога в Kotlin?

Чек-лист сдачи работы

Перед сдачей работы преподавателю убедитесь, что выполнены все пункты ниже. Отсутствие любого пункта снижает оценку минимум на один балл.

- □ Создан проект NotesApp с package ru.notesapp
- □ minSdk = 24, compileSdk = 34 (или выше)
- □ Используется Kotlin DSL (build.gradle.kts, не build.gradle)
- □ Созданы подпакеты domain, data, ui, ui.theme, viewmodel, util
- □ В domain/Note.kt есть sealed class NoteType с тремя data object-наследниками
- □ В domain/Note.kt есть data class Note с точными именами и типами полей
- □ В domain/NoteExt.kt есть функция-расширение Note.summary(maxChars: Int = 50): String
- □ В summary() используется when по sealed class (без else) и buildString
- □ В MainActivity.onCreate создаются 3 заметки разных типов, их summary выводятся в Logcat с TAG "NotesApp"
- □ Приложение запускается на эмуляторе API 24+ без crash

Что сдавать

В результате выполнения практической работы студент предоставляет:

1. Git-репозиторий с коммитом «Практика 1» (или ZIP-архив проекта);

2. Скриншот Logcat с тремя выводами summary;

3. Краткий отчёт (1 страница): что сделано, какие возникли трудности, ответы на 3 контрольных вопроса по выбору преподавателя.

Практика 2. Основы Jetpack Compose

Трудоёмкость: 6 академических часов. **Форма работы:** индивидуальная, в IDE Android Studio. **Форма отчёта:** исходный код проекта + устная защита.

Цель и задачи

Цель работы — освоить декларативный подход к построению UI в Jetpack Compose и перейти от императивной модели View к композиции Composable-функций. Студент должен понять разницу между *imperative* и *declarative* UI, научиться описывать интерфейс как функцию от состояния и освоить базовый набор Composable: Text, Button, Image, Column, Row, Box, Spacer, LazyColumn.

По итогам работы студент должен уметь:

1. объяснить, что такое recomposition и почему Composable-функции должны быть без побочных эффектов;

2. использовать remember и mutableStateOf для локального состояния;

3. строить композиции из Column/Row/Box и применять Modifier для оформления;

4. использовать LazyColumn с ключами для списков;

5. применять preview для разработки без запуска приложения.

Теоретическая часть

2.2.1. Декларативный UI: что это и зачем

Императивный UI (классические View) требует описать **как** достичь нужного состояния: найти элемент через findViewById, вызвать setText, setVisibility и т. д. Декларативный UI описывает **что** должно отображаться при данном состоянии: Composable-функция принимает состояние и возвращает дерево элементов. При изменении состояния Compose сам определяет, какие части дерева нужно перерисовать (recompose).

Главные преимущества: меньше boilerplate-кода, отсутствие findViewById и setOnClickListener, естественная поддержка тёмной темы, простая анимация состояний, хорошая тестируемость. Минусы: выше порог входа, нужно думать о производительности recomposition, не все библиотеки поддерживают Compose.

2.2.2. @Composable и @Preview

Аннотация @Composable превращает обычную функцию Kotlin в Composable-функцию. Правила: функция может вызывать другие @Composable-функции; принимает параметры любых типов; возвращает Unit; должна быть чистой (без побочных эффектов — никаких записи в БД, сетевых запросов, изменения глобального состояния внутри Composable). Аннотация @Preview позволяет видеть результат в окне Android Studio без запуска приложения.

KOTLIN

```
import androidx.compose.runtime.Composable
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.material3.Text
import androidx.compose.material3.MaterialTheme

@Composable
fun Greeting(name: String) {
Text(
text = "Привет, $name!",
style = MaterialTheme.typography.bodyLarge,
)
}

@Preview(showBackground = true, name "Greeting preview")
@Composable
private fun GreetingPreview() {
Greeting(name = "Мир")
}
```

2.2.3. Recomposition — как Compose обновляет UI

Recomposition — повторный вызов Composable-функций при изменении состояния, от которого они зависят. Compose отслеживает чтение State<T> внутри Composable и при изменении значения помечает зависимые функции к перерисовке. Чтобы recomposition работала корректно и предсказуемо, Composable-функции должны быть:

- **без побочных эффектов** — не изменять глобальное состояние, не делать I/O (база данных, сеть, логи — только через LaunchedEffect);
- **идемпотентными** — многократный вызов с теми же параметрами даёт тот же результат;
- **быстрыми** — тяжёлые вычисления выносить в ViewModel;
- **stable-параметрами** — типы параметров должны быть *stable* (Immutable / data class с stable-полями), иначе Compose может пропускать оптимизации.

> **Главное правило Compose**
>
> Composable-функция описывает UI как функцию от состояния. Не пытайтесь «запомнить» Composable и мутировать его снаружи — меняйте состояние, и Compose сам перерисует то, что нужно.

2.2.4. Состояние: remember, mutableStateOf, derivedStateOf

remember { ... } сохраняет значение между recomposition в рамках одной композиции. mutableStateOf обёртывает значение в наблюдаемое состояние — при изменении Compose перерисует зависимые Composable. Идиоматичный способ — использовать делегат by:

```kotlin
import androidx.compose.runtime.*
import androidx.compose.material3.Button
import androidx.compose.material3.Text

@Composable
fun Counter() {
// count переживает recomposition; изменение триггерит перерисовку
var count by remember { mutableStateOf(0) }
Button(onClick = { count++ }) {
Text("Нажато: $count")
}
}
```

derivedStateOf создаёт производное состояние, которое пересчитывается только когда изменились исходные State. Полезно для тяжёлых вычислений и для предотвращения избыточной recomposition.

```kotlin
@Composable
fun TodoList(todos: List<Todo>) {
val doneCount by remember {
derivedStateOf { todos.count { it.isDone } }
}
Text("Выполнено: $doneCount из ${todos.size}")
}
```

2.2.5. Modifier — цепочка оформления

Modifier — неизменяемый объект-декоратор. Методы вида padding, fillMaxWidth, background, clickable возвращают новый Modifier, поэтому их вызовы объединяют в цепочку через точку. Порядок важен: Modifier.padding(16.dp).background(Red) даст красный фон с отступом снаружи, а Modifier.background(Red).padding(16.dp) — красный фон с отступом внутри (контент отделён от фона прозрачной полосой).

Modifier	Назначение	Часто комбинируют с
`padding(all)`	Внешний отступ	`fillMaxWidth`
`fillMaxWidth(f)`	Занять всю ширину (или долю f)	`padding`
`fillMaxSize()`	Занять всю доступную область	`background`
`size(w, h)`	Фиксированный размер	`clip`
`background(c)`	Фон (цвет / Brush)	`clip`
`clip(shape)`	Скруглить / обрезать по форме	`background`
`clickable { }`	Обработчик клика с ripple	`padding`
`weight(f)`	Доля в Row/Column (только внутри них)	`fillMaxWidth`
`align(a)`	Выравнивание внутри Box	`padding`
`aspectRatio(r)`	Соотношение сторон	`fillMaxWidth`

2.2.6. Базовые Composable: Column, Row, Box

Три главных layout-контейнера Compose:

• **Column** — вертикальный стек (как LinearLayout vertical). Параметры: verticalArrangement, horizontalAlignment.

• **Row** — горизонтальный стек. Параметры: horizontalArrangement, verticalAlignment.

• **Box** — стек с наложением (как FrameLayout). Параметр contentAlignment. Дочерние элементы выравниваются через Modifier.align().

KOTLIN

```
Column(
modifier = Modifier.fillMaxWidth().padding(16.dp),
verticalArrangement = Arrangement.spacedBy(12.dp),
) {
Text("Заголовок", style = MaterialTheme.typography.titleLarge)
Row(
modifier = Modifier.fillMaxWidth(),
horizontalArrangement = Arrangement.SpaceBetween,
) {
Text("Левый")
Text("Правый")
}
Box(
modifier = Modifier
.fillMaxWidth()
.height(120.dp)
.background(Color.LightGray),
contentAlignment = Alignment.Center,
) {
Text("По центру Box")
}
}
```

2.2.7. LazyColumn — списки в Compose

LazyColumn — аналог RecyclerView, перерисовывает только видимые элементы. Обязателен параметр key — уникальный идентификатор элемента (обычно id из БД). Без ключа Compose может путать элементы при анимациях вставки/удаления. Также поддерживает itemsIndexed, stickyHeader, horizontalLayout (через LazyRow).

```kotlin
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items

@Composable
fun NoteList(notes: List<Note>, onClick: (Note) -> Unit) {
LazyColumn(
contentPadding = PaddingValues(16.dp),
verticalArrangement = Arrangement.spacedBy(8.dp),
) {
items(
items = notes,
key = { it.id }, // уникальный ключ — обязательно!
) { note ->
NoteRow(note = note, onClick = onClick)
}
}
}
```

2.2.8. Производительность: stability и пропуск recomposition

Compose оптимизирует recomposition через проверку параметров: если все параметры Composable stable и не изменились — функция не вызывается повторно. **Stable** тип — это примитивы, String, все Immutable-типы, data class где все поля stable, и интерфейсы, помеченные @Stable. List, Map, Set по умолчанию *unstable* — Compose предполагает, что они могли измениться, и принудительно recompose. Решения: использовать ImmutableList из kotlinx.collections.immutable или помечать классы аннотацией @Immutable.

2.2.9. Effects API — LaunchedEffect, DisposableEffect, rememberCoroutineScope

Effects — Composable-функции, запускающие побочные эффекты (coroutine launch, callback registration, etc.) в контролируемой форме. Главное правило: Composable должен быть без побочных эффектов, но иногда они нужны — и для этого есть специальный API. Базовые эффекты:

Effect	Когда запускается	Когда отменяется
`LaunchedEffect(key) { }`	При входе в композицию (или смене key)	При выходе из композиции или смене key
`DisposableEffect(key) { }`	При входе / смене key	При выходе / смене key — вызывает блок onDispose
`rememberCoroutineScope()`	Сразу, возвращает CoroutineScope	При выходе из композиции
`rememberUpdatedState(value)`	При recomposition	Не отменяется (просто обновляет значение)
`derivedStateOf { }`	При изменении зависимостей	Не отменяется
`produceState(initial) { }`	При входе в композицию	При выходе — отменяет корутину
`snapshotFlow { }`	При подписке	При отписке

KOTLIN

```
// LaunchedEffect — запустить корутину при появлении
@Composable
fun NotesListScreen(viewModel: NotesViewModel) {
val uiState by viewModel.uiState.collectAsStateWithLifecycle()
LaunchedEffect(Unit) { // Unit = запуск один раз
viewModel.loadNotes()
}
/* ... */
}

// DisposableEffect — подписка на callback
@Composable
fun NetworkMonitor(onOnline: () -> Unit) {
DisposableEffect(Unit) {
val receiver = object : BroadcastReceiver() {
override fun onReceive(c: Context?, i: Intent?) { onOnline() }
}
context.registerReceiver(receiver, IntentFilter(ConnectivityManager.CONNECTIVITY_ACTION))
onDispose {
context.unregisterReceiver(receiver)
}
}
}

// rememberCoroutineScope — для обработки кликов
@Composable
fun SaveButton(onSave: suspend () -> Unit) {
val scope = rememberCoroutineScope()
var saving by remember { mutableStateOf(false) }
Button(
onClick = {
scope.launch {
saving = true
onSave()
saving = false
}
},
enabled = !saving,
) {
if (saving) CircularProgressIndicator(modifier = Modifier.size(16.dp))
else Text("Сохранить")
}
}
```

Главное правило LaunchedEffect

key определяет, когда эффект перезапустится. Если key = Unit — эффект запустится один раз и больше не перезапустится. Если key = someState — при каждом изменении someState старая корутина отменяется и запускается новая. Используйте ключи с умом, иначе эффект будет перезапускаться на каждый чих.

2.2.10. derivedStateOf и snapshotFlow — мост между Compose и Flow

derivedStateOf { } создаёт производное состояние, которое пересчитывается только когда изменились исходные State. Полезно для тяжёлых вычислений и для предотвращения избыточной recomposition. snapshotFlow { } — обратный мост: превращает чтение Compose State в cold Flow, который эммитит новое значение при каждом изменении State.

KOTLIN

```
// derivedStateOf — производное состояние
@Composable
fun TodoList(todos: List<Todo>) {
val doneCount by remember {
derivedStateOf { todos.count { it.isDone } }
}
val progress by remember {
derivedStateOf {
if (todos.isEmpty()) 0f else doneCount.toFloat() / todos.size
}
}
LinearProgressIndicator(progress = { progress })
Text("$doneCount из ${todos.size}")
}

// snapshotFlow — мост Compose → Flow
@Composable
fun SearchTracker(searchField: String) {
LaunchedEffect(Unit) {
snapshotFlow { searchField }
.debounce(300)
.distinctUntilChanged()
.filter { it.length >= 2 }
.collect { query ->
// выполнить поиск
}
}
}
```

2.2.11. CompositionLocal — неявная передача данных

CompositionLocal — способ передать данные вниз по дереву Composable без явных параметров. Полезно для данных, которые нужны многим компонентам на разных уровнях (тема, язык, NavController, SnackbarHostState). Создаётся через compositionLocalOf { default } (динамический) или staticCompositionLocalOf { default } (статический, не вызывает recomposition при смене значения).

```kotlin
// Объявление
val LocalSnackbarHost = compositionLocalOf<SnackbarHostState> {
error("No SnackbarHostState provided")
}

// Провайдинг
@Composable
fun AppRoot() {
val snackbarHostState = remember { SnackbarHostState() }
CompositionLocalProvider(LocalSnackbarHost provides snackbarHostState) {
NavHost(...)
}
}

// Использование в любом вложенном Composable
@Composable
fun DeepButton() {
val snackbarHost = LocalSnackbarHost.current
Button(onClick = {
snackbarHost.scope.launch {
snackbarHost.showSnackbar("Clicked deep inside!")
}
}) { Text("Click me") }
}
```

Используйте CompositionLocal осторожно: он усложняет понимание, откуда берутся данные. Хорошее правило: если данные нужны более чем 3 уровням вложенности — CompositionLocal оправдан. Если только 1-2 — лучше передавать параметром.

2.2.12. Custom layouts — Layout composable

Если ни один стандартный layout не подходит, можно написать свой через Composable Layout { }. Это аналог onMeasure + onLayout в View. Алгоритм: измерить дочерние Composable → вычислить свой размер → разместить дочерние. Для простых случаев удобнее BoxWithConstraints, но для полноценного custom layout нужен Layout.

```kotlin
// Vertical Staggered Layout — каждый элемент занимает свою высоту
@Composable
fun StaggeredVerticalGrid(
columns: Int,
modifier: Modifier = Modifier,
content: @Composable () -> Unit,
) {
Layout(content = content, modifier = modifier) { measurables, constraints ->
val colWidth = constraints.maxWidth / columns
val colHeights = IntArray(columns) { 0 }
val placeables = measurables.map { measurable ->
val col = (0 until columns).minByOrNull { colHeights[it] } ?: 0
val placeable = measurable.measure(
constraints.copy(maxWidth = colWidth)
)
colHeights[col] += placeable.height
placeable to col
}
val totalHeight = colHeights.maxOrNull() ?: 0
layout(constraints.maxWidth, totalHeight) {
val yOffsets = IntArray(columns) { 0 }
placeables.forEach { (placeable, col) ->
placeable.place(
x = col * colWidth,
y = yOffsets[col],
)
yOffsets[col] += placeable.height
}
}
}
}
```

2.2.13. Анимации в Compose — основы

Compose имеет богатый API анимаций. Главные инструменты:

- **animate<T>AsState(targetValue, animationSpec)** — анимировать одно значение. animateColorAsState, animateDpAsState, animateFloatAsState. Самый частый паттерн для hover/press-эффектов.
- **AnimatedVisibility(visible) { }** — появление/исчезновение Composable с анимацией.
- **AnimatedContent(targetState) { }** — кросс-фейд между разными Composable при смене state.
- **updateTransition(state)** — несколько анимаций от одного state.
- **rememberInfiniteTransition()** — бесконечная анимация (для shimmer, прогресс-баров).
- **Modifier.animateContentSize()** — анимация при изменении размера Composable.

KOTLIN

```
// animateColorAsState для кнопки
@Composable
fun ColorButton(isActive: Boolean, onClick: () -> Unit) {
val bgColor by animateColorAsState(
targetValue = if (isActive) Color.Blue else Color.Gray,
animationSpec = tween(durationMillis = 300),
label = "button-bg",
)
Button(
onClick = onClick,
colors = ButtonDefaults.buttonColors(containerColor = bgColor),
) { Text("Click") }
}

// AnimatedVisibility для snackbar-like уведомлений
@Composable
fun ToastMessage(visible: Boolean, message: String) {
AnimatedVisibility(
visible = visible,
enter = slideInVertically { it } + fadeIn(),
exit = slideOutVertically { -it } + fadeOut(),
) {
Box(
modifier = Modifier
.fillMaxWidth()
.background(Color.Black.copy(alpha = 0.8f))
.padding(16.dp),
) {
Text(message, color = Color.White)
}
}
}
```

Примеры кода

Пример 2.1. Счётчик с сохранением состояния

KOTLIN

```
@Composable
fun CounterScreen() {
var count by remember { mutableStateOf(0) }
var step by remember { mutableStateOf(1) }

Column(
modifier = Modifier.fillMaxSize().padding(24.dp),
horizontalAlignment = Alignment.CenterHorizontally,
verticalArrangement = Arrangement.spacedBy(16.dp, Alignment.CenterVertically),
) {
Text("Count = $count", style = MaterialTheme.typography.headlineMedium)
Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
Button(onClick = { count -= step }) { Text("−$step") }
Button(onClick = { count += step }) { Text("+$step") }
}
OutlinedTextField(
value = step.toString(),
onValueChange = { step = it.toIntOrNull() ?: 1 },
label = { Text("Шаг") },
keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
)
}
}
```

Пример 2.2. Список элементов с кликом

```kotlin
data class Task(val id: Long, val title: String, val isDone: Boolean)

@Composable
fun TaskList(tasks: List<Task>, onToggle: (Long) -> Unit) {
LazyColumn(
contentPadding = PaddingValues(vertical = 8.dp),
verticalArrangement = Arrangement.spacedBy(4.dp),
) {
items(tasks, key = { it.id }) { task ->
TaskRow(task = task, onToggle = { onToggle(task.id) })
}
}
}

@Composable
private fun TaskRow(task: Task, onToggle: () -> Unit) {
Row(
modifier = Modifier
.fillMaxWidth()
.clickable { onToggle() }
.padding(horizontal = 16.dp, vertical = 12.dp),
verticalAlignment = Alignment.CenterVertically,
) {
Checkbox(checked = task.isDone, onCheckedChange = { onToggle() })
Spacer(Modifier.width(12.dp))
Text(
text = task.title,
style = if (task.isDone) MaterialTheme.typography.bodyLarge
.copy(textDecoration = TextDecoration.LineThrough)
else MaterialTheme.typography.bodyLarge,
)
}
}
```

Задание

В проекте **NotesApp** (из практики 1) создайте UI-слой для отображения списка заметок. На этой практике ViewModel ещё не нужен — данные можно захардкодить в самой Composable через remember. Имена функций и параметров должны точно совпадать с указанными.

Шаг 1. Composable NoteCard

В файле ui/NoteCard.kt создайте Composable-функцию с сигнатурой:

```kotlin
@Composable
fun NoteCard(
note: Note,
onClick: () -> Unit,
modifier: Modifier = Modifier,
)
```

Требования к виду карточки:

1. Используется androidx.compose.material3.Card (Material 3).

2. Слева — цветной кружок 12 dp, цвет зависит от note.type: Text → синий (#3592C0), Image → зелёный (#3DDC84), Audio → оранжевый (#FFA726).

3. Заголовок note.title — MaterialTheme.typography.titleMedium, одна строка с эллипсисом (maxLines = 1, overflow = Ellipsis).

4. Подзаголовок note.summary() — bodyMedium, maxLines = 2.

5. В правом нижнем углу — текстовая метка с типом заметки («ТЕКСТ»/«КАРТИНКА»/«АУДИО»), labelSmall, цвет MaterialTheme.colorScheme.outline.

6. Весь Card обёрнут в Modifier.clickable { onClick() }.

Шаг 2. Composable NoteList

В файле ui/NoteList.kt создайте:

```kotlin
@Composable
fun NoteList(
notes: List<Note>,
onClick: (Note) -> Unit,
modifier: Modifier = Modifier,
)
```

1. Использовать LazyColumn.

2. Параметр key = { it.id } — обязательно.

3. contentPadding = PaddingValues(16.dp).

4. verticalArrangement = Arrangement.spacedBy(8.dp).

5. Если список пуст — показать EmptyState Composable с текстом «Пока нет заметок».

Шаг 3. Composable EmptyState

В файле ui/EmptyState.kt создайте:

```kotlin
@Composable
fun EmptyState(
title: String,
subtitle: String,
modifier: Modifier = Modifier,
)
```

Реализация: Column с центрированием, иконка-плейсхолдер (Icons.Filled.NoteAdd), title в titleMedium, subtitle в bodyMedium цвета outline.

Шаг 4. Подключение в MainActivity

В MainActivity.onCreate замените HelloWorld на вызов NoteList с захардкоженным списком из 5 заметок разных типов. Клик выводит в Logcat TAG "NotesApp" сообщение «Clicked: *id*».

Шаг 5. @Preview

Для каждой Composable-функции из шагов 1–3 создайте @Preview-функцию в том же файле. Preview для NoteList должен показывать список из 3 заметок; для EmptyState — пустое состояние; для NoteCard — карточку с длинным текстом (для проверки эллипсиса).

Контрольные вопросы

Ответьте письменно на следующие вопросы. Подготовьте устные ответы для защиты практической работы — преподаватель может задать любой из них или попросить пояснить код вашего решения.

1. Чем декларативный UI отличается от императивного? Приведите пример одной и той же задачи на View и на Compose.

2. Что такое recomposition? Когда Compose решает перерисовать Composable?

3. Какие правила должна соблюдать Composable-функция, чтобы recomposition работала корректно?

4. Зачем нужен remember? Чем он отличается от mutableStateOf?

5. Что делает derivedStateOf? Приведите пример, когда он необходим.

6. Почему порядок вызовов в цепочке Modifier важен? Приведите пример с padding и background.

7. Какие параметры у Column и за что они отвечают?

8. Зачем в LazyColumn нужен key? Что произойдёт, если его не указать?

9. Что такое *stability* в Compose? Какие типы stable по умолчанию, а какие — нет?

10. Как пометить класс как @Immutable и зачем это делать?

11. Чем Box отличается от Column? Какие у него есть параметры выравнивания?

12. Как сделать так, чтобы текст в Text обрезался многоточием после двух строк?

Чек-лист сдачи работы

Перед сдачей работы преподавателю убедитесь, что выполнены все пункты ниже. Отсутствие любого пункта снижает оценку минимум на один балл.

- □ В ui/NoteCard.kt есть @Composable fun NoteCard(note, onClick, modifier)
- □ NoteCard использует androidx.compose.material3.Card
- □ Цветной кружок 12 dp меняется по NoteType (#3592C0 / #3DDC84 / #FFA726)
- □ Title в одну строку с эллипсисом, summary в две строки
- □ Метка типа в правом нижнем углу, labelSmall
- □ Card обёрнут в Modifier.clickable
- □ В ui/NoteList.kt есть @Composable fun NoteList(notes, onClick, modifier)
- □ Используется LazyColumn с key = { it.id }
- □ contentPadding = PaddingValues(16.dp) и verticalArrangement = Arrangement.spacedBy(8.dp)
- □ При пустом списке показывается EmptyState
- □ В ui/EmptyState.kt реализован EmptyState(title, subtitle, modifier)
- □ Для каждой Composable-функции есть @Preview
- □ MainActivity отображает список из 5 заметок, клик логируется

Что сдавать

В результате выполнения практической работы студент предоставляет:

1. Git-коммит «Практика 2» с тремя новыми файлами в каталоге ui/;

2. Скриншот приложения со списком заметок и пустым состоянием;

3. Скриншот Android Studio с открытым @Preview для NoteCard.

Практика 3. Material 3 и MaterialTheme

Трудоёмкость: 6 академических часов. **Форма работы:** индивидуальная, в IDE Android Studio. **Форма отчёта:** исходный код проекта + устная защита.

Цель и задачи

Цель работы — освоить систему дизайна Material 3 (Material You) в Jetpack Compose: цветовые схемы, типографику, формы и базовые компоненты (TopAppBar, FAB, Scaffold, NavigationBar, Snackbar). Студент должен понять философию динамических цветов (Android 12+), научиться создавать кастомные темы и переключать светлую/тёмную схему.

1. объяснить иерархию Material 3: colorScheme → typography → shapes → components;
2. создать кастомную цветовую схему и подключить её через MaterialTheme;
3. использовать Scaffold с TopAppBar, FAB и Snackbar;
4. включить тёмную тему и динамические цвета (Android 12+);
5. применять Material 3 компоненты: Card, Button, IconButton, Switch, TextField.

Теоретическая часть

3.2.1. Material You и Material 3

Material Design 3 (Material You) — третья версия дизайн-системы Google, представленная в 2021 году вместе с Android 12. Главное нововведение — **динамические цвета**: система извлекает палитру из обоев пользователя и применяет её ко всему интерфейсу. Даже если динамические цвета недоступны (Android 11 и ниже или явное отключение), приложение должно использовать продуманную статическую цветовую схему. В Compose Material 3 живёт в артефакте androidx.compose.material3:material3 и подключается отдельно от Material 1 (старый androidx.compose.material).

3.2.2. ColorScheme — роли цветов

ColorScheme — это набор из ~30 цветовых ролей. Каждая роль имеет семантическое назначение и пару для светлой/тёмной темы. Главные роли:

- **primary / onPrimary** — основной акцент приложения и цвет текста/иконок на нём. Используется для FAB, активных кнопок.
- **secondary / onSecondary** — менее заметный акцент (фильтры, чипы).
- **tertiary / onTertiary** — контрастный третий цвет для баланса.
- **background / onBackground** — фон экрана и текст на нём.
- **surface / onSurface** — фон карточек и контент на них.
- **surfaceVariant / onSurfaceVariant** — приглушённая surface (второстепенные карточки).
- **error / onError** — ошибки и разрушительные действия.
- **outline** — границы, разделители, плейсхолдеры.

Каждую роль нужно брать из MaterialTheme.colorScheme.<role>, а не хардкодить HEX. Это гарантирует, что при переключении темы или динамических цветов интерфейс обновится автоматически.

3.2.3. Typography — типографическая шкала

Material 3 определяет 15 типографических ролей: 5 шкал (Display, Headline, Title, Body, Label) × 3 размера (Large, Medium, Small). Каждая роль — это TextStyle с параметрами fontFamily, fontWeight, fontSize, lineHeight, letterSpacing. По умолчанию используется системный шрифт (Roboto на AOSP), но его можно заменить на любой кастомный.

Роль	Размер (M)	Вес	Типичное применение
`displayLarge`	`57`	`400`	Огромные числа в пустых состояниях
`headlineLarge`	`32`	`400`	Заголовки экранов
`titleLarge`	`22`	`400`	TopAppBar заголовок
`titleMedium`	`16`	`500`	Заголовок карточки
`bodyLarge`	`16`	`400`	Основной текст
`bodyMedium`	`14`	`400`	Описание, подзаголовок
`labelLarge`	`14`	`500`	Текст кнопок
`labelSmall`	`11`	`500`	Метка типа, badge

3.2.4. Shapes — формы

Material 3 определяет 5 форм: extraSmall, small, medium, large, extraLarge. По умолчанию corner radius: 4, 8, 12, 16, 28 dp соответственно. Компоненты Material 3 используют эти роли: Card — medium, Button — full (полностью круглый), TextField — small. Кастомные Composable тоже должны брать формы из MaterialTheme.shapes, а не хардкодить значения.

3.2.5. Theme.kt — стандартная структура

Тема Compose Material 3 обычно состоит из трёх файлов в пакете ui.theme: Color.kt (HEX-коды цветов), Type.kt (Typography), Theme.kt (сборка всего в MaterialTheme). Шаблон Theme.kt:

KOTLIN

```
// ui/theme/Theme.kt
@Composable
fun NotesAppTheme(
darkTheme: Boolean = isSystemInDarkTheme(),
dynamicColor: Boolean = true,
content: @Composable () -> Unit,
) {
val context = LocalContext.current
val colorScheme = when {
dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S ->
if (darkTheme) dynamicDarkColorScheme(context)
else dynamicLightColorScheme(context)
darkTheme -> DarkColors
else -> LightColors
}
MaterialTheme(
colorScheme = colorScheme,
typography = NotesAppTypography,
shapes = NotesAppShapes,
content = content,
)
}
```

3.2.6. Динамические цвета (Android 12+)

Функции dynamicLightColorScheme(context) и dynamicDarkColorScheme(context) возвращают ColorScheme, извлечённый из системных обоев. На Android 11 и ниже нужно использовать статическую схему. Идиоматичный паттерн — предоставить пользователю переключатель «использовать динамические цвета» в настройках, а по умолчанию включить их на Android 12+.

3.2.7. Scaffold — каркас экрана

Scaffold — это Composable, который реализует базовый layout Material-экрана: TopAppBar, BottomAppBar, NavigationBar, FAB, Snackbar, Drawer. Он автоматически вычисляет отступы (innerPadding), которые нужно передать контенту. Scaffold обязателен для использования — он гарантирует соответствие Material гайдлайнам и корректную работу с системными жестами.

```kotlin
@Composable
fun NotesScreen() {
val snackbarHostState = remember { SnackbarHostState() }
val scope = rememberCoroutineScope()

Scaffold(
topBar = {
TopAppBar(
title = { Text("Заметки") },
actions = {
IconButton(onClick = { /* поиск */ }) {
Icon(Icons.Default.Search, contentDescription = "Поиск")
}
},
)
},
floatingActionButton = {
FloatingActionButton(onClick = {
scope.launch {
snackbarHostState.showSnackbar("Создание новой заметки")
}
}) {
Icon(Icons.Default.Add, contentDescription = "Добавить")
}
},
snackbarHost = { SnackbarHost(snackbarHostState) },
) { innerPadding ->
NoteList(notes = sampleNotes, onClick = {}, modifier = Modifier.padding(innerPadding))
}
}
```

3.2.8. Snackbar и SnackbarHost

Snackbar — это Material-аналог Toast с двумя преимуществами: показывается над контентом (не перекрывает системные области), может содержать кнопку действия. Для показа нужен SnackbarHostState — именно его метод showSnackbar(message, actionLabel) вызывается из корутины. Показ Snackbar — suspend-функция, её нужно запускать в rememberCoroutineScope или в ViewModel.

3.2.9. Тёмная тема

Тёмная тема активируется через системные настройки. В Compose функция isSystemInDarkTheme() возвращает текущее состояние. Идиоматично: передавать darkTheme как параметр в Theme, внутри которой выбирается нужная ColorScheme. Не хардкодите цвета в зависимости от темы — пусть этим занимается MaterialTheme. Если пользователю дать выбор («Системная / Светлая / Тёмная»), сохраняйте выбор в DataStore и читайте в Theme через StateFlow.

3.2.10. Material 3 компоненты — расширенный обзор

Material 3 содержит значительно больше компонентов, чем Material 2. Кроме уже рассмотренных Scaffold, TopAppBar, FAB, Snackbar, NavigationBar, в проекте будут полезны:

Компонент	Назначение	Ключевые параметры
`ModalNavigationDrawer`	Боковое меню (выезжает поверх)	drawerContent, gesturesEnabled

Компонент	Назначение	Ключевые параметры
`PermanentNavigationDrawer`	Боковое меню (всегда видно на планшетах)	drawerContent
`ModalBottomSheet`	Модальный лист снизу	onDismissRequest, sheetState
`DatePicker`	Выбор даты	state, showModeToggle
`TimePicker`	Выбор времени	state
`Slider`	Ползунок	value, onValueChange, valueRange
`RangeSlider`	Диапазонный ползунок	value (Pair), onValueChange
`SegmentedButton`	Сегментированные кнопки	SingleChoiceSegmentedButtonRow / MultiChoiceSegmentedButtonRow
`Card variants`	Elevated / Outlined / Filled	Через CardDefaults.elevatedCardColors() и т.п.
`AssistChip / FilterChip / InputChip / SuggestionChip`	Чипы разных типов	Зависит от типа
`Badge`	Бейдж с числом или точкой	badgeContent
`LinearProgressIndicator`	Линейный прогресс	progress (determinate) или без параметра (indeterminate)
`Divider / HorizontalDivider`	Разделитель	—

3.2.11. ModalBottomSheet — модальный лист

ModalBottomSheet — модальный лист, выезжающий снизу. Используется для коротких действий: выбор опции, быстрая форма. В отличие от диалога, лист можно смахнуть вниз и он не перекрывает весь экран.

```kotlin
@Composable
fun FilterBottomSheet(
onDismiss: () -> Unit,
onApply: (FilterOptions) -> Unit,
) {
val sheetState = rememberModalBottomSheetState()
ModalBottomSheet(
onDismissRequest = onDismiss,
sheetState = sheetState,
) {
Column(modifier = Modifier.padding(16.dp)) {
Text("Фильтры", style = MaterialTheme.typography.titleLarge)
Spacer(Modifier.height(16.dp))
// ... содержимое фильтров
Button(
onClick = { onApply(currentFilter) },
modifier = Modifier.fillMaxWidth(),
) { Text("Применить") }
}
}
}
```

3.2.12. NavigationDrawer — боковое меню

ModalNavigationDrawer выезжает поверх контента, PermanentNavigationDrawer — всегда видим на планшетах. Содержимое drawer-а обычно строится через ModalDrawerSheet с NavigationDrawerItem.

```kotlin
@Composable
fun AppDrawer(currentRoute: String, onNavigate: (String) -> Unit) {
ModalNavigationDrawer(
drawerContent = {
ModalDrawerSheet {
Text("NotesApp", modifier = Modifier.padding(16.dp),
style = MaterialTheme.typography.titleMedium)
HorizontalDivider()
NavigationDrawerItem(
label = { Text("Заметки") },
selected = currentRoute == "notes",
onClick = { onNavigate("notes") },
icon = { Icon(Icons.Default.Note, null) },
)
NavigationDrawerItem(
label = { Text("Настройки") },
selected = currentRoute == "settings",
onClick = { onNavigate("settings") },
icon = { Icon(Icons.Default.Settings, null) },
)
}
},
) {
// Контент экрана
AppContent()
}
}
```

3.2.13. Adaptive layouts — WindowSizeClass и List-Detail

Material 3 рекомендует адаптивные layout-ы для разных размеров экрана. Библиотека androidx.compose.material3:material3-window-size-class предоставляет WindowSizeClass — Compact / Medium / Expanded по ширине и высоте. На Compact (телефон портрет) — одна колонка, на Expanded (планшет) — две. Паттерн List-Detail: слева список, справа детали выбранного элемента.

KOTLIN

```
@Composable
fun AdaptiveNotesScreen(windowSizeClass: WindowSizeClass) {
when (windowSizeClass.widthSizeClass) {
WindowWidthSizeClass.Compact -> {
// Телефон: одна колонка, переход по клику
NavHost(...) {
composable("list") { NotesListScreen(onNoteClick = { navController.navigate("detail/$it")
}) }
composable("detail") { NoteDetailScreen() }
}
}
WindowWidthSizeClass.Expanded -> {
// Планшет: две колонки рядом
Row(modifier = Modifier.fillMaxSize()) {
NotesListScreen(
modifier = Modifier.weight(1f),
onNoteClick = { id -> selectedId = id },
)
NoteDetailScreen(
noteId = selectedId,
modifier = Modifier.weight(2f),
)
}
}
else -> { /* Medium — на ваше усмотрение */ }
}
}
```

3.2.14. Downloadable fonts через Google Fonts

Material 3 позволяет загружать шрифты из Google Fonts на лету через библиотеку androidx.compose.ui:ui-text-google-fonts. Это уменьшает размер APK (шрифты не вшиты) и позволяет менять шрифт без перекомиляции. Шрифт кэшируется на устройстве после первой загрузки.

```kotlin
// ui/theme/Type.kt
import androidx.compose.ui.text.googlefonts.GoogleFont
import androidx.compose.ui.text.googlefonts.GoogleFontProvider

val provider = GoogleFont.Provider(
providerAuthority = "com.google.android.gms.fonts",
providerPackage = "com.google.android.gms",
certificates = R.array.com_google_android_gms_fonts_certs,
)

val InterFont = FontFamily(
Font(googleFont = GoogleFont("Inter"), fontProvider = provider),
Font(googleFont = GoogleFont("Inter"), fontProvider = provider, weight = FontWeight.Bold),
)

val NotesAppTypography = Typography(
bodyLarge = TextStyle(fontFamily = InterFont, fontSize = 16.sp),
titleLarge = TextStyle(fontFamily = InterFont, fontSize = 22.sp, fontWeight =
FontWeight.Bold),
// ...
)
```

Для шрифтов нужен массив сертификатов провайдера — он есть в документации Compose. Альтернатива: вшить шрифт в res/font/ как TTF — это увеличивает APK, но работает офлайн.

3.2.15. Material 3 Expressive — новый этап Material

С 2024-2025 Google развивает **Material 3 Expressive** — расширение Material 3 с более свободным подходом к цвету, типографике и motion. Главные нововведения: более выразительная motion-система, расширенная типографическая шкала, контентно-зависимые цвета. На момент написания практикума Material 3 Expressive доступен в экспериментальных версиях androidx.compose.material3 — для production рекомендуется stable Material 3, который мы и используем.

Примеры кода

Пример 3.1. Кастомная цветовая схема

KOTLIN

```
// ui/theme/Color.kt
import androidx.compose.material3.darkColorScheme
import androidx.compose.material3.lightColorScheme
import androidx.compose.ui.graphics.Color

private val PastelPrimary = Color(0xFF3592C0)
private val PastelOnPrimary = Color(0xFFFFFFFF)
private val PastelSecondary = Color(0xFF7BC4DB)
private val PastelBackground = Color(0xFFF7FBFD)
private val PastelSurface = Color(0xFFFFFFFF)
private val PastelOnBackground = Color(0xFF1A1C1E)
private val PastelOnSurface = Color(0xFF1A1C1E)
private val PastelOutline = Color(0xFF7B8388)

val PastelLightScheme = lightColorScheme(
primary = PastelPrimary, onPrimary = PastelOnPrimary,
secondary = PastelSecondary,
background = PastelBackground, onBackground = PastelOnBackground,
surface = PastelSurface, onSurface = PastelOnSurface,
outline = PastelOutline,
)

val PastelDarkScheme = darkColorScheme(
primary = Color(0xFF8FD0E8),
onPrimary = Color(0xFF003444),
secondary = Color(0xFFB3CCD9),
background = Color(0xFF111417),
onBackground = Color(0xFFE2E5E8),
surface = Color(0xFF1A1F23),
onSurface = Color(0xFFE2E5E8),
outline = Color(0xFF8B9498),
)
```

Пример 3.2. Scaffold с FAB и Snackbar

KOTLIN

```
@Composable
fun CounterWithScaffold() {
var count by remember { mutableStateOf(0) }
val snackbarHostState = remember { SnackbarHostState() }
val scope = rememberCoroutineScope()

Scaffold(
topBar = {
TopAppBar(
title = { Text("Счётчик") },
colors = TopAppBarDefaults.topAppBarColors(
containerColor = MaterialTheme.colorScheme.primaryContainer,
titleContentColor = MaterialTheme.colorScheme.onPrimaryContainer,
),
)
},
floatingActionButton = {
FloatingActionButton(
onClick = {
count++
if (count % 5 == 0) {
scope.launch {
snackbarHostState.showSnackbar(
message = "Достигнуто $count!",
actionLabel = "Сбросить",
)
}
}
},
) { Icon(Icons.Default.Add, contentDescription = null) }
},
snackbarHost = { SnackbarHost(snackbarHostState) },
) { inner ->
Box(
modifier = Modifier.padding(inner).fillMaxSize(),
contentAlignment = Alignment.Center,
) {
Text("$count", style = MaterialTheme.typography.displayLarge)
}
}
}
```

Задание

В проекте **NotesApp** реализуйте Material 3 тему с двумя статическими схемами (Pastel и Forest), переключателем между ними и поддержкой тёмной темы. Замените «голый» NoteList на полноценный экран со Scaffold, TopAppBar, FAB и Snackbar. Все цвета и формы должны браться из MaterialTheme, а не хардкодиться.

Шаг 1. Файл Color.kt

В ui/theme/Color.kt опишите две цветовых схемы (по 8 ролей в каждой: primary, onPrimary, secondary, background, onBackground, surface, onSurface, outline). Должны получиться четыре объекта ColorScheme:

1. PastelLightScheme — основной #3592C0, фон #F7FBFD
2. PastelDarkScheme — основной #8FD0E8, фон #111417

3. ForestLightScheme — основной #2D6B4A, фон #F2F8F4

4. ForestDarkScheme — основной #6FB289, фон #0F1813

Шаг 2. Файл Type.kt

В ui/theme/Type.kt опишите кастомную Typography на основе системного шрифта, но с увеличенными размерами для titleLarge (24 dp вместо 22) и bodyLarge (17 dp вместо 16). Используйте функцию Typography() с именованными параметрами.

Шаг 3. Файл Theme.kt

В ui/theme/Theme.kt создайте Composable-функцию NotesAppTheme с параметрами:

```kotlin
@Composable
fun NotesAppTheme(
darkTheme: Boolean = isSystemInDarkTheme(),
themeVariant: ThemeVariant = ThemeVariant.PASTEL,
dynamicColor: Boolean = false, // по умолчанию off — пусть будут статические
content: @Composable () -> Unit,
)

enum class ThemeVariant { PASTEL, FOREST }
```

Логика выбора схемы:

- если dynamicColor && Build.VERSION.SDK_INT >= S — динамическая схема;
- иначе — Pastel/Forest × Light/Dark по параметрам.

Шаг 4. Экран NotesListScreen

В ui/NotesListScreen.kt создайте Composable со следующей структурой:

1. Scaffold с topBar, floatingActionButton, snackbarHost.

2. TopAppBar: заголовок «Заметки», action — иконка поиска (Icons.Default.Search) с contentDescription = "Поиск".

3. FAB: Icons.Default.Add, по клику показывает Snackbar «Создание новой заметки» с actionLabel «Отмена».

4. Контент — NoteList из практики 2, обёрнутый в Modifier.padding(innerPadding).

5. Snackbar с действием «Отмена» логирует в Logcat TAG "NotesApp" сообщение «Action: ОТМЕНА».

Шаг 5. Тёмная тема

Проверьте, что при переключении системы в тёмную тему приложение корректно меняет цвета. Если в NoteCard цвета типов (Text=синий, Image=зелёный, Audio=оранжевый) были захардкожены в практике 2 — оставьте их как есть (это акцентные цвета, которые не зависят от темы), но фон карточки должен браться из MaterialTheme.colorScheme.surface.

Шаг 6. Превью для двух тем

Создайте два @Preview для NotesListScreen: один с NotesAppTheme(darkTheme = false), второй с darkTheme = true. Также по одному preview для каждой ThemeVariant (Pastel и Forest).

Контрольные вопросы

Ответьте письменно на следующие вопросы. Подготовьте устные ответы для защиты практической работы — преподаватель может задать любой из них или попросить пояснить код вашего решения.

1. Что такое Material You? Чем Material 3 отличается от Material 2?

2. Перечислите минимум 5 ролей цветов в ColorScheme и их назначение.

3. Почему нельзя хардкодить цвета в Composable, а нужно брать из MaterialTheme.colorScheme?

4. Какие 5 шкал типографики определяет Material 3? Назовите по одной роли из каждой шкалы.

5. Что такое динамические цвета? На каких версиях Android они доступны? Как их включить?

6. Зачем нужен Scaffold? Какие слоты он предоставляет?

7. Чем Snackbar отличается от Toast? Почему Snackbar предпочтительнее?

8. Как показать Snackbar с кнопкой действия? Где хранится состояние?

9. Как переключить тему в тёмную/светлую в зависимости от системы?

10. Какие параметры имеет TopAppBar в Material 3? Чем отличается от MediumTopAppBar?

11. Что такое innerPadding в Scaffold и зачем его нужно передавать в контент?

12. Как создать кастомную Typography? Можно ли использовать кастомный шрифт (например, Roboto Flex)?

Чек-лист сдачи работы

Перед сдачей работы преподавателю убедитесь, что выполнены все пункты ниже. Отсутствие любого пункта снижает оценку минимум на один балл.

- □ В ui/theme/Color.kt есть 4 ColorScheme-объекта с указанными HEX
- □ В ui/theme/Type.kt кастомная Typography с titleLarge=24, bodyLarge=17
- □ В ui/theme/Theme.kt есть enum class ThemeVariant { PASTEL, FOREST }
- □ NotesAppTheme принимает параметры: darkTheme, themeVariant, dynamicColor, content
- □ Логика выбора схемы: dynamicColor → else if darkTheme → else lightTheme
- □ В ui/NotesListScreen.kt используется Scaffold с TopAppBar, FAB, SnackbarHost
- □ TopAppBar содержит заголовок «Заметки» и action-иконку Search
- □ FAB по клику показывает Snackbar с actionLabel «Отмена»
- □ innerPadding из Scaffold передаётся в NoteList через Modifier.padding
- □ Созданы 4 @Preview: light/dark × Pastel/Forest
- □ При переключении системы в тёмную тему цвета меняются корректно
- □ В Logcat при нажатии action в Snackbar выводится «Action: ОТМЕНА»

Что сдавать

В результате выполнения практической работы студент предоставляет:

1. Git-коммит «Практика 3» с обновлёнными Color/Type/Theme.kt и новым NotesListScreen.kt;

2. Скриншот приложения в светлой и тёмной теме (Pastel);

3. Скриншот Snackbar с кнопкой «Отмена».

Практика 4. Layouts и навигация в Compose

Трудоёмкость: 6 академических часов. **Форма работы:** индивидуальная, в IDE Android Studio. **Форма отчёта:** исходный код проекта + устная защита.

Цель и задачи

Цель работы — освоить композицию сложных layout-ов в Jetpack Compose и научиться реализовывать многоэкранную навигацию с помощью библиотеки Navigation-Compose. Студент должен уметь описать граф переходов, передавать аргументы между экранами, обрабатывать кнопку «Назад» и использовать Bottom Navigation для переключения между верхними экранами.

1. использовать Box, Row, Column, FlowRow, BoxWithConstraints для построения адаптивных layout-ов;
2. определять маршруты и аргументы в NavHost;
3. использовать rememberNavController() для программных переходов;
4. строить Bottom Navigation с переключением экранов;
5. обрабатывать системную кнопку Back.

Теоретическая часть

4.2.1. Стандартные layout-ы Compose

Compose предлагает несколько layout-контейнеров с разной семантикой. Выбор контейнера определяется тем, как должны располагаться дочерние элементы по двум осям. Главные контейнеры:

Контейнер	Поведение	Аналог в View
`Column`	Вертикальный стек	`LinearLayout vertical`
`Row`	Горизонтальный стек	`LinearLayout horizontal`
`Box`	Наложение (z-stack)	`FrameLayout / ConstraintLayout`
`FlowRow`	Row с переносом на новую строку	`FlexboxLayout`
`FlowColumn`	Column с переносом в новую колонку	—
`BoxWithConstraints`	Box с доступом к доступным размерам	`OnMeasure`
`LazyColumn`	Вертикальный список с рециклингом	`RecyclerView vertical`

Контейнер	Поведение	Аналог в View
`LazyRow`	Горизонтальный список	`RecyclerView horizontal`
`LazyVerticalGrid`	Сетка	`GridLayoutManager`
`ConstraintLayout`	Constraint-решения	`ConstraintLayout`

4.2.2. BoxWithConstraints — адаптивный layout

BoxWithConstraints даёт доступ к доступным размерам (maxWidth, maxHeight) и позволяет выбирать разный layout в зависимости от них. Это ключевой инструмент для адаптивных интерфейсов: на телефоне — одна колонка, на планшете — две. Внутри лямбды доступен BoxScope, но дополнительно есть this.maxWidth и minWidth как Dp и как Constraints.

KOTLIN

```
import androidx.compose.foundation.layout.BoxWithConstraints
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row

@Composable
fun AdaptiveDetail(note: Note) {
BoxWithConstraints {
if (maxWidth < 600.dp) {
// Телефон: вертикальная раскладка
Column { NoteHeader(note); NoteBody(note) }
} else {
// Планшет: горизонтальная с делением 1:2
Row {
NoteHeader(note, modifier = Modifier.weight(1f))
NoteBody (note, modifier = Modifier.weight(2f))
}
}
}
}
```

4.2.3. WindowSizeClass — адаптивность Material 3

Библиотека androidx.compose.material3:material3-window-size-class предоставляет функцию calculateWindowSizeClass(activity), которая возвращает WindowWidthSizeClass (Compact / Medium / Expanded) и WindowHeightSizeClass. Границы: Compact < 600 dp, Medium 600–840 dp, Expanded > 840 dp. Это стандартизированный способ описать breakpoint-ы и принять решение о layout-е.

4.2.4. Navigation-Compose: основы

Navigation-Compose — библиотека androidx.navigation:navigation-compose, позволяющая описать граф переходов между Composable-экранами. Главные компоненты:

- **NavController** — управляет стеком back-stack. Создаётся через rememberNavController().
- **NavHost** — контейнер, который хранит граф переходов и отображает текущий destination.
- **composable(route)** — описывает один экран в графе. route — уникальная строка-маршрут.
- **navController.navigate(route)** — программный переход. Можно указать launchSingleTop = true, popUpTo(...).

• **navController.popBackStack()** — вернуться на предыдущий экран.

4.2.5. Маршруты и аргументы

Маршрут — это строка, возможно с placeholder-ами для аргументов. Например, "note/{id}". Аргументы объявляются в composable() через arguments — список namedArgument, где указывается имя, тип и значение по умолчанию. В Composable-функции аргумент достаётся через backStackEntry.arguments?.getLong("id") или через toRoute() (с Navigation 2.8+ и type-safe навигацией).

```kotlin
NavHost(
navController = navController,
startDestination = "notes",
) {
composable("notes") {
NotesListScreen(
onNoteClick = { note ->
navController.navigate("note/${note.id}")
},
)
}
composable(
route = "note/{id}",
arguments = listOf(navArgument("id") { type = NavType.LongType }),
) { backStackEntry ->
val noteId = backStackEntry.arguments?.getLong("id") ?: -1L
NoteDetailScreen(noteId = noteId, onBack = { navController.popBackStack() })
}
composable("settings") {
SettingsScreen()
}
}
```

4.2.6. Bottom Navigation

Bottom Navigation в Material 3 реализуется через NavigationBar + NavigationBarItem. Чтобы она переключала экраны, её размещают в Scaffold через параметр bottomBar, а текущий пункт определяют по navController.currentBackStackEntryAsState(). При клике вызывается navController.navigate(route) с опциями launchSingleTop = true и restoreState = true, чтобы не плодить копии экранов в стеке.

```kotlin
@Composable
fun BottomBar(navController: NavController) {
val currentBackStackEntry by navController.currentBackStackEntryAsState()
val currentRoute = currentBackStackEntry?.destination?.route

NavigationBar {
bottomItems.forEach { item ->
NavigationBarItem(
selected = currentRoute == item.route,
onClick = {
navController.navigate(item.route) {
popUpTo(navController.graph.findStartDestination().id) { saveState = true }
launchSingleTop = true
restoreState = true
}
},
icon = { Icon(item.icon, contentDescription = item.title) },
label = { Text(item.title) },
)
}
}
}
```

4.2.7. Type-safe навигация (Navigation 2.8+)

С Kotlin 2.0+ и Navigation 2.8+ появилась type-safe навигация: вместо строковых маршрутов можно использовать @Serializable объекты или data классы. Компилятор проверяет типы аргументов на этапе компиляции — если аргумент не совпадает, будет ошибка сборки, а не crash в рантайме. Это особенно полезно для сложных графов навигации.

KOTLIN

```
import kotlinx.serialization.Serializable
import androidx.navigation.compose.composable
import androidx.navigation.NavType
import androidx.navigation.toRoute

// Маршруты как @Serializable классы/объекты
@Serializable object Notes
@Serializable data class NoteDetail(val id: Long)
@Serializable object Settings

@Composable
fun AppNavHost() {
val navController = rememberNavController()
NavHost(navController, startDestination = Notes) {
composable<Notes> {
NotesListScreen(
onNoteClick = { note -> navController.navigate(NoteDetail(note.id)) },
)
}
composable<NoteDetail> { backStackEntry ->
val args: NoteDetail = backStackEntry.toRoute()
NoteDetailScreen(noteId = args.id, onBack = { navController.popBackStack() })
}
composable<Settings> {
SettingsScreen()
}
}
}
```

Преимущества type-safe: 1) IDE показывает опечатки сразу; 2) рефакторинг переименований работает автоматически; 3) типы аргументов проверяются компилятором (Long вместо String). Недостаток: нужно использовать kotlinx.serialization. На этой практике можно остаться на строковых маршрутах для простоты, но в финальном проекте (практика 14) рекомендуется type-safe.

4.2.8. Глубокие ссылки (deep links)

Navigation-Compose поддерживает deep links — внешние URI, которые ведут на конкретный экран. Объявляются через composable(..., deepLinks = listOf(navDeepLink { uriPattern = "notesapp://note/{id}" })). При переходе по deep link система Android открывает приложение и автоматически подставляет аргументы. Глубокие ссылки полезны для уведомлений и шаринга.

KOTLIN

```
NavHost(...) {
composable(
route = "note/{id}",
arguments = listOf(navArgument("id") { type = NavType.LongType }),
deepLinks = listOf(navDeepLink {
uriPattern = "https://notesapp.example.com/note/{id}"
// или кастомная схема: "notesapp://note/{id}"
}),
) { entry ->
val id = entry.arguments?.getLong("id") ?: -1L
NoteDetailScreen(noteId = id, onBack = { navController.popBackStack() })
}
}
```

Чтобы приложение реагировало на deep link, нужно зарегистрировать intent-filter в AndroidManifest. Для URL вида https://notesapp.example.com/note/{id} добавляется <intent-filter> с android:scheme="https", android:host="notesapp.example.com", android:pathPrefix="/note/". App Links (verified links) требуют ещё размещения assetlinks.json на сервере.

XML

```
<!-- AndroidManifest.xml -->
<activity android:name=".MainActivity" android:exported="true">
<intent-filter android:autoVerify="true">
<action android:name="android.intent.action.VIEW" />
<category android:name="android.intent.category.DEFAULT" />
<category android:name="android.intent.category.BROWSABLE" />
<data android:scheme="https"
android:host="notesapp.example.com"
android:pathPrefix="/note/" />
</intent-filter>
<!-- Для кастомной схемы -->
<intent-filter>
<action android:name="android.intent.action.VIEW" />
<category android:name="android.intent.category.DEFAULT" />
<data android:scheme="notesapp" android:host="note" />
</intent-filter>
</activity>
```

4.2.9. Вложенные навигационные графы (nested nav graphs)

Для сложных приложений с несколькими независимыми потоками (например, onboarding, основной flow, профиль) удобно разбить навигацию на вложенные графы. Каждый вложенный граф имеет свой route и может быть показан/скрыт целиком. Например, граф «auth» содержит логин/регистрацию; после успешного входа приложение popUpTo(auth) { inclusive = true } — весь граф удаляется из back stack.

KOTLIN

```
NavHost(navController, startDestination = "auth") {
// Вложенный граф аутентификации
navigation(startDestination = "login", route = "auth") {
composable("login") { LoginScreen(onSuccess = { navController.navigate("main") {
popUpTo("auth") { inclusive = true } } }) }
composable("register") { RegisterScreen() }
composable("forgot") { ForgotPasswordScreen() }
}

// Основной граф после входа
navigation(startDestination = "notes", route = "main") {
composable("notes") { NotesListScreen(...) }
composable("note/{id}") { NoteDetailScreen(...) }
composable("settings") { SettingsScreen() }
}
}
```

4.2.10. Условная навигация (conditional navigation)

Часто нужно решать, какой экран показать, в рантайме — например, в зависимости от того, залогинен ли пользователь. Решение: вынести логику в LaunchEffect при старте, и через navController.navigate перейти на нужный экран. Альтернатива — использовать popUpTo чтобы очистить back stack.

```kotlin
@Composable
fun RootNavHost(isAuthenticated: Boolean) {
val navController = rememberNavController()
val startDestination = if (isAuthenticated) "main" else "auth"

NavHost(navController, startDestination = startDestination) {
// ... графы
}

// Реакция на изменение состояния аутентификации
LaunchedEffect(isAuthenticated) {
if (isAuthenticated) {
navController.navigate("main") {
popUpTo("auth") { inclusive = true }
}
} else {
navController.navigate("auth") {
popUpTo("main") { inclusive = true }
}
}
}
}
```

4.2.11. Кастомные переходы (transitions)

Navigation-Compose позволяет настраивать анимации переходов через параметры enterTransition, exitTransition, popEnterTransition, popExitTransition на уровне composable() или всего NavHost. Анимации строятся через AnimatedContentTransitionScope и slideIntoContainer / fadeIn / scaleIn.

```kotlin
NavHost(
navController = navController,
startDestination = "notes",
// Глобальные переходы
enterTransition = { fadeIn(animationSpec = tween(300)) },
exitTransition = { fadeOut(animationSpec = tween(300)) },
) {
composable(
route = "note/{id}",
// Локальный переход для этого экрана
enterTransition = {
slideIntoContainer(
towards = AnimatedContentTransitionScope.SlideDirection.Start,
animationSpec = tween(300),
)
},
popExitTransition = {
slideOutOfContainer(
towards = AnimatedContentTransitionScope.SlideDirection.End,
animationSpec = tween(300),
)
},
) {
NoteDetailScreen(...)
}
}
```

4.2.12. Сохранение состояния при повороте

NavController автоматически сохраняет back stack и аргументы при смене конфигурации (поворот экрана). Однако состояние UI-элементов (scroll position, текст в TextField) нужно сохранять вручную через rememberSaveable. NavController также поддерживает SaveableStateHolder для сохранения состояния каждого экрана при переходах между ними — но в большинстве случаев достаточно rememberSaveable.

```kotlin
@Composable
fun NotesListScreen() {
// Сохраняется при повороте
var searchQuery by rememberSaveable { mutableStateOf("") }
val listState = rememberLazyListState() // тоже автоматически сохраняется

Column {
TextField(value = searchQuery, onValueChange = { searchQuery = it })
LazyColumn(state = listState) { /* ... */ }
}
}
```

4.2.13. Bottom Sheet навигация

В Material 3 можно использовать ModalBottomSheet как полноценный destination через bottomSheet() вместо composable(). Это удобно для модальных экранов: фильтры, выбор опции, быстрое редактирование. Sheet можно смахнуть вниз или нажать Back — NavController сам обработает это.

```kotlin
NavHost(navController, startDestination = "notes") {
composable("notes") { NotesListScreen(...) }
bottomSheet("filters") {
FilterBottomSheet(
onApply = { /* ... */ navController.popBackStack() },
onDismiss = { navController.popBackStack() },
)
}
}

// Вызов:
navController.navigate("filters")
```

4.2.14. Несколько NavHost — когда это оправдано

В большинстве случаев один NavHost на уровне Activity достаточен. Но иногда удобно иметь несколько — например, отдельный NavHost для bottom sheet-ов (через bottomSheet destination) или для отдельной колонки на планшете (List-Detail pattern). Главный нюанс: каждый NavHost имеет свой back stack и свой NavController — между ними нет автоматической связи. В финальном проекте мы остаёмся на одном NavHost.

Примеры кода

Пример 4.1. Минимальный NavHost с тремя экранами

KOTLIN

```
@Composable
fun AppNavHost() {
val navController = rememberNavController()
NavHost(navController = navController, startDestination = "home") {
composable("home") {
HomeScreen(
onOpenDetails = { id -> navController.navigate("details/$id") },
onOpenSettings = { navController.navigate("settings") },
)
}
composable(
route = "details/{id}",
arguments = listOf(navArgument("id") { type = NavType.StringType }),
) { entry ->
DetailsScreen(
id = entry.arguments?.getString("id").orEmpty(),
onBack = { navController.popBackStack() },
)
}
composable("settings") {
SettingsScreen(onBack = { navController.popBackStack() })
}
}
}
```

Пример 4.2. Bottom Navigation + Scaffold

KOTLIN

```
data class BottomItem(val route: String, val title: String, val icon: ImageVector)

private val bottomItems = listOf(
BottomItem("notes", "Заметки", Icons.Default.Note),
BottomItem("settings", "Настройки", Icons.Default.Settings),
)

@Composable
fun RootScreen() {
val navController = rememberNavController()
val currentBackStackEntry by navController.currentBackStackEntryAsState()
val currentRoute = currentBackStackEntry?.destination?.route

Scaffold(
bottomBar = {
if (currentRoute in bottomItems.map { it.route }) {
NavigationBar {
bottomItems.forEach { item ->
NavigationBarItem(
selected = currentRoute == item.route,
onClick = {
navController.navigate(item.route) {
popUpTo(navController.graph.findStartDestination().id) { saveState = true }
launchSingleTop = true
restoreState = true
}
},
icon = { Icon(item.icon, contentDescription = item.title) },
label = { Text(item.title) },
)
}
}
}
},
) { innerPadding ->
NavHost(
navController = navController,
startDestination = "notes",
modifier = Modifier.padding(innerPadding),
) {
composable("notes") {
NotesListScreen(onNoteClick = { note ->
navController.navigate("note/${note.id}")
})
}
composable(
route = "note/{id}",
arguments = listOf(navArgument("id") { type = NavType.LongType }),
) { entry ->
val id = entry.arguments?.getLong("id") ?: -1L
NoteDetailScreen(noteId = id, onBack = { navController.popBackStack() })
}
composable("settings") {
SettingsScreen()
}
}
}
}
```

Задание

В проекте **NotesApp** реализуйте три экрана и навигацию между ними. Экраны: **NotesListScreen** (список заметок), **NoteDetailScreen** (детали одной заметки), **SettingsScreen** (настройки темы). Bottom Navigation переключает между «Заметками» и «Настройками». Клик по карточке заметки открывает экран деталей с передачей id.

Шаг 1. Маршруты и sealed class Routes

В файле ui/navigation/Routes.kt опишите маршруты:

```kotlin
object Routes {
const val NOTES = "notes"
const val NOTE = "note/{id}"
const val SETTINGS = "settings"

fun note(id: Long) = "note/$id"
}
```

Шаг 2. NoteDetailScreen

В файле ui/NoteDetailScreen.kt создайте Composable:

```kotlin
@Composable
fun NoteDetailScreen(
noteId: Long,
onBack: () -> Unit,
)
```

Требования:

1. TopAppBar с кнопкой-стрелкой «Назад» (Icons.AutoMirrored.Filled.ArrowBack) и заголовком «Заметка #*id*».

2. На этом этапе данные можно захардкодить: найти заметку с указанным id в статическом списке из 5 заметок (тот же, что в практике 2). Если id не найден — показать EmptyState «Заметка не найдена».

3. В контенте: заголовок (titleLarge), тип заметки (labelMedium, цвет по типу), дата создания в формате «dd.MM.yyyy HH:mm», полный текст заметки (bodyLarge).

4. Дата форматируется через SimpleDateFormat или DateTimeFormatter — на ваш выбор.

Шаг 3. SettingsScreen

В файле ui/SettingsScreen.kt создайте Composable:

```kotlin
@Composable
fun SettingsScreen(
onThemeChange: (ThemeVariant) -> Unit,
currentTheme: ThemeVariant,
)
```

Требования:

1. TopAppBar с заголовком «Настройки».

2. Секция «Тема приложения» с двумя RadioButton: Pastel и Forest.

3. Секция «Тёмная тема» с Switch (пока просто переключатель, значение можно хранить в remember — на практике 6 перенесём в ViewModel).

4. Секция «О приложении» с текстом версии: «NotesApp v0.4.0 (Практика 4)».

Шаг 4. RootScreen с NavHost и BottomBar

В файле ui/RootScreen.kt создайте Composable RootScreen(), который содержит Scaffold с BottomBar и NavHost с тремя экранами. Используйте шаблон из примера 4.2.

1. BottomBar показывается только на маршрутах notes и settings. На note/{id} BottomBar скрыт.

2. На notes клик по NoteCard вызывает onNoteClick → navController.navigate(Routes.note(note.id)).

3. На note/{id} кнопка «Назад» вызывает navController.popBackStack().

4. Системная кнопка Back должна работать (NavController обрабатывает это автоматически — убедитесь, что ничего не переопределяет её).

Шаг 5. Подключение в MainActivity

В MainActivity.setContent оберните RootScreen() в NotesAppTheme. Вся навигация теперь управляется из RootScreen.

Шаг 6. Превью

Создайте @Preview для каждого экрана: NotesListScreen, NoteDetailScreen (с захардкоженным noteId=1), SettingsScreen. Также @Preview для RootScreen.

Контрольные вопросы

Ответьте письменно на следующие вопросы. Подготовьте устные ответы для защиты практической работы — преподаватель может задать любой из них или попросить пояснить код вашего решения.

1. Чем Box отличается от Column? Когда выбрать Box?

2. Что делает BoxWithConstraints? Приведите пример адаптивной раскладки.

3. Что такое WindowSizeClass? Какие значения может принимать?

4. Зачем нужен NavController? Можно ли создать несколько NavHost в одном приложении?

5. Как передать аргумент типа Long между экранами? Какие типы аргументов поддерживает Navigation?

6. Как работает popUpTo с saveState = true? Зачем это нужно в Bottom Navigation?

7. Чем launchSingleTop = true отличается от поведения по умолчанию?

8. Как скрыть BottomBar на определённом экране? Где хранить информацию о видимости?

9. Как обработать системную кнопку Back? Что произойдёт, если не обрабатывать её вручную?

10. Что такое deep link в Navigation-Compose? Как объявить deep link на маршрут «note/{id}»?

11. Чем type-safe навигация (Navigation 2.8+) отличается от строковых маршрутов? В каких случаях стоит её использовать?

12. Как сделать анимацию перехода между экранами в Navigation-Compose?

Чек-лист сдачи работы

Перед сдачей работы преподавателю убедитесь, что выполнены все пункты ниже. Отсутствие любого пункта снижает оценку минимум на один балл.

- □ В ui/navigation/Routes.kt есть объект Routes с тремя константами и функцией note(id)
- □ В ui/NoteDetailScreen.kt есть Composable с сигнатурой fun NoteDetailScreen(noteId: Long, onBack: () -> Unit)
- □ TopAppBar содержит кнопку-стрелку Back и заголовок «Заметка #id»
- □ При невалидном id показывается EmptyState «Заметка не найдена»
- □ Дата форматируется в «dd.MM.yyyy HH:mm»
- □ В ui/SettingsScreen.kt есть Composable с сигнатурой fun SettingsScreen(onThemeChange, currentTheme)
- □ SettingsScreen содержит RadioButton для Pastel и Forest
- □ SettingsScreen содержит Switch для тёмной темы и текст версии
- □ В ui/RootScreen.kt есть Scaffold + BottomBar + NavHost с тремя маршрутами
- □ BottomBar показывается только на notes и settings
- □ Системная кнопка Back работает корректно на всех экранах
- □ MainActivity вызывает RootScreen внутри NotesAppTheme
- □ Созданы 4 @Preview: для NotesListScreen, NoteDetailScreen, SettingsScreen, RootScreen

Что сдавать

В результате выполнения практической работы студент предоставляет:

1. Git-коммит «Практика 4» с тремя новыми экранами и Routes.kt;
2. Скриншот NotesListScreen с BottomBar;
3. Скриншот NoteDetailScreen с кнопкой Back и форматированной датой;
4. Скриншот SettingsScreen с RadioButton для выбора темы.

Практика 5. Coroutines и Flow: основы

Трудоёмкость: 6 академических часов. **Форма работы:** индивидуальная, в IDE Android Studio. **Форма отчёта:** исходный код проекта + устная защита.

Цель и задачи

Цель работы — заложить фундамент асинхронного программирования в Kotlin, прежде чем использовать его в ViewModel (практика 6), Room (практика 7) и Retrofit (практика 8). На предыдущих практиках корутины если и появлялись, то как «магические» viewModelScope.launch и suspend-методы. Теперь студент разберётся, как они устроены изнутри, научится управлять областями видимости, диспетчерами и обработкой ошибок, а также познакомится с Flow — реактивным потоком значений, на котором построен весь современный Android-стек.

1. объяснять, что такое корутина и чем она отличается от потока ОС;
2. применять **structured concurrency** — иерархию корутин и scope-ов;
3. правильно выбирать Dispatcher (Main, IO, Default, Unconfined);
4. запускать параллельные корутины через async/await;
5. обрабатывать отмену и исключения: Job, SupervisorJob, coroutineScope, supervisorScope;
6. объяснять, что такое suspend-функция и как компилятор превращает её в state machine;
7. создавать и собирать Flow через flow { emit(...) } и collect;
8. различать **cold Flow** и **hot StateFlow/SharedFlow**;
9. применять базовые операторы: map, filter, onEach, flowOn, catch, retry;
10. тестировать корутины через runTest и TestDispatcher.

Теоретическая часть

5.2.1. Что такое корутина

Корутина (co-operative routine) — это «лёгкий поток», приостанавливаемый посреди выполнения через suspend-функции. В отличие от потоков ОС, корутины не требуют переключения контекста ядра и занимают около 100 байт памяти каждая. На одном потоке можно запускать тысячи корутин. В Kotlin корутины реализованы через компилятор (генерирует state machine) и библиотеку kotlinx.coroutines.

Ключевое свойство корутин — **cooperative**: корутина сама решает, когда уступить управление, вызывая suspend-функцию. Это значит, что между вызовами suspend-функций корутина выполняется атомарно (не прерывается). Поэтому длинные вычисления без suspend внутри корутины блокируют поток так же, как обычный код — для них нужен Dispatchers.Default и периодический вызов yield().

5.2.2. suspend-функции — как они работают

suspend — модификатор функции, означающий, что её можно приостановить и возобновить позже. suspend-функция может вызываться только из корутины или из другой suspend-функции. Под капотом компилятор Kotlin превращает suspend-функцию в конечный автомат (state machine), состояния которого соответствуют точкам приостановки. Контекст выполнения передаётся через скрытый параметр

Continuation.

```kotlin
suspend fun fetchUser(id: Long): User {
delay(1000) // suspend-точка 1
val response = api.get(id) // suspend-точка 2 (если api.get suspend)
return response.toUser()
}

// Компилятор превращает это примерно в:
// class FetchUserSM(val cont: Continuation<User>) : Continuation<...> {
// int state = 0
// User result
// fun resume(value) {
// when (state) {
// 0 -> { state = 1; delay(1000, this) }
// 1 -> { state = 2; api.get(id, this) }
// 2 -> { result = value.toUser(); cont.resume(result) }
// }
// }
// }
```

Это значит, что suspend-функция **не блокирует** поток, на котором запущена. Когда она доходит до suspend-точки, она возвращает управление потоку, а когда операция завершается — возобновляется с того же места. Поэтому можно написать val user = fetchUser(1) в корутине на Dispatchers.Main, и UI не зависнет, хотя код выглядит как синхронный.

5.2.3. CoroutineScope — область видимости

CoroutineScope — это объект, который создаёт и отслеживает корутины. Каждая корутина должна запускаться в каком-то scope. Главное правило: scope не завершится, пока не завершатся все его дочерние корутины. Если scope отменяется (scope.cancel()) — отменяются все дочерние.

Scope	Создание	Когда отменяется
`GlobalScope`	Singleton	Никогда (весь процесс приложения) — НЕ ИСПОЛЬЗОВАТЬ
`CoroutineScope(context)`	Вручную	При явном scope.cancel()
`viewModelScope`	В ViewModel (extension property)	При ViewModel.onCleared()
`lifecycleScope`	В Activity/Fragment (androidx)	При Lifecycle.onDestroy
`rememberCoroutineScope()`	В Composable	При уходе Composable из композиции
`runBlocking { }`	Top-level функция	Когда завершится блок (блокирует поток!)
`runTest { }`	В тестах	Когда завершится блок (виртуальное время)

> **Главное правило structured concurrency**

> Никогда не используйте GlobalScope в Android-приложении. Она переживает Activity/Fragment, что приводит к утечкам памяти и работе с уничтоженным UI. Всегда запускайте корутины в viewModelScope, lifecycleScope или rememberCoroutineScope() — они автоматически отменяются при уничтожении владельца.

5.2.4. Dispatchers — на каком потоке выполняется корутина

CoroutineDispatcher определяет, на каком пуле потоков выполняется корутина. Стандартные диспетчеры:

Dispatcher	Пул потоков	Назначение
`Dispatchers.Main`	`Main thread (UI)`	UI-операции, Compose, обращение к View
`Dispatchers.Main.immediate`	`Main thread (без переключения если уже на нём)`	По умолчанию в viewModelScope
`Dispatchers.IO`	`Пул до 64 потоков`	Сеть, БД, файлы — любые I/O-операции
`Dispatchers.Default`	`Пул = # ядер CPU`	CPU-тяжёлые вычисления: JSON-парсинг, сортировка, крипто
`Dispatchers.Unconfined`	`Текущий поток`	Только для top-level; обычно НЕ нужен

Сменить диспетчер внутри корутины можно функцией withContext(dispatcher) — она приостанавливает корутину, переключает на другой пул и возвращает результат. В отличие от launch, withContext возвращает значение.

KOTLIN

```
suspend fun loadUser(id: Long): User = withContext(Dispatchers.IO) {
// Этот блок выполняется на IO-пуле
val json = network.fetch("https://api.example.com/users/$id")
parseUser(json) // если CPU-тяжело — лучше обернуть в withContext(Default)
}
```

5.2.5. Job, SupervisorJob — иерархия и отмена

Job — дескриптор корутины, через который можно узнать состояние, дождаться завершения или отменить. Каждая корутина, запущенная через launch, получает свой Job. По умолчанию Job-ы образуют **иерархию**: дочерняя корутина наследует контекст родителя, и при отмене родителя автоматически отменяются все дети.

Важный нюанс — обработка исключений. Если дочерняя корутина бросает необработанное исключение, оно распространяется на родителя и отменяет всех сиблингов (других детей того же родителя). Это поведение по умолчанию для Job. Чтобы отключить его, используют SupervisorJob — исключение ребёнка не отменяет сиблингов.

Свойство	Job (по умолчанию)	SupervisorJob
Исключение ребёнка отменяет сиблингов	`Да`	`Нет`
Отмена родителя отменяет детей	`Да`	`Да`

Свойство	Job (по умолчанию)	SupervisorJob
Создание	`Job()` или автоматически	`SupervisorJob()`
Scope-эквивалент	`coroutineScope { }`	`supervisorScope { }`
Типичный use case	Связанные задачи (либо все успешны, либо все отменены)	Независимые задачи (загрузка нескольких ресурсов)

KOTLIN

```
// Связанные задачи: один упал → все отменены
suspend fun syncAll(): List<Data> = coroutineScope {
val a = async { fetchA() } // если fetchA бросит, fetchB отменится
val b = async { fetchB() }
listOf(a.await(), b.await())
}

// Независимые задачи: один упал → остальные продолжают
suspend fun syncAllSafe(): List<Result<Data>> = supervisorScope {
val a = async { Result.success(fetchA()) }
val b = async { Result.success(fetchB()) }
try {
listOf(a.await(), b.await())
} catch (e: Exception) {
listOf(Result.failure(e))
}
}
```

5.2.6. async и await — параллельное выполнение

async { } — запускает корутину, которая возвращает результат через Deferred<T>. await() — приостанавливает до готовности результата. Несколько async запускаются параллельно, если их запускать один за другим (без await между ними).

KOTLIN

```
suspend fun loadDashboard(): Dashboard = coroutineScope {
val user = async { fetchUser() } // запустили
val feed = async { fetchFeed() } // запустили параллельно
val settings = async { fetchSettings() } // запустили параллельно
Dashboard(
user = user.await(), // ждём результат (уже готов, если быстрая)
feed = feed.await(),
settings = settings.await(),
)
}
```

Если вызвать awaitAll() на списке Deferred, получится дождаться всех разом. Если любая упадёт — все остальные отменятся (в coroutineScope) или продолжат (в supervisorScope).

5.2.7. Отмена и таймауты

Корутина считается отменённой, когда её Job переводится в состояние Cancelling. Внутри suspend-функций проверяется состояние Job, и при отмене бросается CancellationException. Поэтому важно не глотать CancellationException в catch-блоках — это нарушит механизм отмены.

```kotlin
// [BAD] Плохо — глотает CancellationException
try {
doWork()
} catch (e: Exception) {
log(e) // поймает и CancellationException, нарушив отмену
}

// [OK] Хорошо — пробрасываем CancellationException
try {
doWork()
} catch (e: CancellationException) {
throw e
} catch (e: Exception) {
log(e)
}

// Таймаут:
val result = withTimeout(3000) {
fetchFromNetwork() // бросит TimeoutCancellationException через 3 сек
}
```

5.2.8. Flow — реактивный поток значений

Flow<T> — асинхронный поток значений, последовательно эммитящихся во времени. Похож на ReactiveX Observable, но построен на корутинах. Главное отличие от Sequence — асинхронность (между эммитами могут быть suspend-точки). Главное отличие от Channel — холодность: никто не вычисляет значения, пока нет подписчика.

Cold Flow — вычисления стартуют при подписке. Каждый новый подписчик запускает выполнение заново, получая свой поток значений. Это идеально для одноразовых операций: сетевой запрос → результат, чтение файла → данные. Hot Flow (StateFlow, SharedFlow) — вычисляется независимо от подписчиков; рассмотрим в разделе 5.2.11.

```kotlin
fun countdown(from: Int): Flow<Int> = flow {
for (i in from downTo 0) {
emit(i) // эммитим значение
delay(1000) // suspend-точка
}
}

// Подписка (collect — suspend-функция, только из корутины)
scope.launch {
countdown(5).collect { value ->
println(value)
}
println("Пуск!")
}
```

5.2.9. Операторы Flow — преобразование потока

Операторы трансформируют Flow, создавая новый. Главные базовые операторы:

Оператор	Назначение	Пример
`map { }`	Преобразовать каждое значение	`flow.map { it * 2 }`
`filter { }`	Оставить только подходящие	`flow.filter { it > 0 }`
`onEach { }`	Side-effect на каждое значение	`flow.onEach { println(it) }`
`mapNotNull { }`	map + filter null	`flow.mapNotNull { it.toIntOrNull() }`
`flowOn(disp)`	Сменить Dispatcher восходящего потока	`flow.flowOn(Dispatchers.IO)`
`catch { }`	Обработка исключений (не отменяет поток)	`flow.catch { emit(-1) }`
`retry(n) { }`	Повтор при ошибке	`flow.retry(3)`
`onStart { }`	Действие до первого эммита	`flow.onStart { emit(0) }`
`onCompletion { }`	Действие при завершении (успех или отмена)	`flow.onCompletion { println("done") }`
`take(n)`	Взять первые n значений и завершиться	`flow.take(10)`
`debounce(ms)`	Ждать паузы перед эммитом	`flow.debounce(300)`
`distinctUntilChanged()`	Пропустить подряд идущие дубли	`flow.distinctUntilChanged()`

KOTLIN

```
// Пример: поток целых от 1 до 10, фильтруем чётные, умножаем на 10
flow {
for (i in 1..10) emit(i)
}
.filter { it % 2 == 0 } // 2, 4, 6, 8, 10
.map { it * 10 } // 20, 40, 60, 80, 100
.onEach { println("emit: $it") }
.collect { value ->
println("got: $value")
}
```

5.2.10. flowOn vs LaunchIn vs collect — частая путаница

Эти три функции часто путают, потому что все три «что-то делают с потоком». Разница:

• flowOn(dispatcher) — меняет Dispatcher, на котором выполняются **восходящие** операторы (до flowOn). Не запускает Flow.

• collect { } — терминальный оператор, запускает выполнение Flow. **Suspend-функция** — блокирует корутину до завершения потока.

• launchIn(scope) — то же, что scope.launch { collect { } }, но короче. Не suspend, запускает сборку в фоне и сразу возвращает Job.

```kotlin
// Три эквивалентных способа подписаться:

// 1. В корутине через collect
scope.launch {
flow.collect { value -> handle(value) }
}

// 2. Через launchIn + onEach
flow
.onEach { value -> handle(value) }
.launchIn(scope)

// Оба варианта дают одинаковый эффект. launchIn удобен, когда у вас
// длинная цепочка операторов и вы хотите подписаться "в конце". collect
// удобен, когда логика обработки простая.
```

5.2.11. StateFlow и SharedFlow — горячие потоки

У cold Flow есть ограничение: каждый подписчик запускает выполнение заново. Это неудобно для состояния, которое должно быть общим для нескольких подписчиков (UI-состояние экрана, например). Для этого есть **hot** потоки:

• **StateFlow<T>** — горячий поток, всегда имеет текущее значение. Новые подписчики сразу получают последнее. Идеально для UI-состояния. **Conflation**: если подписчик не успевает, получает только последнее значение.

• **SharedFlow<T>** — горячий поток без обязательного начального значения. Поддерживает replay — сколько последних значений получают новые подписчики. Идеально для событий (snackbar, навигация, тосты).

```kotlin
// StateFlow — состояние экрана
class CounterHolder {
private val _count = MutableStateFlow(0)
val count: StateFlow<Int> = _count.asStateFlow()

fun inc() { _count.value++ }
fun dec() { _count.value-- }
}

// SharedFlow — события
class EventBus {
private val _events = MutableSharedFlow<String>(extraBufferCapacity = 16)
val events: SharedFlow<String> = _events.asSharedFlow()

suspend fun emit(event: String) { _events.emit(event) }
}
```

5.2.12. channelFlow и callbackFlow — bridging callbacks

Иногда нужно превратить callback-based API в Flow. Для этого есть два специальных билдера:

• channelFlow { } — Flow с поддержкой отправки значений из другого потока (через send()). Полезно, когда эммит происходит из callback-а.

• callbackFlow { } — то же + ждёт регистрации callback-а и обеспечивает очистку через awaitClose { }. Идеально для обёртывания listener-ов.

```kotlin
// Превращаем callback API в Flow
fun locationUpdates(provider: String): Flow<Location> = callbackFlow {
val listener = object : LocationListener {
override fun onLocationChanged(location: Location) {
trySend(location) // эммитим в Flow
}
}

locationManager.requestLocationUpdates(provider, 1000, 1f, listener)

// Ждём отмены подписки, чтобы снять listener
awaitClose {
locationManager.removeUpdates(listener)
}
}
```

5.2.13. Тестирование корутин — runTest и TestDispatcher

В тестах нельзя использовать реальные задержки — тесты должны быть мгновенными. Библиотека kotlinx-coroutines-test предоставляет функцию runTest { }, которая виртуализирует время: delay(1000) мгновенно завершается, withTimeout не требует реального ожидания.

```kotlin
// Зависимость: testImplementation("org.jetbrains.kotlinx:kotlinx-coroutines-test:1.8.1")

@Test
fun `delay advances instantly`() = runTest {
var fired = false
launch {
delay(1000)
fired = true
}
assertFalse(fired) // 0 ms прошло
advanceTimeBy(1000) // виртуально ждём 1 сек
assertTrue(fired) // корутина завершена
}

@Test
fun `flow emits values`() = runTest {
val flow = flow {
emit(1); delay(100)
emit(2); delay(100)
emit(3)
}
val values = mutableListOf<Int>()
flow.collect { values.add(it) }
assertEquals(listOf(1, 2, 3), values)
}
```

runTest использует TestDispatcher — специальный диспетчер, который управляет виртуальными часами. Два варианта: StandardTestDispatcher (нужно явно двигать время через advanceTimeBy / runCurrent) и UnconfinedTestDispatcher (выполняет корутины сразу). Первый — для тонкого контроля, второй — для простых тестов.

Примеры кода

Пример 5.1. Параллельная загрузка с supervisorScope

```kotlin
data class Dashboard(val profile: Profile, val feed: List<Post>)

suspend fun loadDashboard(): Dashboard = supervisorScope {
val profileDeferred = async { fetchProfile() }
val feedDeferred = async { fetchFeed() }
Dashboard(
profile = profileDeferred.await(),
feed = feedDeferred.await(),
)
}

// Если fetchProfile бросит, fetchFeed продолжит работать (в supervisorScope)
// await() всё равно бросит — но можно обернуть в try/catch
```

Пример 5.2. Flow с debounce для поиска

```kotlin
// Простой поиск с задержкой 300 мс
fun searchFlow(
queries: Flow<String>,
search: suspend (String) -> List<String>,
): Flow<List<String>> = queries
.debounce(300) // ждём окончания ввода
.distinctUntilChanged() // пропускаем дубли
.filter { it.length >= 2 } // минимум 2 символа
.map { q -> search(q) } // выполняем поиск
.catch { e -> emit(emptyList()) } // обработка ошибок
.flowOn(Dispatchers.IO) // на IO-пуле

// Использование:
val queries = MutableStateFlow("")
scope.launch {
searchFlow(queries) { q ->
// здесь могла бы быть ваша реализация поиска
listOf("result1 for $q", "result2 for $q")
}.collect { results ->
println("Найдено: ${results.size}")
}
}
queries.value = "пр"
queries.value = "прив"
queries.value = "привет" // через 300 мс выполнится 1 поиск ("привет")
```

Пример 5.3. StateFlow как состояние

```kotlin
class Counter {
private val _count = MutableStateFlow(0)
val count: StateFlow<Int> = _count.asStateFlow()

fun inc() { _count.update { it + 1 } } // атомарное обновление
fun dec() { _count.update { it - 1 } }
fun reset() { _count.value = 0 }
}

// Несколько подписчиков получают одно и то же значение
val counter = Counter()
val s1 = scope.launch { counter.count.collect { println("s1: $it") } }
val s2 = scope.launch { counter.count.collect { println("s2: $it") } }
delay(100)
counter.inc() // s1 и s2 оба увидят 1
counter.inc() // s1 и s2 оба увидят 2
```

Задание

Реализуйте три небольших компонента на корутинах и Flow. Все они — standalone-утилиты, не зависящие от NotesApp и от Repository/Room/Retrofit. Это фундамент, на котором построены последующие практики.

Шаг 1. Утилита CountdownTimer

В файле util/CountdownTimer.kt создайте функцию:

```kotlin
fun countdown(
fromSeconds: Int,
tickMs: Long = 1000,
): Flow<Int>
```

Требования:

1. Возвращает Flow, эммитящий значения от fromSeconds до 0 включительно.
2. Между эммитами — delay(tickMs).
3. После эммита 0 — поток завершается (никаких специальных сообщений, просто обычное завершение).
4. Если подписчик отменит сбор (отпишется), таймер должен остановиться — никаких утечек корутин.

Шаг 2. Параллельная загрузка нескольких URL

В файле util/ParallelFetcher.kt создайте функцию:

```kotlin
suspend fun fetchAll(
urls: List<String>,
fetch: suspend (String) -> String,
): List<Result<String>>
```

Требования:

1. Использовать supervisorScope + async.

2. Запустить запросы по всем URL параллельно.

3. Если один запрос упал — остальные продолжают.

4. Возвращает список Result<String> — Result.success для успешных, Result.failure для упавших.

5. Порядок результатов должен совпадать с порядком URL.

В качестве fetch передавайте функцию, которая имитирует сетевой запрос: { url -> delay(500 + Random.nextLong(1000)); "Response for $url" }. В production здесь будет реальный Retrofit-вызов (практика 8).

Шаг 3. SearchFlow с debounce

В файле util/SearchFlow.kt создайте функцию:

```kotlin
fun <T> searchFlow(
queries: Flow<String>,
minLength: Int = 2,
debounceMs: Long = 300,
search: suspend (String) -> List<T>,
): Flow<List<T>>
```

Требования:

1. Использовать debounce(debounceMs).

2. Использовать distinctUntilChanged().

3. Использовать filter { it.length >= minLength || it.isEmpty() }.

4. При пустом запросе — эммитить пустой список (без вызова search).

5. Использовать map { search(it) } для непустых запросов.

6. Использовать catch { emit(emptyList()) } — ошибки не пробрасывать, а возвращать пустой результат.

7. Применить flowOn(Dispatchers.IO).

Эта функция будет переиспользована в практике 6 (ViewModel) для поиска заметок — но с реальным Repository вместо лямбды search. Поэтому сигнатура должна точно соответствовать указанной.

Шаг 4. Тесты

В файле src/test/.../CoroutinesTest.kt напишите unit-тесты с использованием runTest:

1. test_countdown_emits_from_to_zero — проверка последовательности эммитов

2. test_countdown_completes_after_zero — поток завершается, toList() возвращает конечный список

3. test_countdown_cancellable — при отмене collect корутина завершается (можно проверить через withTimeoutOrNull)

4. test_fetchAll_parallel_returns_all_results — 3 URL, все успешны, возвращает 3 Result.success

5. test_fetchAll_one_fails_others_succeed — один URL бросает, остальные возвращают Result.success, упавший — Result.failure

6. test_searchFlow_debounce_skips_intermediate — 3 быстрых ввода подряд → только последний доходит до search

7. test_searchFlow_empty_query_returns_empty_list — пустой запрос не вызывает search

8. test_searchFlow_min_length_filter — ввод 1 символа не вызывает search

9. test_searchFlow_catch_emits_empty_on_error — если search бросает, эмитится пустой список

Шаг 5. Запуск тестов

Запустите ./gradlew test --tests "ru.notesapp.util.*". Все 9 тестов должны быть зелёными. Если используете TestDispatcher — убедитесь, что advanceTimeBy(300) вызывается перед проверкой debounce.

Контрольные вопросы

Ответьте письменно на следующие вопросы. Подготовьте устные ответы для защиты практической работы — преподаватель может задать любой из них или попросить пояснить код вашего решения.

1. Что такое корутина? Чем она отличается от потока ОС?

2. Что делает suspend-модификатор? Можно ли вызвать suspend-функцию из обычной функции?

3. Как компилятор Kotlin превращает suspend-функцию в state machine? Что такое Continuation?

4. Что такое structured concurrency? Почему GlobalScope нельзя использовать в Android?

5. Чем Dispatchers.IO отличается от Dispatchers.Default? Когда использовать каждый?

6. Зачем нужен withContext? Чем отличается от launch?

7. Чем SupervisorJob отличается от обычного Job? Приведите пример, когда нужен именно SupervisorJob.

8. Как async связан с await? Что будет, если вызвать await до завершения дочерней корутины?

9. Почему важно не глотать CancellationException в catch-блоках? Что произойдёт, если это сделать?

10. Чем cold Flow отличается от hot StateFlow/SharedFlow?

11. Что делает оператор debounce? Почему он важен для поиска?

12. Чем flowOn отличается от launchIn? А от collect?

13. Зачем нужен callbackFlow? Чем отличается от обычного flow { }?

14. Что делает runTest? Зачем нужен TestDispatcher?

15. Можно ли превратить обычный callback API в Flow? Как?

Чек-лист сдачи работы

Перед сдачей работы преподавателю убедитесь, что выполнены все пункты ниже. Отсутствие любого пункта снижает оценку минимум на один балл.

- □ В util/CountdownTimer.kt есть fun countdown(fromSeconds, tickMs): Flow
- □ Между эммитами — delay(tickMs)
- □ После эммита 0 — поток завершается
- □ В util/ParallelFetcher.kt есть suspend fun fetchAll(urls, fetch): List>
- □ Используется supervisorScope + async
- □ Если один запрос упал — остальные продолжают
- □ В util/SearchFlow.kt есть fun searchFlow(queries, minLength, debounceMs, search): Flow>
- □ Используются debounce, distinctUntilChanged, filter, map, catch, flowOn
- □ Пустой запрос возвращает emptyList() без вызова search
- □ В src/test/.../CoroutinesTest.kt есть 9+ тестов

- □ Все тесты проходят (./gradlew test --tests "ru.notesapp.util.*")
- □ Используется runTest и TestDispatcher
- □ Нет утечек корутин (таймер останавливается при отмене collect)

Что сдавать

В результате выполнения практической работы студент предоставляет:

1. Git-коммит «Практика 5» с тремя новыми файлами в util/ и тестами;
2. Скриншот прохождения тестов (./gradlew test --tests "ru.notesapp.util.*");
3. Краткий отчёт: какие операторы Flow вы использовали и почему именно их; что было непонятно.

Практика 6. State management и ViewModel

Трудоёмкость: 6 академических часов. **Форма работы:** индивидуальная, в IDE Android Studio. **Форма отчёта:** исходный код проекта + устная защита.

Цель и задачи

Цель работы — внедрить архитектурный паттерн MVVM (Model-View-ViewModel) в NotesApp: вынести состояние из Composable в ViewModel, предоставить его через StateFlow, научиться подписываться на поток с учётом жизненного цикла. После этой практики UI становится stateless, а вся логика управления данными концентрируется в ViewModel — это ключевой шаг к тестируемой и поддерживаемой архитектуре.

1. объяснить разницу между stateful и stateless Composable;
2. применять паттерн state hoisting;
3. создавать ViewModel с жизненным циклом, переживаемым смену конфигурации;
4. использовать StateFlow как single source of truth;
5. применять sealed UiState для представления Loading/Success/Error/Empty;
6. подписываться на StateFlow в Compose через collectAsStateWithLifecycle().

Теоретическая часть

6.2.1. Stateful vs Stateless Composable

Stateful Composable — функция, которая сама хранит своё состояние через remember и mutableStateOf. Пример: счётчик из практики 2. Это удобно для прототипа, но плохо для тестируемости и переиспользования: нельзя подменить состояние в тесте, нельзя использовать один и тот же Composable с двумя разными источниками данных. **Stateless Composable** получает всё состояние через параметры и сообщает об изменениях через callback-и. Идиома Compose: «state goes down, events go up».

KOTLIN

```
// Stateful — состояние внутри
@Composable
fun CounterStateful() {
var count by remember { mutableStateOf(0) }
Button(onClick = { count++ }) { Text("$count") }
}

// Stateless — состояние снаружи
@Composable
fun CounterStateless(count: Int, onIncrement: () -> Unit) {
Button(onClick = onIncrement) { Text("$count") }
}
```

6.2.2. State hoisting — вынос состояния

State hoisting — паттерн перемещения состояния вверх по дереву Composable, к родителю. Цель: сделать дочерний компонент stateless и переиспользуемым. Правила: состояние поднимается до минимального

общего предка всех потребителей; события обрабатываются там же, где живёт состояние. В MVVM состоянием владеет ViewModel — это конечная точка подъёма для UI-состояния.

6.2.3. ViewModel — жизненный цикл и хранение состояния

ViewModel — класс из библиотеки androidx.lifecycle:lifecycle-viewmodel-compose, который хранит состояние UI и переживает смену конфигурации (поворот экрана, смену языка, dark mode). ViewModel НЕ должен хранить ссылку на Activity, Fragment или Context — это утечка памяти. Если нужен Context, используется AndroidViewModel с application внутри, или (лучше) зависимости пробрасываются через конструктор с помощью Hilt.

Получить ViewModel в Composable можно двумя способами:

- viewModel<MyViewModel>() — привязан к текущему ViewModelStoreOwner (Activity / Fragment / NavGraph);
- hiltViewModel() — то же, но с инстанциацией через Hilt (практика 8).

Жизненный цикл ViewModel: создаётся при первом обращении, уничтожается когда соответствующий ViewModelStoreOwner финально уничтожается (Activity.finish, Fragment.destroy). Метод onCleared() — последнее место для освобождения ресурсов.

6.2.4. viewModelScope — корутины в ViewModel

ViewModel имеет встроенный viewModelScope — CoroutineScope, привязанный к жизненному циклу. Все корутины, запущенные в нём, отменяются при onCleared(). Это правильное место для сетевых запросов, работы с БД, таймеров. По умолчанию viewModelScope использует Dispatchers.Main.immediate, поэтому для I/O-операций нужно явно переключаться на Dispatchers.IO.

KOTLIN

```
class NotesViewModel(
private val repository: NotesRepository,
) : ViewModel() {

fun addNote(title: String, content: String) {
viewModelScope.launch {
withContext(Dispatchers.IO) {
repository.insert(Note(
id = 0,
title = title,
content = content,
createdAt = System.currentTimeMillis(),
type = NoteType.Text,
))
}
}
}
}
```

6.2.5. StateFlow vs SharedFlow vs LiveData

Три механизма предоставления потока данных во View. Сравнение:

Свойство	StateFlow	SharedFlow	LiveData
Hot / Cold	Hot	Hot	Hot

Свойство	StateFlow	SharedFlow	LiveData
Начальное значение	Обязательно	Нет	Обязательно
Replay (значения новым подписчикам)	1 (последнее)	0..n	1 (последнее)
Conflation (объединение)	Да	Нет (по умолч.)	Да
Поддержка в Compose	collectAsStateWithLifecycle	collectAsState	observeAsState
Идеальный use case	UI state	One-shot events (snackbar, navigation)	Legacy проекты

Для UI-состояния в современной разработке используется StateFlow: у него всегда есть значение, он автоматически применяет conflation (если подписчик не успевает, получит только последнее), и работает с collectAsStateWithLifecycle() — который отменяет подписку когда View не на экране.

6.2.6. collectAsStateWithLifecycle — правильная подписка

Функция collectAsStateWithLifecycle() из библиотеки androidx.lifecycle:lifecycle-runtime-compose — это рекомендованный способ чтения StateFlow в Compose. В отличие от старого collectAsState(), она автоматически отменяет сборку когда Lifecycle переходит в состояние < STARTED. Это экономит батарею и предотвращает лишние recomposition когда пользователь не видит экран.

KOTLIN

```
@Composable
fun NotesListScreen(
viewModel: NotesViewModel = viewModel(),
) {
val uiState by viewModel.uiState.collectAsStateWithLifecycle()
when (val s = uiState) {
is NotesUiState.Loading -> LoadingIndicator()
is NotesUiState.Empty -> EmptyState("Нет заметок", "Добавьте первую")
is NotesUiState.Success -> NoteList(notes = s.notes, onClick = { /* ... */ })
is NotesUiState.Error -> ErrorState(s.message)
}
}
```

6.2.7. Sealed UiState — представление состояния экрана

Каждый экран может находиться в одном из нескольких состояний: Loading (идёт загрузка), Success (есть данные), Empty (данных нет), Error (произошла ошибка). Идиома — описать это sealed class и в Composable делать when по нему. Это гарантирует, что разработчик не забудет обработать ни одну ветку (компилятор укажет на неполный when).

KOTLIN

```
sealed interface NotesUiState {
data object Loading : NotesUiState
data object Empty : NotesUiState
data class Success(val notes: List<Note>) : NotesUiState
data class Error(val message: String) : NotesUiState
}
```

6.2.8. Single source of truth

Принцип «single source of truth» (SSOT) требует, чтобы каждое данное хранилось в одном месте. В MVVM этим местом является ViewModel (или Repository за ним). Composable не должны иметь собственной копии списка заметок, фильтра, поискового запроса — все эти данные читаются из ViewModel через StateFlow. Если несколько экранов используют одно состояние, они делят один ViewModel (или общий Repository, к которому каждый экран обращается через свой ViewModel).

6.2.9. Unidirectional Data Flow (UDF)

UDF — паттерн, при котором данные текут в одном направлении: состояние хранится в ViewModel → передаётся в View через StateFlow → View отображает состояние → пользовательские действия (events) передаются в ViewModel через методы → ViewModel обновляет состояние → цикл повторяется. UDF гарантирует: нет циклических зависимостей, состояние предсказуемо, легко тестировать (подал event — проверил state).

Примеры кода

Пример 6.1. Минимальный ViewModel с StateFlow

KOTLIN

```
class CounterViewModel : ViewModel() {
private val _count = MutableStateFlow(0)
val count: StateFlow<Int> = _count.asStateFlow()

fun increment() { _count.update { it + 1 } }
fun reset() { _count.value = 0 }
}

@Composable
fun CounterScreen(viewModel: CounterViewModel = viewModel()) {
val count by viewModel.count.collectAsStateWithLifecycle()
Column(horizontalAlignment = Alignment.CenterHorizontally) {
Text("$count", style = MaterialTheme.typography.displayLarge)
Button(onClick = viewModel::increment) { Text("++") }
TextButton(onClick = viewModel::reset) { Text("Сброс") }
}
}
```

Пример 6.2. ViewModel с UiState и обработкой ошибок

```kotlin
class NotesViewModel(
private val repo: NotesRepository,
) : ViewModel() {

private val _uiState = MutableStateFlow<NotesUiState>(NotesUiState.Loading)
val uiState: StateFlow<NotesUiState> = _uiState.asStateFlow()

init { loadNotes() }

fun loadNotes() {
_uiState.value = NotesUiState.Loading
viewModelScope.launch {
try {
val notes = withContext(Dispatchers.IO) { repo.getAll() }
_uiState.value =
if (notes.isEmpty()) NotesUiState.Empty
else NotesUiState.Success(notes)
} catch (e: Exception) {
_uiState.value = NotesUiState.Error(e.message ?: "Неизвестная ошибка")
}
}
}
}
```

Задание

В NotesApp создайте **NotesViewModel**, который управляет состоянием экрана списка заметок. На этом практике данные всё ещё хранятся в памяти (in-memory repository), БД появится в практике 6. Главное — вынести состояние из Composable в ViewModel и предоставить его через StateFlow.

Шаг 1. InMemoryNotesRepository

В файле data/InMemoryNotesRepository.kt создайте класс-заглушку:

```kotlin
class InMemoryNotesRepository {
private val notes = mutableListOf(
Note(1, "Первая заметка", "Текст первой заметки",
System.currentTimeMillis(), NoteType.Text),
Note(2, "Картинка", "Описание картинки",
System.currentTimeMillis(), NoteType.Image),
)

fun getAll(): List<Note> = notes.toList()
fun getById(id: Long): Note? = notes.find { it.id == id }
fun add(note: Note): Long {
val newId = (notes.maxOfOrNull { it.id } ?: 0) + 1
notes.add(note.copy(id = newId))
return newId
}
fun delete(id: Long): Boolean = notes.removeIf { it.id == id }
fun search(query: String): List<Note> =
notes.filter { it.title.contains(query, ignoreCase = true) }
}
```

Шаг 2. NotesUiState

В файле viewmodel/NotesUiState.kt опишите sealed interface с состояниями: Loading, Empty, Success(notes: List<Note>), Error(message: String). Используйте data object для Loading и Empty, data class для Success и Error.

Шаг 3. NotesViewModel

В файле viewmodel/NotesViewModel.kt создайте класс NotesViewModel (наследник ViewModel) с публичным StateFlow<NotesUiState> и методами:

1. loadNotes() — загружает все заметки из репозитория, обновляет UiState. Должен имитировать загрузку: delay(500) в Dispatchers.IO перед получением данных, чтобы показать Loading.

2. addNote(title: String, content: String, type: NoteType) — добавляет заметку и обновляет UiState.

3. deleteNote(id: Long) — удаляет заметку и обновляет UiState.

4. searchNotes(query: String) — фильтрует заметки по title, обновляет UiState. Если query пустой, показывает все.

В init вызывается loadNotes(). Все операции запускаются в viewModelScope.

Шаг 4. Подключение к NotesListScreen

В NotesListScreen.kt замените захардкоженный список на чтение из ViewModel:

```kotlin
@Composable
fun NotesListScreen(
onNoteClick: (Note) -> Unit,
viewModel: NotesViewModel = viewModel(),
) {
val uiState by viewModel.uiState.collectAsStateWithLifecycle()
when (val s = uiState) {
is NotesUiState.Loading -> Box(Modifier.fillMaxSize()) { CircularProgressIndicator() }
is NotesUiState.Empty -> EmptyState("Пока нет заметок", "Нажмите +, чтобы добавить")
is NotesUiState.Success -> NoteList(notes = s.notes, onClick = onNoteClick)
is NotesUiState.Error -> ErrorState(message = s.message, onRetry = { viewModel.loadNotes()
})
}
}
```

Шаг 5. FAB добавления заметки

В NotesListScreen FAB должен показывать простой диалог (можно AlertDialog) с двумя TextField: «Заголовок» и «Содержание». По кнопке «Сохранить» вызывается viewModel.addNote(title, content, NoteType.Text). Диалог закрывается.

Шаг 6. Удаление по свайпу (бонусно)

Опционально: реализуйте swipe-to-delete через SwipeToDismissBox из Material 3. После свайпа вызывается viewModel.deleteNote(id) и показывается Snackbar с кнопкой «Отменить» (пока просто логируется).

Контрольные вопросы

Ответьте письменно на следующие вопросы. Подготовьте устные ответы для защиты практической работы — преподаватель может задать любой из них или попросить пояснить код вашего решения.

1. Чем stateful Composable отличается от stateless? Зачем делать Composable stateless?

2. Что такое state hoisting? Сформулируйте правила.

3. Зачем нужен ViewModel? Почему нельзя хранить состояние в Activity?

4. Почему ViewModel не должен хранить ссылку на Activity / Context? К чему это приведёт?

5. Что такое viewModelScope? Когда он отменяется?

6. Чем StateFlow отличается от SharedFlow? Какой выбрать для UI-состояния?

7. Зачем нужно начальное значение у StateFlow?

8. Чем collectAsStateWithLifecycle() лучше collectAsState()?

9. Что такое sealed UiState? Какие преимущества даёт использование sealed class для состояний экрана?

10. Сформулируйте принцип Single Source of Truth. К чему приводит его нарушение?

11. Опишите цикл Unidirectional Data Flow. Кто хранит состояние, кто порождает events?

12. Как пережить смену конфигурации (поворот экрана) без потери введённого текста? Что нужно сделать?

Чек-лист сдачи работы

Перед сдачей работы преподавателю убедитесь, что выполнены все пункты ниже. Отсутствие любого пункта снижает оценку минимум на один балл.

- □ В data/InMemoryNotesRepository.kt есть класс с методами getAll, getById, add, delete, search
- □ В viewmodel/NotesUiState.kt есть sealed interface NotesUiState
- □ 4 состояния: Loading (data object), Empty (data object), Success (data class), Error (data class)
- □ В viewmodel/NotesViewModel.kt класс наследует ViewModel
- □ NotesViewModel имеет публичный StateFlow<NotesUiState>
- □ Методы loadNotes / addNote / deleteNote / searchNotes запускают корутины в viewModelScope
- □ loadNotes имитирует задержку через delay(500) в Dispatchers.IO
- □ init { } вызывает loadNotes()
- □ NotesListScreen использует collectAsStateWithLifecycle
- □ when по UiState покрывает все 4 ветки без else
- □ FAB показывает AlertDialog с двумя TextField
- □ При пустом заголовке кнопка «Сохранить» отключена или показывает ошибку
- □ При повороте экрана состояние сохраняется (список не сбрасывается)

Что сдавать

В результате выполнения практической работы студент предоставляет:

1. Git-коммит «Практика 5» с InMemoryNotesRepository, NotesUiState, NotesViewModel;

2. Скриншот экрана в состоянии Loading и в состоянии Success со списком заметок;

3. Скриншот диалога добавления новой заметки;

4. Запись экрана (или 2 скриншота) demonstrating сохранение состояния при повороте.

Практика 7. Room: локальная база данных

Трудоёмкость: 6 академических часов. **Форма работы:** индивидуальная, в IDE Android Studio. **Форма отчёта:** исходный код проекта + устная защита.

Цель и задачи

Цель работы — заменить in-memory хранилище NotesApp на постоянное с помощью библиотеки Room (официальная ORM от Android Jetpack). Студент научится описывать сущности, DAO и Database, использовать TypeConverters для пользовательских типов и писать миграции при изменении схемы. После этой практики заметки сохраняются между запусками приложения.

1. объяснить трёхслойную архитектуру Room: Entity / DAO / Database;

2. использовать аннотации @Entity, @PrimaryKey, @ColumnInfo, @Dao, @Query, @Insert, @Update, @Delete;

3. создавать TypeConverters для sealed class NoteType;

4. возвращать Flow<List<Note>> из DAO для реактивного обновления UI;

5. писать миграции при изменении схемы (добавление поля);

6. тестировать DAO на in-memory Room.

Теоретическая часть

7.2.1. Архитектура Room

Room — это высокоуровневая обёртка над SQLite, входящая в Android Jetpack. Она предоставляет три ключевых компонента:

• **@Entity** — класс данных, отображаемый на таблицу SQLite. Каждое свойство = колонка, класс = таблица.

• **@Dao** (Data Access Object) — интерфейс с методами для работы с таблицей. Room генерирует реализацию на этапе компиляции и проверяет SQL-запросы.

• **@Database** — абстрактный класс, представляющий базу данных. Содержит список сущностей и DAO-методов доступа. Singleton на уровне приложения.

Принципиальное преимущество Room перед «голым» SQLite — проверка SQL на этапе компиляции: если в запросе опечатка в имени колонки, сборка упадёт. Также Room нативно работает с корутинами и Flow: метод DAO может быть suspend и/или возвращать Flow<T> — тогда при изменении данных подписчик автоматически получит новое значение.

7.2.2. Entity — описание таблицы

Аннотация @Entity над data class указывает Room, что класс — таблица. Параметры: tableName (по умолчанию имя класса), indices (для ускорения запросов), foreignKeys (связи между таблицами). Каждое свойство — колонка. Аннотация @PrimaryKey помечает первичный ключ; для автоинкремента — autoGenerate = true. @ColumnInfo позволяет задать имя колонки и тип.

```kotlin
@Entity(tableName = "notes")
data class NoteEntity(
@PrimaryKey(autoGenerate = true)
val id: Long = 0,

@ColumnInfo(name = "title")
val title: String,

@ColumnInfo(name = "content")
val content: String,

@ColumnInfo(name = "created_at")
val createdAt: Long,

@ColumnInfo(name = "type")
val type: NoteType, // потребует TypeConverter
)
```

7.2.3. DAO — запросы к базе

DAO — интерфейс с методами, описывающими операции с таблицей. Четыре способа описать операцию:

Аннотация	Описание	SQL
`@Insert`	Вставка строки (или нескольких). onConflict определяет стратегию	`INSERT OR ...`
`@Update`	Обновление по первичному ключу	`UPDATE ... WHERE id=?`
`@Delete`	Удаление по первичному ключу	`DELETE ... WHERE id=?`
`@Query`	Произвольный SQL-запрос (только SELECT)	`SELECT ...`

```kotlin
@Dao
interface NoteDao {
@Query("SELECT * FROM notes ORDER BY created_at DESC")
fun getAll(): Flow<List<NoteEntity>>

@Query("SELECT * FROM notes WHERE id = :id")
suspend fun getById(id: Long): NoteEntity?

@Query("SELECT * FROM notes WHERE title LIKE '%' || :query || '%'")
fun searchByTitle(query: String): Flow<List<NoteEntity>>

@Insert(onConflict = OnConflictStrategy.REPLACE)
suspend fun insert(note: NoteEntity): Long

@Update
suspend fun update(note: NoteEntity)

@Query("DELETE FROM notes WHERE id = :id")
suspend fun deleteById(id: Long)
}
```

Методы могут быть suspend (выполняются на Dispatchers.IO под капотом) или возвращать Flow<T> для реактивного обновления. Mix не разрешён: Flow нельзя сделать suspend.

7.2.4. Database — Singleton базы

@Database — абстрактный класс, наследующий RoomDatabase. Аннотация принимает: entities (массив классов-сущностей), version (номер версии схемы, увеличивается при миграциях), exportSchema (сохранять ли JSON-схему в каталог schemas/ — рекомендуется true для возможности миграций и тестов). Доступ к Database должен быть Singleton — обычно через companion object с двойной проверкой или через Hilt (практика 9).

KOTLIN

```
@Database(
entities = [NoteEntity::class],
version = 1,
exportSchema = true,
)
abstract class AppDatabase : RoomDatabase() {
abstract fun noteDao(): NoteDao

companion object {
@Volatile private var INSTANCE: AppDatabase? = null

fun getInstance(context: Context): AppDatabase =
INSTANCE ?: synchronized(this) {
INSTANCE ?: Room.databaseBuilder(
context.applicationContext,
AppDatabase::class.java,
"notes.db",
).build().also { INSTANCE = it }
}
}
}
```

7.2.5. TypeConverters — пользовательские типы

SQLite хранит только примитивы: Int, Long, String, Float, ByteArray. Если в Entity есть поле пользовательского типа (например, sealed class NoteType), нужен конвертер. Аннотация @TypeConverter помечает методы-конвертеры; @TypeConverters на уровне Database регистрирует класс-держатель конвертеров.

```kotlin
class NoteTypeConverter {
@TypeConverter
fun fromType(t: NoteType): String = when (t) {
is NoteType.Text -> "TEXT"
is NoteType.Image -> "IMAGE"
is NoteType.Audio -> "AUDIO"
}

@TypeConverter
fun toType(s: String): NoteType = when (s) {
"TEXT" -> NoteType.Text
"IMAGE" -> NoteType.Image
"AUDIO" -> NoteType.Audio
else -> throw IllegalArgumentException("Unknown NoteType: $s")
}
}

@Database(
entities = [NoteEntity::class],
version = 2,
exportSchema = true,
)
@TypeConverters(NoteTypeConverter::class)
abstract class AppDatabase : RoomDatabase() { ... }
```

7.2.6. Миграции — изменение схемы

Когда в Entity добавляется поле (или меняется тип), нужно увеличить version и предоставить миграцию. Миграция описывается как Migration(fromVersion, toVersion) с переопределённым migrate(database), в котором выполняется SQL ALTER TABLE. Миграции регистрируются в Room.databaseBuilder().addMigrations(...). Если миграция не предоставлена, Room упадёт с исключением — если только не вызван fallbackToDestructiveMigration() (полное пересоздание, потеря данных — только для разработки).

```kotlin
val MIGRATION_1_2 = object : Migration(1, 2) {
override fun migrate(db: SupportSQLiteDatabase) {
db.execSQL("ALTER TABLE notes ADD COLUMN updated_at INTEGER NOT NULL DEFAULT 0")
}
}

// В builder:
Room.databaseBuilder(context, AppDatabase::class.java, "notes.db")
.addMigrations(MIGRATION_1_2)
.build()
```

7.2.7. Flow-возвращаемые DAO — реактивность

Если метод DAO возвращает Flow<List<NoteEntity>>, Room автоматически перечитывает таблицу при любом изменении и эммитит новое значение в поток. Это идеальный способ связать UI с базой: ViewModel подписывается на Flow, любой INSERT/UPDATE/DELETE автоматически отражается в UI. Важно: Room пересылает только изменённые значения — если список не поменялся, Flow не эммитит.

7.2.8. Зависимости и конфигурация

В app/build.gradle.kts добавьте:

```kotlin
plugins {
id("com.google.devtools.ksp")
}

dependencies {
val roomVersion = "2.6.1"
implementation("androidx.room:room-runtime:$roomVersion")
implementation("androidx.room:room-ktx:$roomVersion")
ksp("androidx.room:room-compiler:$roomVersion")

// Для тестов:
androidTestImplementation("androidx.room:room-testing:$roomVersion")
}
```

Плагин KSP (Kotlin Symbol Processing) генерирует реализацию DAO на этапе компиляции. Альтернатива — kapt, но он медленнее и объявлен deprecated для новых проектов. Не забудьте добавить плагин в plugins-блок корневого build.gradle.kts.

7.2.9. Mapping Entity ↔ Domain

Хорошая практика — разделять классы Entity (для БД) и Domain (для бизнес-логики). Это позволяет менять схему БД без каскадных изменений в UI. Mapping делается функциями-расширениями или mapper-объектом. На этой практике сделайте это минимально: NoteEntity уже почти совпадает с Note, но для порядка создайте явные fun NoteEntity.toDomain(): Note и fun Note.toEntity(): NoteEntity.

Примеры кода

Пример 7.1. Полный цикл Entity → Dao → Database

KOTLIN

```kotlin
// data/local/NoteEntity.kt
@Entity(tableName = "notes", indices = [Index("created_at")])
data class NoteEntity(
@PrimaryKey(autoGenerate = true) val id: Long = 0,
@ColumnInfo(name = "title") val title: String,
@ColumnInfo(name = "content") val content: String,
@ColumnInfo(name = "created_at") val createdAt: Long,
@ColumnInfo(name = "updated_at") val updatedAt: Long = 0L,
@ColumnInfo(name = "type") val type: NoteType,
)

// data/local/NoteDao.kt
@Dao
interface NoteDao {
@Query("SELECT * FROM notes ORDER BY created_at DESC")
fun observeAll(): Flow<List<NoteEntity>>

@Query("SELECT * FROM notes WHERE id = :id")
suspend fun getById(id: Long): NoteEntity?

@Insert(onConflict = OnConflictStrategy.REPLACE)
suspend fun insert(note: NoteEntity): Long

@Update
suspend fun update(note: NoteEntity)

@Query("DELETE FROM notes WHERE id = :id")
suspend fun deleteById(id: Long)
}

// data/local/AppDatabase.kt
@Database(entities = [NoteEntity::class], version = 2, exportSchema = true)
@TypeConverters(NoteTypeConverter::class)
abstract class AppDatabase : RoomDatabase() {
abstract fun noteDao(): NoteDao
}
```

Пример 7.2. Mappers

KOTLIN

```kotlin
// data/local/Mappers.kt
fun NoteEntity.toDomain(): Note = Note(
id = id,
title = title,
content = content,
createdAt = createdAt,
type = type,
)

fun Note.toEntity(): NoteEntity = NoteEntity(
id = id,
title = title,
content = content,
createdAt = createdAt,
updatedAt = System.currentTimeMillis(),
type = type,
)
```

Задание

В NotesApp внедрите Room для постоянного хранения заметок. Замените InMemoryNotesRepository на RoomNotesRepository, который читает данные из DAO. Все заметки должны сохраняться между запусками приложения.

Шаг 1. Зависимости и плагины

В build.gradle.kts (project) добавьте плагин KSP в plugins-блок. В build.gradle.kts (app) добавьте Room dependencies (см. теорию 6.2.8). Включите ksp { arg("room.schemaLocation", "$projectDir/schemas") } для сохранения JSON-схемы.

Шаг 2. NoteEntity и NoteTypeConverter

В data/local/NoteEntity.kt создайте Entity с полями: id (autoGenerate), title, content, created_at, updated_at (Long, default 0), type (NoteType). В data/local/NoteTypeConverter.kt создайте класс с двумя методами @TypeConverter для NoteType ↔ String (по образцу 6.2.5).

Шаг 3. NoteDao

В data/local/NoteDao.kt создайте интерфейс с методами:

1. fun observeAll(): Flow<List<NoteEntity>> — все заметки, отсортированные по created_at DESC
2. suspend fun getById(id: Long): NoteEntity?
3. fun searchByTitle(query: String): Flow<List<NoteEntity>>
4. suspend fun insert(note: NoteEntity): Long
5. suspend fun update(note: NoteEntity)
6. suspend fun deleteById(id: Long)

Шаг 4. AppDatabase

В data/local/AppDatabase.kt создайте абстрактный класс AppDatabase с аннотациями @Database (version=1, exportSchema=true) и @TypeConverters. Внутри — абстрактный метод noteDao(): NoteDao. Пока Singleton можно оставить как в примере 6.2.4 — на практике 8 мы заменим его Hilt.

Шаг 5. Миграция 1 → 2

Важно: сразу создайте version 2 и миграцию 1→2, добавляющую поле updated_at. Это нужно для демонстрации. Шаги:

1. Запустите приложение с version=1, добавьте заметку.
2. Поменяйте version на 2, добавьте поле updated_at в Entity.
3. Создайте MIGRATION_1_2 (как в примере 6.2.6).
4. Зарегистрируйте миграцию в Room.databaseBuilder.
5. Перезапустите приложение — старая заметка должна сохраниться.

Шаг 6. RoomNotesRepository

В data/RoomNotesRepository.kt создайте класс-обёртку над DAO:

```kotlin
class RoomNotesRepository(private val dao: NoteDao) {

fun observeAll(): Flow<List<Note>> =
dao.observeAll().map { list -> list.map { it.toDomain() } }

suspend fun getById(id: Long): Note? =
dao.getById(id)?.toDomain()

fun searchByTitle(query: String): Flow<List<Note>> =
dao.searchByTitle(query).map { list -> list.map { it.toDomain() } }

suspend fun add(note: Note): Long =
dao.insert(note.toEntity())

suspend fun update(note: Note) =
dao.update(note.toEntity())

suspend fun delete(id: Long) =
dao.deleteById(id)
}
```

Шаг 7. Обновление NotesViewModel

Измените NotesViewModel так, чтобы он принимал NotesRepository (interface, реализованный RoomNotesRepository) через конструктор. Метод loadNotes() замените на подписку на repo.observeAll() через collect в viewModelScope. UiState.Success обновляется при каждом изменении БД.

Шаг 8. Подключение в MainActivity

В MainActivity создайте AppDatabase, получите NoteDao, создайте RoomNotesRepository, передайте в NotesViewModel (через фабрику или напрямую — на практике 9 заменится Hilt). Запустите приложение — список должен сохраняться между запусками.

Контрольные вопросы

Ответьте письменно на следующие вопросы. Подготовьте устные ответы для защиты практической работы — преподаватель может задать любой из них или попросить пояснить код вашего решения.

1. Что такое Room? Какие три главных компонента она определяет?

2. Чем @Insert отличается от @Update? Как работает onConflict = REPLACE?

3. Почему SQL-запросы в @Query проверяются на этапе комиляции? Как это реализовано технически (KSP)?

4. Зачем нужен TypeConverter? Что произойдёт, если его не предоставить для поля- sealed class?

5. В чём разница между suspend-методом DAO и методом, возвращающим Flow? Можно ли смешивать?

6. Как Room понимает, что таблица изменилась, и эммитит новое значение в Flow?

7. Что произойдёт, если увеличить version базы без предоставления миграции? Какие есть стратегии?

8. Чем fallbackToDestructiveMigration отличается от явной миграции? Когда допустимо его использование?

9. Зачем нужен exportSchema = true? Где сохраняется схема?

10. Почему Entity и Domain-классы принято разделять? Что даёт наличие mapper-функций?

11. Можно ли использовать Room без корутин? Что нужно изменить?

12. Как протестировать DAO без устройства (в JVM)? Что такое in-memory Room?

Чек-лист сдачи работы

Перед сдачей работы преподавателю убедитесь, что выполнены все пункты ниже. Отсутствие любого пункта снижает оценку минимум на один балл.

- □ В app/build.gradle.kts добавлены room-runtime, room-ktx, ksp room-compiler
- □ В корневом build.gradle.kts подключён плагин com.google.devtools.ksp
- □ В data/local/NoteEntity.kt есть @Entity с 6 полями
- □ id — Long с @PrimaryKey(autoGenerate = true)
- □ created_at имеет индекс через indices = [Index("created_at")]
- □ В data/local/NoteTypeConverter.kt есть 2 метода @TypeConverter
- □ В data/local/NoteDao.kt есть 6 методов с правильными аннотациями
- □ observeAll() возвращает Flow, отсортированный по created_at DESC
- □ В data/local/AppDatabase.kt есть @Database с version=2
- □ Класс @TypeConverters(NoteTypeConverter::class) на уровне Database
- □ AppDatabase реализован как Singleton (companion object + @Volatile)
- □ Создана MIGRATION_1_2 с ALTER TABLE notes ADD COLUMN updated_at
- □ Миграция зарегистрирована через .addMigrations(MIGRATION_1_2)
- □ В data/RoomNotesRepository.kt есть mapper и Flow-методы
- □ NotesViewModel использует RoomNotesRepository через interface
- □ loadNotes заменён на collect по observeAll()
- □ После перезапуска приложения заметки сохраняются

Что сдавать

В результате выполнения практической работы студент предоставляет:

1. Git-коммит «Практика 6» с каталогом data/local/;

2. Скриншот каталога schemas/ с двумя JSON-файлами (v1 и v2);

3. Скриншот приложения с заметками после перезапуска (те же данные);

4. Краткий лог Logcat: TAG="NotesApp", сообщения об INSERT и observeAll.

Практика 8. Retrofit: работа с сетью

Трудоёмкость: 6 академических часов. **Форма работы:** индивидуальная, в IDE Android Studio. **Форма отчёта:** исходный код проекта + устная защита.

Цель и задачи

Цель работы — добавить в NotesApp сетевой слой: загрузка заметок с удалённого сервера, синхронизация локальной Room-БД с сервером. В качестве бэкенда используется публичный фейковый API JSONPlaceholder (https://jsonplaceholder.typicode.com) — он не требует авторизации и идеально подходит для обучения. Студент научится описывать REST-интерфейсы, настраивать OkHttp клиент, обрабатывать сетевые ошибки через sealed-обёртку NetworkResult и объединять сетевые запросы с локальным кэшем.

1. описывать REST API как Kotlin-интерфейс с аннотациями Retrofit;
2. настраивать OkHttp с интерсепторами (логирование, аутентификация, таймауты);
3. использовать suspend-функции и корутины для асинхронных запросов;
4. обрабатывать сетевые ошибки: IOException (нет сети), HttpException (4xx/5xx);
5. объединять сетевой и локальный слои через Repository pattern;
6. применять sealed NetworkResult<T> для типобезопасной обработки результата.

Теоретическая часть

8.2.1. REST API — краткое напоминание

REST (Representational State Transfer) — архитектурный стиль для HTTP-API. Основные принципы: клиент-серверная архитектура, stateless (каждый запрос самодостаточен), единый интерфейс (URL = ресурс, HTTP-метод = действие), использование стандартных кодов состояния. Главные HTTP-методы и их семантика:

Метод	Назначение	Идемпотентность	Безопасный
GET	Получить ресурс	Да	Да
POST	Создать ресурс	Нет	Нет
PUT	Заменить ресурс целиком	Да	Нет
PATCH	Частично изменить ресурс	Нет	Нет
DELETE	Удалить ресурс	Да	Нет

8.2.2. Retrofit — что это и зачем

Retrofit (от Square) — библиотека-обёртка над OkHttp, превращающая HTTP API в Kotlin/Java-интерфейс. Разработчик описывает API как интерфейс с аннотациями @GET, @POST, @PUT, @DELETE, а Retrofit генерирует реализацию на этапе выполнения. Главные преимущества: декларативное описание, типобезопасность, нативная поддержка корутин (suspend-функции), конвертеры для JSON (Moshi, GSON), простая интеграция с перехватчиками OkHttp.

8.2.3. Аннотации Retrofit

Аннотация	Назначение	Пример
`@GET("path")`	HTTP GET	`@GET("notes/{id}")`
`@POST("path")`	HTTP POST	`@POST("notes")`
`@PUT("path")`	HTTP PUT	`@PUT("notes/{id}")`
`@DELETE("path")`	HTTP DELETE	`@DELETE("notes/{id}")`
`@Path`	Подстановка в URL	`fun get(@Path("id") id: Long)`
`@Query`	Query-параметр	`@Query("q") q: String`
`@Body`	Тело запроса (JSON)	`@Body note: NoteDto`
`@Header`	Заголовок	`@Header("Auth") token: String`
`@Headers`	Статические заголовки	`@Headers("Accept: application/json")`

8.2.4. OkHttp — низкоуровневый HTTP-клиент

OkHttp — транспортный слой под Retrofit. Он отвечает за TCP-соединения, TLS, пулинг соединений, кэш, таймауты, перехватчики (Interceptors). Retrofit по умолчанию использует OkHttp, но можно передать кастомный клиент с настроенными перехватчиками. Главные настройки OkHttp:

- **connectTimeout / readTimeout / writeTimeout** — таймауты на установку соединения, чтение и запись. По умолчанию 10 секунд.
- **Interceptor** — перехватчик, видит и может модифицировать каждый запрос и ответ. Бывает application-level (добавляет заголовки) и network-level (видит редиректы).
- **Authenticator** — специальный interceptor для аутентификации (например, refresh-токена при 401).
- **Cache** — HTTP-кэш на диске, подчиняется стандартным Cache-Control заголовкам.

8.2.5. Конвертеры JSON: Moshi vs GSON

Retrofit не умеет сам парсить JSON — для этого нужен конвертер. Два самых популярных:

Свойство	Moshi	GSON
Разработчик	`Square`	`Google`
Kotlin-first	`Да`	`Нет (Java-first)`
Поддержка null-safety	`Нативная`	`Через адаптеры`

Свойство	Moshi	GSON
Размер	Меньше	Больше
Скорость	Быстрее (codegen)	Медленно (reflection)
Рекомендация для новых проектов	Да	Нет (но легаси ОК)

8.2.6. Suspend-функции и корутины

Retrofit с версии 2.6.0 нативно поддерживает корутины: метод DAO интерфейса помечается suspend, и Retrofit сделает запрос асинхронно, вернув результат через Continuation. Никаких callback-ов. Код читается как синхронный, но не блокирует поток. Запускать такие методы нужно в viewModelScope или другом CoroutineScope с подходящим Dispatcher (Dispatchers.IO переключается автоматически внутри Retrofit).

KOTLIN

```
interface NoteApi {
@GET("notes")
suspend fun list(): List<NoteDto>

@GET("notes/{id}")
suspend fun get(@Path("id") id: Long): NoteDto

@POST("notes")
suspend fun create(@Body note: NoteDto): NoteDto

@PUT("notes/{id}")
suspend fun update(@Path("id") id: Long, @Body note: NoteDto): NoteDto

@DELETE("notes/{id}")
suspend fun delete(@Path("id") id: Long): Unit
}
```

8.2.7. Обработка ошибок: HttpException и IOException

При сетевом запросе могут возникнуть два класса ошибок:

- **IOException** — нет сети, сервер недоступен, таймаут. Бросается OkHttp до получения HTTP-ответа.
- **HttpException** — сервер ответил с кодом 4xx или 5xx. Бросается Retrofit после получения ответа. Доступ к коду и телу — через exception.code() и exception.response().

В корутине обе ошибки бросаются как исключения — их нужно ловить в try-catch или оборачивать в sealed-класс NetworkResult.

8.2.8. Sealed NetworkResult — типобезопасная обработка

Идиома: обернуть результат сетевого запроса в sealed class с тремя ветками — Success, Error (серверная ошибка с кодом), NetworkError (нет соединения). Composable делает when по этому классу и показывает разный UI.

```kotlin
sealed interface NetworkResult<out T> {
data class Success<T>(val data: T) : NetworkResult<T>
data class Error(val code: Int, val message: String) : NetworkResult<Nothing>
data class NetworkError(val throwable: Throwable) : NetworkResult<Nothing>
}

suspend inline fun <T> safeApiCall(crossinline block: suspend () -> T): NetworkResult<T> =
try {
NetworkResult.Success(block())
} catch (e: HttpException) {
NetworkResult.Error(e.code(), e.message())
} catch (e: IOException) {
NetworkResult.NetworkError(e)
}
```

8.2.9. Логирование через HttpLoggingInterceptor

Для отладки сетевых запросов используется HttpLoggingInterceptor из библиотеки com.squareup.okhttp3:logging-interceptor. В режиме Level.BODY логирует полный запрос и ответ, включая тело. В release-сборке нужно ставить Level.NONE или BASIC — иначе в логи утекут чувствительные данные.

```kotlin
val logging = HttpLoggingInterceptor().apply {
level = if (BuildConfig.DEBUG) HttpLoggingInterceptor.Level.BODY
else HttpLoggingInterceptor.Level.NONE
}

val client = OkHttpClient.Builder()
.addInterceptor(logging)
.connectTimeout(30, TimeUnit.SECONDS)
.readTimeout(30, TimeUnit.SECONDS)
.build()

val retrofit = Retrofit.Builder()
.baseUrl("https://jsonplaceholder.typicode.com/")
.client(client)
.addConverterFactory(MoshiConverterFactory.create())
.build()

val api: NoteApi = retrofit.create(NoteApi::class.java)
```

8.2.10. Single Source of Truth + offline cache

Современный паттерн — Repository комбинирует сеть и БД: UI всегда читает из Room (быстро, офлайн), а Repository в фоне синхронизирует с сервером. Алгоритм:

1. UI подписан на dao.observeAll() — мгновенно получает закэшированные данные.
2. Repository в фоне делает api.list().
3. При успехе Repository записывает результат в БД через dao.insertAll().
4. Room автоматически эммитит новое значение в Flow, UI обновляется.
5. При ошибке сети Repository логирует её, но UI продолжает показывать кэш. Опционально показывается Snackbar «Нет соединения».

> **Single Source of Truth**
>
> UI читает ТОЛЬКО из локальной БД. Сеть обновляет БД, а БД обновляет UI. Это даёт мгновенный отклик, офлайн-режим и predictable behavior. Никогда не комбинируйте в UI подписку на Network и Room одновременно — будет гонка данных.

Примеры кода

Пример 8.1. Полный API интерфейс для JSONPlaceholder

KOTLIN

```
// data/remote/NoteDto.kt
@JsonClass(generateAdapter = true)
data class NoteDto(
val id: Long?,
val title: String,
val body: String,
val userId: Long?,
)

// data/remote/NoteApi.kt
interface NoteApi {
@GET("posts")
suspend fun listPosts(): List<NoteDto>

@GET("posts/{id}")
suspend fun getPost(@Path("id") id: Long): NoteDto

@POST("posts")
suspend fun createPost(@Body post: NoteDto): NoteDto

@PUT("posts/{id}")
suspend fun updatePost(@Path("id") id: Long, @Body post: NoteDto): NoteDto

@DELETE("posts/{id}")
suspend fun deletePost(@Path("id") id: Long)
}
```

Пример 8.2. RetrofitService — Singleton-фабрика

KOTLIN

```
// data/remote/RetrofitService.kt
object RetrofitService {
private const val BASE_URL = "https://jsonplaceholder.typicode.com/"

private val logging = HttpLoggingInterceptor().apply {
level = if (BuildConfig.DEBUG) HttpLoggingInterceptor.Level.BODY
else HttpLoggingInterceptor.Level.NONE
}

private val client = OkHttpClient.Builder()
.addInterceptor(logging)
.connectTimeout(30, TimeUnit.SECONDS)
.readTimeout(30, TimeUnit.SECONDS)
.build()

private val moshi = Moshi.Builder().build()

private val retrofit = Retrofit.Builder()
.baseUrl(BASE_URL)
.client(client)
.addConverterFactory(MoshiConverterFactory.create(moshi))
.build()

val noteApi: NoteApi by lazy { retrofit.create(NoteApi::class.java) }
}
```

Пример 8.3. SyncedRepository — сеть + БД

KOTLIN

```
class SyncedNotesRepository(
private val dao: NoteDao,
private val api: NoteApi,
) {
fun observeAll(): Flow<List<Note>> =
dao.observeAll().map { list -> list.map { it.toDomain() } }

suspend fun refresh(): NetworkResult<Unit> = safeApiCall {
val posts = api.listPosts()
val entities = posts.map { it.toEntity() }
dao.insertAll(entities)
}

suspend fun add(note: Note): NetworkResult<Long> = safeApiCall {
val dto = note.toDto()
val created = api.createPost(dto)
val entity = created.toEntity(type = NoteType.Text)
dao.insert(entity)
}
}
```

Задание

В NotesApp добавьте сетевой слой: синхронизация локальных заметок с JSONPlaceholder. Главная задача — реализовать offline-first паттерн: UI всегда читает из Room, а Repository в фоне обновляет БД из сети. Также реализуйте создание заметки через POST-запрос.

Шаг 1. Зависимости

Добавьте в app/build.gradle.kts:

```kotlin
dependencies {
// Retrofit
implementation("com.squareup.retrofit2:retrofit:2.11.0")
implementation("com.squareup.retrofit2:converter-moshi:2.11.0")
implementation("com.squareup.okhttp3:logging-interceptor:4.12.0")

// Moshi
implementation("com.squareup.moshi:moshi:1.15.1")
ksp("com.squareup.moshi:moshi-kotlin-codegen:1.15.1")
}
```

Шаг 2. NoteDto

В data/remote/NoteDto.kt создайте data class для ответа сервера JSONPlaceholder (endpoint /posts):

```kotlin
@JsonClass(generateAdapter = true)
data class NoteDto(
val id: Long?,
val title: String,
val body: String,
val userId: Long?,
)
```

Также добавьте mapper-функции: NoteDto.toEntity() (создаёт NoteEntity с type=Text) и Note.toDto().

Шаг 3. NoteApi

В data/remote/NoteApi.kt опишите интерфейс с методами (используйте path «posts» для совместимости с JSONPlaceholder):

1. suspend fun listPosts(): List<NoteDto> — GET posts
2. suspend fun getPost(@Path("id") id: Long): NoteDto — GET posts/{id}
3. suspend fun createPost(@Body post: NoteDto): NoteDto — POST posts
4. suspend fun updatePost(@Path("id") id: Long, @Body post: NoteDto): NoteDto — PUT posts/{id}
5. suspend fun deletePost(@Path("id") id: Long) — DELETE posts/{id}

Шаг 4. NetworkResult

В data/remote/NetworkResult.kt создайте sealed interface с тремя состояниями (Success, Error, NetworkError) и функцию-обёртку safeApiCall (как в примере 7.2.8).

Шаг 5. RetrofitService

В data/remote/RetrofitService.kt создайте object Singleton с базовым URL «https://jsonplaceholder.typicode.com/», OkHttp с логированием (BASIC уровень — body слишком шумный), таймаутами 30 сек, Moshi-конвертером. Опубликуйте свойство noteApi: NoteApi.

Шаг 6. SyncedNotesRepository

В data/SyncedNotesRepository.kt создайте класс, принимающий NoteDao и NoteApi. Реализуйте методы:

1. fun observeAll(): Flow<List<Note>> — подписка на DAO (как в практике 7).

2. suspend fun refresh(): NetworkResult<Unit> — загружает posts с сервера, конвертирует в NoteEntity, вставляет в БД через dao.insertAll(list) (нужно добавить этот метод в DAO).

3. suspend fun add(note: Note): NetworkResult<Long> — делает POST на сервер, при успехе вставляет в БД.

4. suspend fun delete(id: Long): NetworkResult<Unit> — DELETE на сервер (он вернёт 200 даже если не существует — особенность JSONPlaceholder), затем удаляет из БД.

Все сетевые вызовы оборачиваются в safeApiCall { ... }. При ошибке — возвращается NetworkResult.Error или NetworkResult.NetworkError, локальная БД не модифицируется.

Шаг 7. Обновление NotesViewModel

В NotesViewModel добавьте метод refresh(), который:

1. Вызывает repo.refresh().

2. При Success показывает Snackbar «Обновлено: *N* заметок».

3. При Error показывает Snackbar «Ошибка сервера: *code*».

4. При NetworkError показывает Snackbar «Нет соединения с интернетом».

5. Параллельно с refresh запускает флаг isRefreshing = true (можно через отдельное StateFlow), который показывает ProgressBar в TopAppBar.

Шаг 8. UI: иконка refresh в TopAppBar

В NotesListScreen добавьте action-иконку Icons.Default.Refresh в TopAppBar. По клику вызывается viewModel.refresh(). Пока isRefreshing = true — иконка заменяется на CircularProgressIndicator размером 24 dp.

Шаг 9. Проверка offline

Отключите интернет на эмуляторе (или в Airplane Mode). Запустите приложение, нажмите Refresh — должно показаться Snackbar «Нет соединения», но уже загруженные заметки остаются в списке. Включите интернет, нажмите Refresh — должны подгрузиться новые заметки с JSONPlaceholder.

Контрольные вопросы

Ответьте письменно на следующие вопросы. Подготовьте устные ответы для защиты практической работы — преподаватель может задать любой из них или попросить пояснить код вашего решения.

1. Что такое REST? Назовите 5 основных HTTP-методов и их семантику.

2. Чем Retrofit отличается от «голого» OkHttp? Зачем нужен слой Retrofit поверх OkHttp?

3. Какие аннотации Retrofit вы знаете? Чем @Path отличается от @Query?

4. Что такое suspend-функция в Retrofit-интерфейсе? Как она выполняется под капотом?

5. Чем HttpException отличается от IOException? Какие ситуации приводят к каждой из них?

6. Зачем нужен HttpLoggingInterceptor? Какие уровни логирования он поддерживает?

7. Почему в release-сборке нужно уменьшать уровень логирования? Какие данные могут утечь?

8. Что такое NetworkResult? Зачем оборачивать результат в sealed class вместо try-catch в каждом вызове?

9. Опишите паттерн Single Source of Truth. Как он применяется в Repository с сетью и БД?

10. Что такое offline-first архитектура? Какие у неё преимущества?

11. Зачем нужны таймауты в OkHttp? Что произойдёт, если их не установить?

12. Чем Moshi лучше GSON? Зачем нужен codegen (KSP) для Moshi?

Чек-лист сдачи работы

Перед сдачей работы преподавателю убедитесь, что выполнены все пункты ниже. Отсутствие любого пункта снижает оценку минимум на один балл.

- □ В app/build.gradle.kts добавлены retrofit, converter-moshi, logging-interceptor, moshi, moshi-kotlin-codegen
- □ В data/remote/NoteDto.kt есть data class с @JsonClass(generateAdapter = true)
- □ В data/remote/NoteApi.kt есть 5 методов с правильными аннотациями
- □ В data/remote/NetworkResult.kt есть sealed interface с 3 ветками
- □ Реализована функция safeApiCall, ловящая HttpException и IOException
- □ В data/remote/RetrofitService.kt есть Singleton с baseUrl jsonplaceholder
- □ HttpLoggingInterceptor с уровнем BASIC для debug, NONE для release
- □ OkHttp имеет connectTimeout и readTimeout 30 секунд
- □ В data/SyncedNotesRepository.kt есть методы observeAll, refresh, add, delete
- □ DAO дополнен методом insertAll(entities: List)
- □ NotesViewModel.refresh() обновляет флаг isRefreshing
- □ TopAppBar содержит иконку Refresh, заменяемую на CircularProgressIndicator
- □ Snackbar показывает разные сообщения для Success/Error/NetworkError
- □ Offline-режим: при отсутствии сети показывается кэш + Snackbar

Что сдавать

В результате выполнения практической работы студент предоставляет:

1. Git-коммит «Практика 7» с каталогом data/remote/;

2. Скриншот Logcat с полным запросом и ответом (BASIC level) на GET /posts;

3. Скриншот приложения: список подгружен с JSONPlaceholder (10+ постов);

4. Скриншот Snackbar «Нет соединения» при отключенном интернете.

Практика 9. Dagger Hilt: внедрение зависимостей

Трудоёмкость: 6 академических часов. **Форма работы:** индивидуальная, в IDE Android Studio. **Форма отчёта:** исходный код проекта + устная защита.

Цель и задачи

Цель работы — внедрить Dagger Hilt в NotesApp и избавиться от ручного создания зависимостей. После этой практики все классы (Repository, ViewModel, Api, Database) получают зависимости через конструктор, а Hilt собирает граф объектов автоматически. Это последний инфраструктурный шаг перед тематическими практиками (изображения, интероп, рефлексия).

1. объяснить, что такое Dependency Injection и зачем он нужен;
2. различать Dagger, Hilt и их компоненты;
3. использовать @HiltAndroidApp, @AndroidEntryPoint, @Inject;
4. создавать модули: @Module, @InstallIn, @Provides, @Binds;
5. применять скоупы: @Singleton, @ViewModelScoped;
6. инжектить ViewModel через hiltViewModel().

Теоретическая часть

9.2.1. Что такое Dependency Injection и зачем

Dependency Injection (DI) — паттерн, при котором объект не создаёт свои зависимости сам, а получает их извне (через конструктор, поле или setter). Пример без DI: класс A создаёт B, B создаёт C, C создаёт D — менять реализации нужно в десятке мест, тестировать невозможно. С DI: класс A объявляет «мне нужен B», контейнер DI находит (или создаёт) B и передаёт A. Тесты тривиально подменяют B на mock.

KOTLIN

```
// Без DI — класс сам создаёт зависимости
class NotesViewModel {
private val repo = SyncedNotesRepository(
dao = AppDatabase.getInstance(app).noteDao(),
api = RetrofitService.noteApi,
)
}

// С DI — зависимости приходят через конструктор
class NotesViewModel @Inject constructor(
private val repo: NotesRepository, // интерфейс
) : ViewModel()
```

9.2.2. Dagger vs Hilt

Dagger — библиотека DI от Square (теперь Google), основанная на генерации кода на этапе компиляции (через KSP / kapt). Очень быстрая в рантайме, типобезопасная, но сложная в настройке: для

Android-приложения нужно вручную создавать компоненты для Application, Activity, Fragment, ViewModel. **Hilt** — надстройка над Dagger от Google, стандартизирующая Dagger для Android. Hilt предопределяет компоненты и скоупы, уменьшая boilerplate в 5–10 раз. Для новых Android-проектов рекомендуется Hilt; чистый Dagger имеет смысл только в нестандартных сценариях (multi-module gradle с динамическими фичами).

9.2.3. Главные аннотации Hilt

Аннотация	Назначение
`@HiltAndroidApp`	Помечает Application класс — точка входа Hilt. Обязательно для всех Hilt-приложений.
`@AndroidEntryPoint`	Помечает Activity / Fragment / View / Service — Hilt инжектит в них зависимости.
`@Inject`	Constructor: Hilt знает как создать класс. Field: Hilt заполнит поле.
`@Module`	Класс-модуль, содержащий @Provides/@Binds методы.
`@InstallIn`	Указывает, в какой компонент Hilt установить модуль.
`@Provides`	Метод, возвращающий зависимость. Используется когда класс не ваш (Retrofit, Room).
`@Binds`	Связывает интерфейс с реализацией. Только для ваших классов.
`@Singleton`	Скоуп — единственный экземпляр на всё приложение.
`@ViewModelScoped`	Скоуп — один экземпляр на ViewModel.
`@Qualifier`	Пользовательская аннотация для различения зависимостей одного типа.
`@ApplicationContext`	Qualifier — передаёт Application Context.

9.2.4. Компоненты и скоупы Hilt

Hilt предопределяет иерархию компонентов, привязанных к Android-компонентам. Каждый компонент имеет свой скоуп — зависимость, помеченная этим скоупом, живёт столько же, сколько компонент.

Компонент	Скоуп	Создаётся для	Видит зависимости из
`SingletonComponent`	`@Singleton`	Application	—
`ActivityRetainedComponent`	`@ActivityRetainedScoped`	Activity (переживает поворот)	Singleton
`ActivityComponent`	`@ActivityScoped`	Activity	Singleton, ActivityRetained
`ViewModelComponent`	`@ViewModelScoped`	ViewModel	Singleton, ActivityRetained

Компонент	Скоуп	Создаётся для	Видит зависимости из
`FragmentComponent`	`@FragmentScoped`	Fragment	Singleton, Activity, Fragment
`ViewComponent`	`@ViewScoped`	Custom View	Singleton, Activity, View
`ServiceComponent`	`@ServiceScoped`	Service	Singleton

Самые частые скоупы: @Singleton (Database, Retrofit, OkHttp — один на приложение), @ViewModelScoped (Repository, привязанный к ViewModel), @ActivityScoped (что-то живущее пока экран активен). Подавляющее большинство зависимостей — @Singleton.

9.2.5. @Module, @InstallIn, @Provides, @Binds

Модуль — это класс, который учит Hilt создавать объекты, которые он не может создать сам (например, Retrofit из сторонней библиотеки) или связывать интерфейс с реализацией. Два варианта:

• **@Provides** — метод, который сам создаёт и возвращает объект. Используется для классов не из вашего кода (Retrofit, OkHttpClient, Room Database).

• **@Binds** — абстрактный метод, который только связывает интерфейс с реализацией. Используется когда реализация уже имеет @Inject constructor.

KOTLIN

```
// @Provides — для сторонних объектов
@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {

@Provides
@Singleton
fun provideOkHttp(): OkHttpClient = OkHttpClient.Builder()
.addInterceptor(HttpLoggingInterceptor().apply {
level = HttpLoggingInterceptor.Level.BASIC
})
.build()

@Provides
@Singleton
fun provideRetrofit(client: OkHttpClient): Retrofit = Retrofit.Builder()
.baseUrl("https://jsonplaceholder.typicode.com/")
.client(client)
.addConverterFactory(MoshiConverterFactory.create())
.build()

@Provides
@Singleton
fun provideNoteApi(retrofit: Retrofit): NoteApi =
retrofit.create(NoteApi::class.java)
}

// @Binds — для своих интерфейсов
@Module
@InstallIn(SingletonComponent::class)
abstract class RepositoryModule {

@Binds
@Singleton
abstract fun bindNotesRepository(impl: SyncedNotesRepository): NotesRepository
}
```

Правило: @Provides — для объектов, которые нельзя инжектить через конструктор (нет @Inject). @Binds — для интерфейсов, у реализации которых есть @Inject constructor. Это быстрее (меньше генерируемого кода).

9.2.6. @HiltViewModel и hiltViewModel()

Для инъекции в ViewModel используется специальная аннотация @HiltViewModel (вместо ручной фабрики). Hilt автоматически создаёт ViewModel с нужными зависимостями. В Compose получить такой ViewModel можно через функцию hiltViewModel() из библиотеки androidx.hilt:hilt-navigation-compose.

KOTLIN

```
@HiltViewModel
class NotesViewModel @Inject constructor(
private val repo: NotesRepository,
) : ViewModel() { /* ... */ }

@Composable
fun NotesListScreen() {
val viewModel: NotesViewModel = hiltViewModel()
// ...
}
```

9.2.7. @ApplicationContext и другие qualifiers

Иногда нужно передать Context в зависимость. Hilt различает @ApplicationContext (Application context — живёт вечно) и @ActivityContext (Activity context — привязан к жизненному циклу Activity). Используйте @ApplicationContext когда возможен — он не приводит к утечкам памяти. @ActivityContext — только когда нужен именно Activity context (например, для Inflate).

```kotlin
@Module
@InstallIn(SingletonComponent::class)
object DatabaseModule {

@Provides
@Singleton
fun provideAppDatabase(@ApplicationContext context: Context): AppDatabase =
Room.databaseBuilder(context, AppDatabase::class.java, "notes.db")
.addMigrations(MIGRATION_1_2)
.build()

@Provides
@Singleton
fun provideNoteDao(db: AppDatabase): NoteDao = db.noteDao()
}
```

9.2.8. Зависимости и плагины

В build.gradle.kts (project) добавьте плагин Hilt:

```kotlin
plugins {
id("com.google.dagger.hilt.android") version "2.51.1" apply false
}
```

В build.gradle.kts (app) добавьте плагин и зависимости:

```kotlin
plugins {
id("com.google.dagger.hilt.android")
kotlin("kapt") // или id("com.google.devtools.ksp") — Hilt пока не поддерживает KSP
полностью
}

dependencies {
implementation("com.google.dagger:hilt-android:2.51.1")
kapt("com.google.dagger:hilt-compiler:2.51.1")
implementation("androidx.hilt:hilt-navigation-compose:1.2.0")
}
```

Примеры кода

Пример 9.1. Application класс с @HiltAndroidApp

KOTLIN

```
// NotesApp.kt
@HiltAndroidApp
class NotesApp : Application() {
// Точка входа Hilt. Дополнительный код обычно не нужен.
}
```

Пример 9.2. MainActivity с @AndroidEntryPoint

KOTLIN

```
@AndroidEntryPoint
class MainActivity : ComponentActivity() {
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContent {
NotesAppTheme {
RootScreen() // навигация
}
}
}
}
```

Пример 9.3. Полный набор модулей

KOTLIN

```
@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {
@Provides @Singleton
fun provideOkHttp(): OkHttpClient = /* ... */

@Provides @Singleton
fun provideRetrofit(client: OkHttpClient): Retrofit = /* ... */

@Provides @Singleton
fun provideNoteApi(retrofit: Retrofit): NoteApi =
retrofit.create(NoteApi::class.java)
}

@Module
@InstallIn(SingletonComponent::class)
object DatabaseModule {
@Provides @Singleton
fun provideDatabase(@ApplicationContext ctx: Context): AppDatabase = /* ... */

@Provides @Singleton
fun provideNoteDao(db: AppDatabase): NoteDao = db.noteDao()
}

@Module
@InstallIn(SingletonComponent::class)
abstract class RepositoryModule {
@Binds @Singleton
abstract fun bindNotesRepository(impl: SyncedNotesRepository): NotesRepository
}
```

Пример 9.4. @HiltViewModel с инжектом репозитория

```kotlin
@HiltViewModel
class NotesViewModel @Inject constructor(
private val repo: NotesRepository,
) : ViewModel() {

private val _uiState = MutableStateFlow<NotesUiState>(NotesUiState.Loading)
val uiState: StateFlow<NotesUiState> = _uiState.asStateFlow()

private val _isRefreshing = MutableStateFlow(false)
val isRefreshing: StateFlow<Boolean> = _isRefreshing.asStateFlow()

init {
viewModelScope.launch {
repo.observeAll().collect { notes ->
_uiState.value =
if (notes.isEmpty()) NotesUiState.Empty
else NotesUiState.Success(notes)
}
}
}

fun refresh() {
viewModelScope.launch {
_isRefreshing.value = true
val result = repo.refresh()
_isRefreshing.value = false
// показать snackbar по результату
}
}
}
```

Задание

Внедрите Hilt в NotesApp: уберите все ручные создания Retrofit, Room, Repository. Замените создание ViewModel через фабрику на hiltViewModel(). Все зависимости должны собираться графом Hilt.

Шаг 1. Плагины и зависимости

Добавьте Hilt в build.gradle.kts (project) и (app), как описано в 8.2.8. Не забудьте kapt или KSP (последний поддерживается с Hilt 2.48+ через экспериментальный флаг).

Шаг 2. NotesApp класс

В файле NotesApp.kt создайте класс NotesApp наследник Application, помеченный @HiltAndroidApp. Зарегистрируйте его в AndroidManifest.xml через атрибут android:name=".NotesApp" в теге <application>.

Шаг 3. MainActivity с @AndroidEntryPoint

Пометьте MainActivity аннотацией @AndroidEntryPoint. Уберите весь код создания AppDatabase / RetrofitService / Repository из onCreate — теперь это делает Hilt.

Шаг 4. NetworkModule

В файле di/NetworkModule.kt создайте object-модуль с тремя @Provides @Singleton методами: provideOkHttp(), provideRetrofit(client), provideNoteApi(retrofit). Удалите старый RetrofitService object — его заменяет модуль.

Шаг 5. DatabaseModule

В файле di/DatabaseModule.kt создайте object-модуль с двумя @Provides @Singleton методами: provideAppDatabase(@ApplicationContext ctx) и provideNoteDao(db). Включите миграцию MIGRATION_1_2 в builder. Удалите Singleton companion object из AppDatabase — Hilt теперь управляет жизненным циклом.

Шаг 6. RepositoryModule и интерфейс

Создайте интерфейс NotesRepository с теми же методами, что у SyncedNotesRepository (observeAll, refresh, add, delete, getById, search). Сделайте SyncedNotesRepository реализацией этого интерфейса, помеченной @Inject constructor с параметрами dao: NoteDao и api: NoteApi. В файле di/RepositoryModule.kt создайте abstract класс-модуль с методом @Binds, связывающим интерфейс с реализацией.

Шаг 7. @HiltViewModel

Пометьте NotesViewModel аннотацией @HiltViewModel. Добавьте @Inject constructor с параметром repo: NotesRepository. Уберите все ручные фабрики.

Шаг 8. hiltViewModel() в Composable

В NotesListScreen замените viewModel<NotesViewModel>() на hiltViewModel<NotesViewModel>() (нужна зависимость androidx.hilt:hilt-navigation-compose). Аналогично в NoteDetailScreen и SettingsScreen (если у них появятся ViewModel).

Шаг 9. Проверка работоспособности

Запустите приложение. Убедитесь, что всё работает как раньше: список загружается, FAB добавляет заметку, Refresh синхронизирует с сетью, заметки сохраняются между запусками. В Logcat не должно быть ошибок Hilt.

Контрольные вопросы

Ответьте письменно на следующие вопросы. Подготовьте устные ответы для защиты практической работы — преподаватель может задать любой из них или попросить пояснить код вашего решения.

1. Что такое Dependency Injection? Сформулируйте определение и приведите пример «без DI» и «с DI».

2. Чем Dagger отличается от Hilt? Почему для Android-проектов рекомендуют Hilt?

3. Какие аннотации Hilt вы знаете? Назовите минимум 6 с назначением.

4. Что такое @HiltAndroidApp? Зачем он нужен и где размещается?

5. Чем @Provides отличается от @Binds? Когда использовать каждый?

6. Что такое @InstallIn? Какие компоненты Hilt предопределяет?

7. Чем @Singleton отличается от @ViewModelScoped? Приведите пример использования каждого.

8. Зачем нужен @ApplicationContext? Чем отличается от @ActivityContext?

9. Как Hilt создаёт ViewModel? Что делает @HiltViewModel?

10. Чем hiltViewModel() отличается от viewModel()?

11. Что такое Qualifier? Приведите пример, когда он необходим.

12. Как протестировать класс с DI? Что такое @HiltAndroidTest и @UninstallModules?

Чек-лист сдачи работы

Перед сдачей работы преподавателю убедитесь, что выполнены все пункты ниже. Отсутствие любого пункта снижает оценку минимум на один балл.

- □ В build.gradle.kts (project) добавлен плагин com.google.dagger.hilt.android
- □ В build.gradle.kts (app) добавлены hilt-android, kapt hilt-compiler, hilt-navigation-compose
- □ В NotesApp.kt класс с @HiltAndroidApp
- □ В AndroidManifest.xml тег application имеет android:name=".NotesApp"
- □ MainActivity помечен @AndroidEntryPoint
- □ В di/NetworkModule.kt есть object @Module @InstallIn(SingletonComponent::class)
- □ NetworkModule предоставляет OkHttpClient, Retrofit, NoteApi (@Singleton)
- □ Старый RetrofitService object удалён
- □ В di/DatabaseModule.kt есть object @Module @InstallIn(SingletonComponent::class)
- □ DatabaseModule предоставляет AppDatabase и NoteDao (@Singleton, @ApplicationContext)
- □ Singleton companion object из AppDatabase удалён
- □ Создан interface NotesRepository с теми же методами что у SyncedNotesRepository
- □ SyncedNotesRepository помечен @Inject constructor с dao и api параметрами
- □ В di/RepositoryModule.kt есть abstract class с @Binds методом
- □ NotesViewModel помечен @HiltViewModel и @Inject constructor
- □ В Composable используется hiltViewModel() вместо viewModel()
- □ Приложение запускается без ошибок Hilt в Logcat

Что сдавать

В результате выполнения практической работы студент предоставляет:

1. Git-коммит «Практика 8» с каталогом di/ и обновлёнными Application/Activity/ViewModel;
2. Скриншот Logcat: нет Hilt/Dagger ошибок при запуске;
3. Скриншот приложения — список заметок работает как раньше;
4. Краткий отчет: какие классы стали @Inject, какие модули созданы.

Практика 10. Glide: загрузка изображений

Трудоёмкость: 6 академических часов. **Форма работы:** индивидуальная, в IDE Android Studio. **Форма отчёта:** исходный код проекта + устная защита.

Цель и задачи

Цель работы — добавить в NotesApp поддержку изображений: заметки могут содержать прикреплённые картинки, загружаемые с URL. В качестве библиотеки используем **Glide** — проверенную временем библиотеку от Bump Technologies (ныне Google), которая де-факто стала стандартом для загрузки изображений в Android. Студент научится настраивать Glide через AppGlideModule, использовать RequestOptions для трансформаций, встраивать ImageView в Compose через AndroidView, писать собственные трансформации и корректно обрабатывать пустые/ошибочные состояния.

1. объяснить, почему нельзя загружать изображения через URL.openConnection() напрямую в UI-потоке;

2. использовать Glide API: Glide.with(...).load(url).into(view);

3. применять RequestOptions для placeholder, error, transform, override;

4. создавать кастомный AppGlideModule для настройки кэша;

5. встраивать Glide в Compose через AndroidView и фабрику GlideImage;

6. применять встроенные трансформации (CircleCrop, RoundedCorners, CenterCrop) и писать кастомные;

7. работать с BitmapPool и управлением памятьтью;

8. строить сетки изображений через LazyVerticalStaggeredGrid.

Теоретическая часть

10.2.1. Почему не URL.downloadUrl

Самая частая ошибка начинающих — загрузка изображения через URL(url).openStream() в UI-потоке. Это приводит к freeze приложения на время загрузки, к NetworkOnMainThreadException и к полной неработоспособности в офлайне. Кроме того, ручная загрузка не решает: кэширования в памяти и на диске, downsampling (чтобы не OOM на больших bitmap-ах), переиспользования View в списках, корректной отмены при скроллинге. Поэтому для изображений всегда используют специализированные библиотеки.

10.2.2. Glide — обзор библиотеки

Glide — библиотека загрузки изображений для Android, разработанная Bump Technologies и в 2014 году приобретённая Google. С тех пор Glide входит в stack большинства Android-приложений и регулярно обновляется. На момент написания практикума актуальная версия — Glide 4.16+, поддерживающая Kotlin, корутины и Compose через официальный модуль glide-compose.

Свойство	Glide 4.x
Разработчик	`Google (Bump Technologies)`
Язык	`Java (с Kotlin extensions)`
Compose-нативная поддержка	`Через glide-compose (AsyncGlideImage)`
API	`Glide.with(ctx).load(url).into(view)`
Кэш в памяти	`Да (LruResourceCache + BitmapPool)`
Кэш на диске	`Да (DiskCacheStrategy)`
Поддержка GIF/animated WebP	`Да`
Трансформации	`CircleCrop, RoundedCorners, CenterCrop, кастомные`
Размер библиотеки	`~500 KB core + ~200 KB okhttp-integration`
Профессиональная замена	`Coil (если нужна нативная корутиная поддержка)`

Главное преимущество Glide — зрелость. Библиотека существует с 2013 года, хорошо документирована, имеет большое сообщество. В этой практике мы используем Glide 4.x с интеграцией OkHttp (вместо дефолтного HttpURLConnection) — это даёт единый HTTP-стек с Retrofit, единую конфигурацию таймаутов и логирования.

10.2.3. Установка и зависимости

В app/build.gradle.kts добавьте:

```kotlin
dependencies {
// Glide core
implementation("com.github.bumptech.glide:glide:4.16.0")
kapt("com.github.bumptech.glide:compiler:4.16.0") // генерация GeneratedAppGlideModule

// Интеграция с OkHttp — заменяет дефолтный HttpURLConnection
implementation("com.github.bumptech.glide:okhttp3-integration:4.16.0")

// Compose-интеграция (опционально, но удобно)
implementation("com.github.bumptech.glide:compose:1.0.0-beta01")

// KSP пока не поддерживается Glide-генератором — используем kapt
}

plugins {
id("kotlin-kapt") // или id("com.google.devtools.ksp") если перейдёте на Coil
}
```

В settings.gradle.kts репозитарий JitPack не нужен — Glide есть в Maven Central. После добавления зависимостей сделайте Gradle Sync.

10.2.4. Базовый API — Glide.with().load().into()

Glide построен вокруг fluent API: Glide.with(context) возвращает RequestManager, привязанный к жизненному циклу context-а. .load(url) возвращает RequestBuilder<Drawable>, а .into(view) запускает

запрос. Главное правило: Glide.with(...) принимает Context, Activity, Fragment или View — Glide автоматически привяжётся к их жизненному циклу и отменит запрос при уничтожении.

KOTLIN

```
// В Activity / Fragment
Glide.with(this)
.load("https://example.com/photo.jpg")
.placeholder(R.drawable.placeholder)
.error(R.drawable.error_image)
.into(imageView)

// В ViewHolder
Glide.with(itemView)
.load(item.imageUrl)
.circleCrop()
.into(avatarView)

// В Compose через AndroidView
AndroidView(
factory = { ctx -> ImageView(ctx) },
update = { iv ->
Glide.with(iv)
.load(url)
.placeholder(R.drawable.placeholder)
.error(R.drawable.error_image)
.into(iv)
},
modifier = Modifier.size(64.dp),
)
```

Важно про with()

Никогда не передавайте applicationContext в Glide.with(...) если потом не вызываете явно onDestroy() — запрос переживёт Activity и приведёт к утечке. Передавайте Activity / Fragment / View — Glide сам подпишется на их жизненный цикл через LifecycleObserver.

10.2.5. RequestOptions — placeholder, error, transform

RequestOptions — immutable-объект, описывающий параметры запроса. Можно переиспользовать между вызовами. Главные методы:

Метод	Назначение	Пример
`placeholder(drawable)`	Показывать пока грузится	`.placeholder(R.drawable.ph)`
`error(drawable)`	Показывать при ошибке	`.error(R.drawable.err)`
`fallback(drawable)`	Показывать если model == null	`.fallback(R.drawable.fb)`
`circleCrop()`	Круглая обрезка	`.circleCrop()`
`centerCrop()`	Center-crop (заполняет View)	`.centerCrop()`
`fitCenter()`	Fit-center (вписывает)	`.fitCenter()`
`roundedCorners(radius)`	Скругление углов	`.roundedCorners(16)`

Метод	Назначение	Пример
`override(w, h)`	Загрузить с указанным разрешением	`.override(400, 400)`
`diskCacheStrategy(s)`	Стратегия кэширования на диске	`.diskCacheStrategy(ALL)`
`skipMemoryCache(true)`	Не кэшировать в памяти	`.skipMemoryCache(true)`
`transition(t)`	Анимация появления	`.transition(DrawableTransitionOptions.withCrossFade())`
`timeout(ms)`	Таймаут загрузки	`.timeout(10000)`

KOTLIN

```
// Создаём переиспользуемые RequestOptions
val avatarOptions = RequestOptions()
.circleCrop()
.placeholder(R.drawable.ic_avatar_placeholder)
.error(R.drawable.ic_avatar_error)
.override(256, 256)

// Используем
Glide.with(this)
.load(user.avatarUrl)
.apply(avatarOptions)
.into(avatarImageView)
```

10.2.6. DiskCacheStrategy — управление дисковым кэшем

Glide кэширует на диске по-разному в зависимости от стратегии:

Стратегия	Что кэширует	Когда использовать
`ALL`	И оригинал, и результат	По умолчанию. Универсально
`NONE`	Ничего	Если изображение меняется при каждом запросе
`SOURCE`	Только оригинал	Если вы делаете кастомные трансформации и не хотите кэшировать результат
`AUTOMATIC`	Эвристика (по умолчанию)	Оставьте как есть для большинства случаев
`DATA`	Синоним SOURCE	—
`RESOURCE`	Только результат	Если оригинал не нужен (например, всегда circleCrop)

10.2.7. AppGlideModule — глобальная конфигурация

Для настройки глобальных параметров Glide (размер кэша, BitmapPool, OkHttp-интеграция, регистрация кастомных ModelLoader-ов) создаётся класс-наследник AppGlideModule с аннотацией @GlideModule. Glide находит его через кодогенерацию (kapt) и использует автоматически — никаких явных вызовов не нужно.

```kotlin
// NotesAppGlideModule.kt
@GlideModule
class NotesAppGlideModule : AppGlideModule() {

override fun applyOptions(context: Context, builder: GlideBuilder) {
// Кэш в памяти: 25% доступной heap
val memoryCache = MemorySizeCalculator.Builder(context)
.setMemoryCacheScreens(2f)
.build()
builder.setMemoryCache(LruResourceCache(memoryCache.memoryCacheSize.toLong()))

// Кэш на диске: 250 MB в подпапке image_cache
val diskCache = DiskLruCacheFactory(
context.cacheDir.absolutePath + "/image_cache",
250L * 1024 * 1024, // 250 MB
)
builder.setDiskCache(diskCache)

// Логирование (только в debug)
if (BuildConfig.DEBUG) {
builder.setLogLevel(Log.VERBOSE)
}
}

// Если используете OkHttp-интеграцию — этот метод нужен чтобы
// Glide не пытался регистрировать дефолтный HttpURLConnection
override fun isManifestParsingEnabled(): Boolean = false

override fun registerComponents(context: Context, glide: Glide, registry: Registry) {
// Заменяем дефолтный HTTP-клиент на наш OkHttp с интерсепторами
registry.replace(
GlideUrl::class.java,
InputStream::class.java,
OkHttpUrlLoader.Factory(myOkHttpClient),
)
}
}
```

10.2.8. Compose-интеграция через AndroidView

В Compose нет нативной поддержки Glide, но есть два пути:

- **Путь 1 (классический):** AndroidView { ImageView(it) } с ручным вызовом Glide.with(...).load(...).into(...) в update. Полный контроль, но больше boilerplate.
- **Путь 2 (Compose-first):** использовать официальную библиотеку glide-compose с Composable AsyncGlideImage — похож на Coil AsyncImage.

В этой практике используем **путь 1** — он универсальнее (работает даже без glide-compose) и лучше показывает, как устроен интероп. Шаблон:

KOTLIN

```
@Composable
fun GlideImage(
model: Any?,
contentDescription: String?,
modifier: Modifier = Modifier,
options: RequestOptions = RequestOptions(),
) {
AndroidView(
factory = { ctx ->
ImageView(ctx).apply {
scaleType = ImageView.ScaleType.CENTER_CROP
}
},
update = { iv ->
Glide.with(iv)
.load(model)
.apply(options)
.transition(DrawableTransitionOptions.withCrossFade(150))
.into(iv)
},
modifier = modifier,
)
}
```

Важно: factory vs update

Factory вызывается ОДИН РАЗ при первом появлении View. Всё, что должно меняться при recomposition, должно быть в update. Никогда не создавайте ImageView повторно в factory — это убьёт производительность и потеряет состояние.

10.2.9. Трансформации — CircleCrop, RoundedCorners, кастомные

Glide применяет трансформации на этапе декодирования, до отображения. Результат кэшируется уже трансформированным. Встроенные трансформации: CircleCrop, RoundedCorners, CenterCrop, FitCenter, Grayscale, Blur (через внешний библиотечный пакет jp.wasabeef:glide-transformations). Можно комбинировать через .transforms(...).

```kotlin
// Несколько трансформаций подряд
Glide.with(view)
.load(url)
.transform(
CenterCrop(),
RoundedCorners(16), // скругление после centerCrop
GrayscaleTransformation(), // ч/б
)
.into(view)

// Кастомная трансформация — наследник BitmapTransformation
class VignetteTransformation : BitmapTransformation() {
override fun transform(pool: BitmapPool, source: Bitmap, outW: Int, outH: Int): Bitmap {
// Реализация vignette-эффекта
val result = pool.get(outW, outH, Bitmap.Config.ARGB_8888)
// ... рисование vignette ...
return result
}

override fun updateDiskCacheKey(messageDigest: MessageDigest) {
messageDigest.update("vignette".toByteArray(KeyUtil.CHARSET))
}
}
```

10.2.10. BitmapPool — управление памятью

Главная причина OOM в Android-приложениях с изображениями — создание новых Bitmap на каждый запрос. Glide решает это через BitmapPool — пул переиспользуемых Bitmap-ов. Когда Bitmap больше не нужен, он возвращается в пул; когда нужен новый — Glide берёт из пула подходящий по размеру и переиспользует. Это снижает нагрузку на GC и предотвращает OOM. Размер пула по умолчанию вычисляется через MemorySizeCalculator, но можно задать вручную.

Главное правило для разработчика: **никогда не вызывайте bitmap.recycle()** на Bitmap, который вы получили из Glide — он сам управляет его жизненным циклом. Если делаете кастомную трансформацию — берите Bitmap через pool.get(w, h, config), а не Bitmap.createBitmap.

10.2.11. Загрузка в списках: RecyclerView + Glide

В классических RecyclerView Glide работает идеально: Glide.with(itemView) автоматически отменяет запрос при рециклинге ViewHolder-а. В Compose списке (LazyColumn) то же самое происходит автоматически при уходе Composable из композиции — AndroidView вызывает View.onDetachedFromWindow, и Glide отменяет запрос.

```kotlin
// Compose: список с разными картинками
LazyColumn {
items(photos, key = { it.id }) { photo ->
GlideImage(
model = photo.url,
contentDescription = photo.description,
modifier = Modifier
.fillMaxWidth()
.aspectRatio(photo.aspectRatio)
.clip(RoundedCornerShape(8.dp)),
)
}
}
```

10.2.12. LazyVerticalStaggeredGrid — галерея с разной высотой

Для галерей изображений Compose предоставляет LazyVerticalStaggeredGrid — сетку с переменной высотой ячеек (как Pinterest). Это оптимальный способ показать список изображений, у каждого своя соотношение сторон. Каждая ячейка содержит Composable с GlideImage фиксированной шириной и aspectRatio от данных.

```kotlin
LazyVerticalStaggeredGrid(
columns = StaggeredGridCells.Fixed(2),
contentPadding = PaddingValues(8.dp),
verticalItemSpacing = 8.dp,
horizontalArrangement = Arrangement.spacedBy(8.dp),
) {
items(photos, key = { it.id }) { photo ->
GlideImage(
model = photo.url,
contentDescription = photo.description,
options = RequestOptions().centerCrop(),
modifier = Modifier
.fillMaxWidth()
.aspectRatio(photo.aspectRatio)
.clip(RoundedCornerShape(8.dp)),
)
}
}
```

10.2.13. Подпись, thumbnail и preload

Glide поддерживает несколько продвинутых техник:

• **Thumbnail** — .thumbnail(0.1f) загружает сначала миниатюру (10% размера), затем полную картинку. Полезно для больших изображений.

• **Preload** — .preload(w, h) загружает в кэш без отображения. Полезно если знаете, что картинка скоро понадобится.

• **Signature** — .signature(ObjectKey(version)) инвалидирует кэш при смене версии. Полезно если URL одинаковый, но контент поменялся.

• **Listener** — .listener(object : RequestListener { ... }) для callback-ов успеха/ошибки.

```kotlin
// Thumbnail + preload
Glide.with(view)
.load(url)
.thumbnail(0.1f) // сначала миниатюра
.diskCacheStrategy(ALL)
.listener(object : RequestListener<Drawable> {
override fun onLoadFailed(e: GlideException?, model: Any?,
target: Target<Drawable>?, isFirstResource: Boolean): Boolean {
log.error("Glide failed", e)
return false // позволяем Glide показать error-drawable
}
override fun onResourceReady(resource: Drawable?, model: Any?,
target: Target<Drawable>?, dataSource: DataSource?, isFirstResource: Boolean): Boolean {
return false // позволяем Glide отобразить
}
})
.into(view)

// Preload (без into)
Glide.with(context).load(url).preload(400, 400)
```

Примеры кода

Пример 10.1. Универсальный Composable GlideImage

```kotlin
@Composable
fun GlideImage(
model: Any?,
contentDescription: String?,
modifier: Modifier = Modifier,
options: RequestOptions = RequestOptions(),
contentScale: ImageView.ScaleType = ImageView.ScaleType.CENTER_CROP,
) {
AndroidView(
factory = { ctx ->
ImageView(ctx).apply {
scaleType = contentScale
}
},
update = { iv ->
Glide.with(iv)
.load(model)
.apply(options)
.transition(DrawableTransitionOptions.withCrossFade(150))
.into(iv)
},
modifier = modifier,
)
}
```

Пример 10.2. Аватарка с CircleCrop

KOTLIN

```
@Composable
fun UserAvatar(url: String?, size: Dp = 48.dp, modifier: Modifier = Modifier) {
val options = remember {
RequestOptions()
.circleCrop()
.placeholder(R.drawable.ic_avatar_placeholder)
.error(R.drawable.ic_avatar_error)
.fallback(R.drawable.ic_avatar_placeholder)
.override(256, 256)
}
GlideImage(
model = url,
contentDescription = "Аватар",
options = options,
modifier = modifier
.size(size)
.background(MaterialTheme.colorScheme.surfaceVariant, CircleShape)
.border(1.dp, MaterialTheme.colorScheme.outline, CircleShape),
)
}
```

Пример 10.3. Сетка галереи с разной высотой

KOTLIN

```
data class Photo(
val id: Long,
val url: String,
val aspectRatio: Float,
val description: String?,
)

@Composable
fun PhotoGallery(photos: List<Photo>) {
val options = remember {
RequestOptions()
.centerCrop()
.placeholder(R.drawable.placeholder_image)
.error(R.drawable.error_image)
}
LazyVerticalStaggeredGrid(
columns = StaggeredGridCells.Fixed(2),
contentPadding = PaddingValues(8.dp),
verticalItemSpacing = 8.dp,
horizontalArrangement = Arrangement.spacedBy(8.dp),
modifier = Modifier.fillMaxSize(),
) {
items(photos, key = { it.id }) { photo ->
GlideImage(
model = photo.url,
contentDescription = photo.description,
options = options,
modifier = Modifier
.fillMaxWidth()
.aspectRatio(photo.aspectRatio)
.clip(RoundedCornerShape(8.dp))
.clickable { /* открыть на весь экран */ },
)
}
}
}
```

Задание

В NotesApp добавьте поддержку изображений для заметок. Заметьте: тип NoteType.Image уже существует в domain-модели. Теперь мы расширяем Note полем imageUrl и показываем картинку в карточке через Glide. Также добавьте экран-галерею изображений.

Шаг 1. Зависимости и kapt

В app/build.gradle.kts добавьте Glide 4.16+ и okhttp3-integration (как в 10.2.3). Подключите плагин kotlin-kapt — он нужен для кодогенерации GeneratedAppGlideModule.

Шаг 2. Расширение модели Note

Добавьте в data class Note новое поле: val imageUrl: String? = null. Обновите Entity (добавьте колонку image_url TEXT). TypeConverter трогать не нужно (String примитив). Создайте миграцию 2→3 для добавления колонки image_url. Не забудьте увеличить version = 3 в AppDatabase.

Шаг 3. NotesAppGlideModule

В файле NotesAppGlideModule.kt создайте класс наследник AppGlideModule с аннотацией @GlideModule (по образцу 10.2.7). Настройте:

1. Кэш в памяти через MemorySizeCalculator — setMemoryCacheScreens(2f).

2. Кэш на диске — 250 MB в подпапке image_cache.

3. Зарегистрируйте OkHttp-интеграцию в registerComponents — используйте тот же OkHttpClient, что и Retrofit (если Retrofit ещё не настроен, можно оставить дефолтный).

4. LogLevel VERBOSE в debug, ERROR в release (через BuildConfig.DEBUG).

Шаг 4. Composable GlideImage

В файле ui/GlideImage.kt создайте Composable-функцию:

```kotlin
@Composable
fun GlideImage(
model: Any?,
contentDescription: String?,
modifier: Modifier = Modifier,
options: RequestOptions = RequestOptions(),
contentScale: ImageView.ScaleType = ImageView.ScaleType.CENTER_CROP,
)
```

Требования:

1. Используется AndroidView с ImageView.

2. factory создаёт ImageView один раз, устанавливает scaleType.

3. update вызывает Glide.with(iv).load(model).apply(options).into(iv).

4. Добавлен crossFade-переход 150 мс через DrawableTransitionOptions.

Шаг 5. NoteImage Composable

В файле ui/NoteImage.kt создайте Composable:

```kotlin
@Composable
fun NoteImage(
url: String?,
contentDescription: String?,
modifier: Modifier = Modifier,
cornerRadius: Dp = 8.dp,
)
```

Требования:

1. Если url null или пустой — показывается placeholder-иконка Icons.Default.Image на фоне surfaceVariant.

2. Используется ваш GlideImage с RequestOptions: placeholder, error, centerCrop, override(800, 600).

3. Применяется Modifier.clip(RoundedCornerShape(cornerRadius)).

4. На время загрузки показывается тот же placeholder-drawable (создайте простейший векторный drawable в res/drawable).

Шаг 6. NoteCard с изображением

Обновите NoteCard из практики 2: если note.type == NoteType.Image и note.imageUrl != null — показывайте NoteImage над текстом (высота 120 dp, width fillMaxWidth). Иначе — только текстовая часть.

Шаг 7. PhotoGalleryScreen

В файле ui/PhotoGalleryScreen.kt создайте экран с LazyVerticalStaggeredGrid (2 колонки). Источник данных — заметки с type=Image, у которых imageUrl != null. Каждая ячейка — GlideImage с aspectRatio 1.0 (квадрат). Клик по картинке открывает экран полного просмотра (можно просто fullscreen GlideImage).

Шаг 8. Добавление изображения в FAB-диалоге

В диалоге добавления заметки (из практики 6) добавьте третье поле «URL изображения» (опциональное). Если оно заполнено — создаётся заметка типа Image с этим URL. Иначе — Text.

Шаг 9. Тестирование офлайна

Используйте реальные URL изображений (например, https://picsum.photos/seed/{id}/600/400). Запустите приложение, дождитесь загрузки. Отключите интернет, перезапустите приложение — изображения должны показываться из дискового кэша.

Контрольные вопросы

Ответьте письменно на следующие вопросы. Подготовьте устные ответы для защиты практической работы — преподаватель может задать любой из них или попросить пояснить код вашего решения.

1. Почему нельзя загружать изображения через URL.openStream() в UI-потоке? Назовите минимум 3 причины.

2. Чем Glide отличается от Coil? Почему вы выбрали Glide?

3. Что делает Glide.with(context)? Зачем нужен context — можно ли передать applicationContext?

4. Чем placeholder отличается от error и fallback? Когда каждый срабатывает?

5. Какие стратегии DiskCacheStrategy вы знаете? Что кэширует ALL?

6. Зачем нужен AppGlideModule? Что произойдёт, если не создать его?

7. Как интегрировать Glide с Compose? Чем отличается использование через AndroidView от glide-compose?

8. Что такое BitmapPool? Зачем он нужен? Можно ли вызывать bitmap.recycle() на Bitmap из Glide?

9. Какие трансформации встроены в Glide? Как создать кастомную?

10. Чем centerCrop() отличается от fitCenter() и circleCrop()?

11. Зачем нужен .thumbnail(0.1f)? В каких случаях использовать?

12. Как сделать preload изображения без отображения?

13. Как инвалидировать кэш для URL, если контент поменялся, а URL остался? (Hint: signature)

14. Что такое LazyVerticalStaggeredGrid? Чем отличается от LazyVerticalGrid?

15. Зачем нужен aspectRatio modifier в сетке? Что будет без него?

Чек-лист сдачи работы

Перед сдачей работы преподавателю убедитесь, что выполнены все пункты ниже. Отсутствие любого пункта снижает оценку минимум на один балл.

- □ В build.gradle.kts добавлены glide:glide, glide:okhttp3-integration, kapt compiler
- □ Подключён плагин kotlin-kapt
- □ В data class Note добавлено поле imageUrl: String? = null
- □ В NoteEntity добавлено @ColumnInfo(name = "image_url") imageUrl: String?
- □ Создана миграция MIGRATION_2_3 с ALTER TABLE notes ADD COLUMN image_url TEXT
- □ В AppDatabase version = 3, миграция зарегистрирована
- □ В NotesAppGlideModule.kt есть класс : AppGlideModule с @GlideModule
- □ Настроены memoryCache (MemorySizeCalculator) и diskCache (250 MB)
- □ OkHttp-интеграция зарегистрирована в registerComponents
- □ LogLevel зависит от BuildConfig.DEBUG
- □ В ui/GlideImage.kt есть @Composable fun GlideImage(model, contentDescription, modifier, options, contentScale)
- □ GlideImage использует AndroidView с ImageView
- □ factory создаёт ImageView один раз, update вызывает Glide.with.into
- □ Добавлен crossFade-переход 150 мс
- □ В ui/NoteImage.kt есть @Composable fun NoteImage(url, contentDescription, modifier, cornerRadius)
- □ NoteImage использует GlideImage с RequestOptions (placeholder, error, centerCrop)
- □ При url=null показывается Icons.Default.Image на surfaceVariant
- □ NoteCard показывает NoteImage при type=Image и imageUrl != null
- □ В ui/PhotoGalleryScreen.kt есть LazyVerticalStaggeredGrid с 2 колонками
- □ В каждой ячейке GlideImage с aspectRatio и key = { it.id }
- □ FAB-диалог содержит третье поле «URL изображения» (опциональное)
- □ При заполненном URL создаётся заметка type=Image
- □ В офлайн-режиме изображения показываются из кэша

Что сдавать

В результате выполнения практической работы студент предоставляет:

1. Git-коммит «Практика 10» с GlideImage, NoteImage, PhotoGalleryScreen, NotesAppGlideModule;
2. Скриншот NoteCard с изображением (тип Image);
3. Скриншот PhotoGalleryScreen с сеткой из 6+ картинок;
4. Скриншот офлайн: изображения показываются при отсутствии сети.

Практика 11. ViewFactory и интероп с View

Трудоёмкость: 6 академических часов. **Форма работы:** индивидуальная, в IDE Android Studio. **Форма отчёта:** исходный код проекта + устная защита.

Цель и задачи

Цель работы — освоить двусторонний интероп между Jetpack Compose и классической системой View. В реальных проектах Compose появляется постепенно: рядом с legacy View-экранами или в виде отдельных Composable внутри Fragment-ов. Студент научится встраивать View в Compose через AndroidView, обратному — вставлять Compose во Fragment/XML через ComposeView и AbstractComposeView, а также использовать ViewFactory в адаптерах.

1. объяснить, когда интероп оправдан, а когда — нет;
2. использовать AndroidView для встраивания View в Compose;
3. создавать ComposeView во Fragment и XML;
4. реализовывать AbstractComposeView для переиспользуемых Composable;
5. применять ViewFactory в RecyclerView.Adapter;
6. корректно управлять жизненным циклом и обновлениями.

Теоретическая часть

11.2.1. Зачем нужен интероп

Интероп View ↔ Compose нужен в двух категориях ситуаций. Во-первых, миграция legacy-проекта: команда переходит с View на Compose постепенно, экран за экраном, и в один момент в старом XML-экране появляется один новый Compose-блок. Во-вторых, использование View-библиотек, у которых нет Compose-аналога: рекламные SDK (AdMob, AppLovin), карты (Google Maps, Yandex Maps), видео-плееры (ExoPlayer с PlayerView), WebView. Все они являются View, и их нужно встроить в Compose-экран. Интероп делает переход плавным и позволяет использовать существующие наработки.

11.2.2. AndroidView — View внутри Compose

AndroidView — главная Composable-функция для встраивания View. Принимает две лямбды: factory (создаёт View один раз при первом вызове) и update (вызывается при каждой recomposition, обновляет свойства View). Жизненный цикл AndroidView корректно связан с Compose: при уходе с экрана View отсоединяется от родителя.

KOTLIN

```
import androidx.compose.ui.viewinterop.AndroidView

@Composable
fun TextViewInterop(text: String, modifier: Modifier = Modifier) {
AndroidView(
factory = { ctx -> TextView(ctx) },
update = { tv: TextView ->
tv.text = text
tv.textSize = 16f
tv.setPadding(16, 16, 16, 16)
},
modifier = modifier,
)
}
```

Важное правило

Factory вызывается ОДИН РАЗ при первом появлении View. Всё, что должно меняться при recomposition, должно быть в update. Никогда не создавайте View повторно в factory — это убьёт производительность и потеряет состояние.

11.2.3. ComposeView — Compose внутри View/Fragment

Обратный интероп: ComposeView — это View, внутри которой рендерится Composable-контент. Создаётся в коде или XML, в setContent { ... } описывается содержимое. Часто используется в Fragment для постепенной миграции: Fragment остаётся, а его layout заменяется на один ComposeView.

KOTLIN

```
class SettingsFragment : Fragment() {
override fun onCreateView(
inflater: LayoutInflater,
container: ViewGroup?,
savedInstanceState: Bundle?,
): View {
return ComposeView(requireContext()).apply {
setViewCompositionStrategy(
ViewCompositionStrategy.DisposeOnViewTreeLifecycleDestroyed
)
setContent {
NotesAppTheme {
SettingsScreen()
}
}
}
}
}
```

11.2.4. ViewCompositionStrategy — жизненный цикл

ComposeView должен знать, когда освобождать ресурсы Composition. Стратегии:

- DisposeOnViewTreeLifecycleDestroyed — disposed когда Lifecycle owner фрагмента уничтожается (по умолчанию для Fragment).
- DisposeOnLifecycleDestroyed — то же, но принимает явный Lifecycle owner.
- DisposeOnDetachedFromWindowOrReleasedFromPool — для RecyclerView (View переиспользуется).

• DisposeOnDetachedFromWindow — disposed когда View удаляется из иерархии. Подходит для Activity.

11.2.5. AbstractComposeView — переиспользуемый ComposeView

Если вы хотите сделать кастомный View, который переиспользуется в XML (например, в SettingsFragment, написанном на PreferenceFragment), и при этом использует Compose внутри — создайте наследника AbstractComposeView. Внутри переопределите Content как @Composable-свойство. Этот View можно использовать в XML как обычный <ru.notesapp.ui.TagInputComposeView>.

KOTLIN

```
class TagInputComposeView @JvmOverloads constructor(
context: Context,
attrs: AttributeSet? = null,
defStyle: Int = 0,
) : AbstractComposeView(context, attrs, defStyle) {

var tags by mutableStateOf<List<String>>(emptyList())
var onTagAdded by mutableStateOf<((String) -> Unit)?>(null)

@Composable
override fun Content() {
var input by remember { mutableStateOf("") }
Column {
Row {
TextField(
value = input,
onValueChange = { input = it },
label = { Text("Ter") },
)
Button(onClick = {
if (input.isNotBlank()) {
onTagAdded?.invoke(input.trim())
input = ""
}
}) { Text("+") }
}
FlowRow {
tags.forEach { tag ->
AssistChip(onClick = {}, label = { Text(tag) })
}
}
}
}
}
```

11.2.6. ViewFactory в RecyclerView

ViewFactory в широком смысле — это любой механизм, создающий View. В контексте RecyclerView это интерфейс Adapter.onCreateViewHolder(parent, viewType), возвращающий ViewHolder с View. Если View — это ComposeView, получается гибрид: RecyclerView управляет переиспользованием ячеек (как обычно), а содержимое ячейки рисуется через Compose. Это компромисс для очень длинных списков с разнородным контентом, где LazyColumn может не хватить гибкости.

```kotlin
class NoteViewHolder(
private val composeView: ComposeView,
) : RecyclerView.ViewHolder(composeView) {

fun bind(note: Note, onClick: (Note) -> Unit) {
composeView.setContent {
NotesAppTheme {
NoteCard(note = note, onClick = { onClick(note) })
}
}
}

companion object {
fun create(parent: ViewGroup): NoteViewHolder {
val view = ComposeView(parent.context).apply {
setViewCompositionStrategy(
ViewCompositionStrategy.DisposeOnDetachedFromWindowOrReleasedFromPool
)
layoutParams = ViewGroup.LayoutParams(
ViewGroup.LayoutParams.MATCH_PARENT,
ViewGroup.LayoutParams.WRAP_CONTENT,
)
}
return NoteViewHolder(view)
}
}
}
```

11.2.7. Интероп с WebView

WebView — один из самых частых случаев интероп а. В Compose нет нативного WebView, поэтому используем AndroidView. Важно: WebView — тяжёлый объект, его создание в factory приемлемо, но состояние (url, настройки) обновляется в update. Для сложных WebView (с JavaScript-интерфейсами) лучше вынести его в отдельный класс с remembered state.

```kotlin
@Composable
fun WebViewScreen(url: String, modifier: Modifier = Modifier) {
AndroidView(
factory = { ctx ->
WebView(ctx).apply {
settings.javaScriptEnabled = true
webViewClient = WebViewClient()
loadUrl(url)
}
},
update = { web -> web.loadUrl(url) },
modifier = modifier.fillMaxSize(),
)
}
```

11.2.8. Производительность интероп а

Интероп не бесплатен. Главные проблемы:

• Каждый AndroidView создаёт полноценный View в иерархии — это стоит памяти и времени на layout / measure.

• Compose не может оптимизировать View внутри AndroidView (нет skip-логики по состоянию View).

• Синхронизация состояния Compose ↔ View может привести к «пинг-понгу» обновлений, если оба источника меняют одно и то же.

Правила оптимизации: 1) используйте интероп только когда нужно; 2) не вкладывайте AndroidView внутрь быстро меняющихся Composable (LazyColumn); 3) для списков предпочитайте LazyColumn с Composable-ячейками, а не RecyclerView с ComposeView-ViewHolder-ами, если нет явной необходимости.

Примеры кода

Пример 11.1. BannerAdView через AndroidView

KOTLIN

```
@Composable
fun BannerAdView(
adUnitId: String,
modifier: Modifier = Modifier,
) {
AndroidView(
factory = { ctx ->
// Имитация рекламного баннера — реальный AdMob требует SDK
TextView(ctx).apply {
text = "Реклама: $adUnitId"
setPadding(16, 16, 16, 16)
setBackgroundColor(Color.LTGRAY)
gravity = Gravity.CENTER
}
},
update = { tv ->
tv.text = "Реклама: $adUnitId"
},
modifier = modifier
.fillMaxWidth()
.height(60.dp),
)
}
```

Пример 11.2. TagInputComposeView (AbstractComposeView)

```kotlin
class TagInputComposeView @JvmOverloads constructor(
ctx: Context, attrs: AttributeSet? = null,
) : AbstractComposeView(ctx, attrs) {

var tags: List<String> by mutableStateOf(emptyList())
var onTagAdded: ((String) -> Unit)? by mutableStateOf(null)

init {
setViewCompositionStrategy(
ViewCompositionStrategy.DisposeOnDetachedFromWindowOrReleasedFromPool
)
}

@Composable
override fun Content() {
/* ... реализация из 10.2.5 ... */
}
}

// Использование в XML:
// <ru.notesapp.ui.TagInputComposeView
// android:id="@+id/tagInput"
// android:layout_width="match_parent"
// android:layout_height="wrap_content" />
```

Задание

В NotesApp добавьте три интероп-компонента: 1) рекламный баннер через AndroidView (имитация), 2) кастомный AbstractComposeView для ввода тегов, который можно использовать из XML, 3) SettingsFragment с PreferenceFragment и встроенным ComposeView для кастомной секции.

Шаг 1. BannerAdView

В файле ui/BannerAdView.kt создайте Composable BannerAdView(adUnitId: String, modifier: Modifier) по образцу 10.2.8 (пример 10.1). Вместо реального AdMob используется TextView-имитация с текстом «Реклама: *adUnitId*» и серым фоном. Разместите этот баннер в NotesListScreen под TopAppBar, над списком заметок.

Шаг 2. TagInputComposeView

В файле ui/TagInputComposeView.kt создайте класс наследник AbstractComposeView с сигнатурой из примера 10.2.5. Реализуйте Content():

1. TextField для ввода нового тега
2. Кнопка «+» добавляет тег в список (вызывает onTagAdded)
3. FlowRow из Material 3 со всеми текущими тегами в виде AssistChip
4. Клик по чипу удаляет его из списка

Шаг 3. SettingsFragment с PreferenceFragment + Compose

Создайте legacy SettingsFragment через PreferenceFragmentCompat. В res/xml/preferences.xml опишите 2-3 стандартные настройки (например, PreferenceCategory «Уведомления» с SwitchPreferenceCompat «Включить уведомления»). Затем добавьте кастомный Preference, который в onCreateView возвращает ComposeView с кастомной Composable-секцией «О приложении»: иконка, название, версия, кнопка

«Оценить в Google Play».

```kotlin
class AboutPreference(context: Context, attrs: AttributeSet) : Preference(context, attrs) {

override fun onBindViewHolder(holder: PreferenceViewHolder) {
super.onBindViewHolder(holder)
// Заменяем обычный контент на ComposeView
val composeView = (holder.itemView as? FrameLayout)?.let { fl ->
fl.removeAllViews()
ComposeView(context).also { cv ->
cv.layoutParams = FrameLayout.LayoutParams(
FrameLayout.LayoutParams.MATCH_PARENT,
FrameLayout.LayoutParams.WRAP_CONTENT,
)
cv.setContent {
NotesAppTheme {
AboutComposeContent()
}
}
fl.addView(cv)
}
}
}
}

@Composable
private fun AboutComposeContent() {
Column(
modifier = Modifier.padding(16.dp),
horizontalAlignment = Alignment.CenterHorizontally,
) {
Icon(Icons.Default.Info, contentDescription = null,
modifier = Modifier.size(48.dp))
Text("NotesApp", style = MaterialTheme.typography.titleLarge)
Text("v0.10.0 (Практика 10)",
style = MaterialTheme.typography.bodySmall,
color = MaterialTheme.colorScheme.outline)
Spacer(Modifier.height(8.dp))
OutlinedButton(onClick = { /* открыть Play Store */ }) {
Text("Оценить в Google Play")
}
}
}
```

Шаг 4. Подключение TagInputComposeView

В экране создания заметки (FAB-диалог из практики 6/10) замените простое TextField для тегов на TagInputComposeView — если вы делали через Compose, то просто вставьте Composable. Альтернативно: создайте отдельный Fragment с XML-layout, содержащим <ru.notesapp.ui.TagInputComposeView>, и продемонстрируйте, что View корректно работает из XML. Достаточно одного варианта — на выбор студента.

Шаг 5. Проверка работоспособности

Запустите приложение. Проверьте:

1. BannerAdView отображается под TopAppBar на NotesListScreen.
2. TagInputComposeView добавляет и удаляет теги.

3. SettingsFragment открывается с PreferenceFragment и Compose-секцией.

4. Кнопка «Оценить» в Compose-секции кликабельна.

Контрольные вопросы

Ответьте письменно на следующие вопросы. Подготовьте устные ответы для защиты практической работы — преподаватель может задать любой из них или попросить пояснить код вашего решения.

1. Зачем нужен интероп View ↔ Compose? Назовите минимум 3 сценария.

2. Что делает AndroidView? Чем factory отличается от update?

3. Почему нельзя создавать View заново в factory при каждой recomposition?

4. Что такое ComposeView? Когда его используют?

5. Зачем нужна ViewCompositionStrategy? Какие стратегии вы знаете?

6. Чем AbstractComposeView отличается от ComposeView?

7. Как сделать кастомный View на базе AbstractComposeView доступным в XML?

8. Как использовать ComposeView внутри RecyclerView? Какие особенности ViewCompositionStrategy?

9. Зачем @JvmOverloads в конструкторе View? Без него что-то не работает?

10. Как встроить WebView в Compose? Какие сложности возникают?

11. Какие проблемы производительности есть у интеропа? Как их минимизировать?

12. Можно ли смешивать Compose и View в одном экране? Какие правила хорошего тона?

Чек-лист сдачи работы

Перед сдачей работы преподавателю убедитесь, что выполнены все пункты ниже. Отсутствие любого пункта снижает оценку минимум на один балл.

- □ В ui/BannerAdView.kt есть @Composable fun BannerAdView(adUnitId, modifier)
- □ Использован AndroidView с factory и update
- □ BannerAdView размещён в NotesListScreen под TopAppBar
- □ В ui/TagInputComposeView.kt есть class TagInputComposeView : AbstractComposeView
- □ Конструктор с @JvmOverloads (context, attrs, defStyle)
- □ Свойство Content помечено @Composable и переопределено
- □ tags и onTagAdded — mutableStateOf
- □ В Content есть TextField, Button, FlowRow с AssistChip
- □ Клик по чипу удаляет тег
- □ Создан SettingsFragment : PreferenceFragmentCompat
- □ В res/xml/preferences.xml есть PreferenceCategory с настройками
- □ Создан AboutPreference : Preference
- □ onBindViewHolder заменяет itemView на ComposeView с AboutComposeContent
- □ AboutComposeContent показывает иконку, название, версию, кнопку
- □ Кнопка «Оценить в Google Play» кликабельна

Что сдавать

В результате выполнения практической работы студент предоставляет:

1. Git-коммит «Практика 10» с BannerAdView, TagInputComposeView, SettingsFragment, AboutPreference;

2. Скриншот NotesListScreen с баннером под TopAppBar;

3. Скриншот SettingsFragment с preference-секцией и Compose-блоком «О приложении»;

4. Скриншот TagInputComposeView с 3+ добавленными тегами.

Практика 12. Рефлексия в Kotlin (ознакомление)

Трудоёмкость: 6 академических часов. **Форма работы:** индивидуальная, в IDE Android Studio. **Форма отчёта:** исходный код проекта + устная защита.

Цель и задачи

Цель работы — познакомиться с механизмом рефлексии в Kotlin, понять, когда его применение оправдано, а когда — нет. Рефлексия позволяет во время выполнения программы исследовать структуру классов: получать списки свойств, методов, аннотаций, конструировать объекты, вызывать методы по имени. Студент реализует простой JSON-сериализатор на основе рефлексии — это классический пример, на котором видно и возможности, и ограничения механизма.

1. объяснить, что такое рефлексия и для чего она применяется;
2. различать Kotlin Reflection (KClass, KProperty) и Java Reflection (Class, Field);
3. получать список свойств и конструкторов класса;
4. вызывать функции и конструкторы через callBy;
5. читать пользовательские аннотации;
6. реализовать простой сериализатор объектов в JSON;
7. оценить производительность рефлексии и знать, когда её избегать.

Теоретическая часть

12.2.1. Что такое рефлексия

Рефлексия — способность программы исследовать и модифицировать собственную структуру во время выполнения. В Java и Kotlin рефлексия позволяет:

• получить класс объекта через ::class или obj.javaClass;

• перечислить все свойства (KClass.members) и конструкторы (KClass.constructors);

• получить значения свойств (KProperty.get(obj));

• вызвать метод или конструктор (KFunction.callBy(args));

• прочитать аннотации на классах, свойствах, функциях (KAnnotatedElement.annotations);

• создать новый экземпляр класса через KClass.createInstance().

12.2.2. Kotlin Reflection vs Java Reflection

В Kotlin есть два параллельных API рефлексии:

Свойство	Kotlin Reflection	Java Reflection
Главный класс	`KClass<T>`	`Class<T>`

Свойство	Kotlin Reflection	Java Reflection
Получить класс	`Note::class`	`note.javaClass` или `Note::class.java`
Свойства	`KClass.members`, `declaredMemberProperties`	`Class.declaredFields`
Методы	`KClass.functions`	`Class.declaredMethods`
Nullable-информация	Да (через `KProperty.returnType.isMarkedNullable`)	Нет
Расширения	Поддерживаются (`extensionFunctions`)	Не видны
Зависимость	`kotlin-reflect` (~3 MB)	Встроена в JDK
Производительность	Медленнее (обёртка над Java)	Быстрее (нативно)

Правило: используйте Kotlin Reflection когда нужны Kotlin-специфичные возможности (nullable-информация, расширения, value classes). Если нужны просто поля и методы — Java Reflection быстрее и не требует тяжёлой зависимости kotlin-reflect.

12.2.3. Зависимость kotlin-reflect

Чтобы использовать KClass, KProperty и др., нужно добавить зависимость:

```kotlin
dependencies {
implementation("org.jetbrains.kotlin:kotlin-reflect:2.0.21")
}
```

Эта зависимость весит около 2–3 MB. В release-сборках её стоит минимизировать через R8/ProGuard, исключая неиспользуемые классы. Если рефлексия в проекте не нужна — не подключайте.

12.2.4. KClass — точка входа

KClass<T> — главный тип Kotlin Reflection. Получить его можно тремя способами:

```kotlin
// 1. Из типа (compile-time)
val kc: KClass<Note> = Note::class

// 2. Из экземпляра (runtime)
val note = Note(1, "x", "y", 0L, NoteType.Text)
val kc2: KClass<out Note> = note::class

// 3. По имени (если есть Class<*>)
val kc3: KClass<*>? = Class.forName("ru.notesapp.domain.Note").kotlin
```

Через KClass доступны:

- simpleName / qualifiedName — имена класса;
- members — все свойства и функции (включая унаследованные);
- declaredMembers — только объявленные в этом классе;
- declaredMemberProperties — только свойства (без функций);

- constructors — все конструкторы;
- primaryConstructor — первичный конструктор;
- annotations — аннотации на классе;
- isData, isSealed, isAbstract — флаги модификаторов.

12.2.5. KProperty и KFunction

KProperty<T> — описание свойства. Главные операции: get(obj) — получить значение, set(obj, value) — установить (только для var). У свойств есть name, returnType, isLateInit, annotations.

```kotlin
val props = Note::class.declaredMemberProperties
for (p in props) {
println("${p.name}: ${p.returnType}, nullable=${p.returnType.isMarkedNullable}")
val value = p.get(note)
println(" → $value")
}
```

KFunction<T> — описание функции или конструктора. Метод call(*args) вызывает с позиционными аргументами, callBy(map) — с именованными. callBy удобнее: не нужно помнить порядок параметров, можно передать только обязательные.

```kotlin
val ctor = Note::class.primaryConstructor
val params = ctor!!.parameters.associateWith { p ->
when (p.name) {
"id" -> 0L
"title" -> "Reflection created"
"content" -> "Made by callBy"
"createdAt" -> System.currentTimeMillis()
"type" -> NoteType.Text
else -> null
}
}
val newNote = ctor.callBy(params)
```

12.2.6. Аннотации

Рефлексия позволяет читать пользовательские аннотации — это основа многих библиотек: Room (читает @Entity, @Dao), Retrofit (читает @GET, @POST), Moshi (читает @Json). Создание аннотации:

```kotlin
@Target(AnnotationTarget.CLASS, AnnotationTarget.PROPERTY)
@Retention(AnnotationRetention.RUNTIME) // RUNTIME — для рефлексии
annotation class MiniSerializable(val name: String = "")

@MiniSerializable(name = "note")
data class Note(
@MiniSerializable(name = "id") val id: Long,
@MiniSerializable(name = "title") val title: String,
val content: String, // без аннотации — сериализуем по имени свойства
val createdAt: Long,
val type: NoteType,
)

// Чтение аннотации:
val kc = Note::class
val classAnno = kc.findAnnotation<MiniSerializable>()
println("Имя в JSON: ${classAnno?.name ?: kc.simpleName}")

for (p in kc.declaredMemberProperties) {
val pAnno = p.findAnnotation<MiniSerializable>()
println("${p.name} → ${pAnno?.name ?: p.name}")
}
```

Важно: @Retention(AnnotationRetention.RUNTIME) обязательна. По умолчанию в Kotlin retention = RUNTIME, в Java — CLASS (не доступна через рефлексию в рантайме).

12.2.7. Сценарии использования

Реальные применения рефлексии в Android-разработке:

• **Сериализация JSON**: Moshi, GSON, kotlinx.serialization (последняя — через кодогенерацию, без рефлексии в рантайме).

• **ORM**: Room читает @Entity и @Dao на этапе компиляции (через KSP), но в рантайме тоже использует рефлексию для создания экземпляров.

• **DI**: Hilt/Dagger используют кодогенерацию, но если встречается @Inject field — нужно через рефлексию проставить значение.

• **Тестирование**: MockK читает сигнатуры методов через рефлексию, чтобы создать mock-объекты.

• **Плагины и динамическая загрузка**: загрузка классов из DEX-файлов, рефлексия по сервисам Android system services.

12.2.8. Когда рефлексию использовать НЕ нужно

Минусы рефлексии:

• **Производительность**: вызов через KFunction.callBy медленнее прямого вызова в 10–100 раз. На горячих путях это критично.

• **Типобезопасность**: типы теряются, появляются ClassCastException в рантайме вместо ошибок компиляции.

• **Защита от обфускации**: R8/ProGuard может переименовать свойства; если ваш код использует рефлексию по именам — нужно keep-правила, иначе всё сломается в release.

• **Читаемость кода**: рефлексия скрывает реальный поток данных, усложняет отладку.

• **Безопасность**: рефлексия позволяет обходить private — это риск в коде, обрабатывающем недоверенные данные.

> **Главное правило**
>
> Если задачу можно решить без рефлексии (через интерфейсы, when, sealed class, кодогенерацию) — решайте без неё. Рефлексия — последний инструмент, когда альтернатив нет.

Примеры кода

Пример 12.1. Перечисление свойств класса

KOTLIN

```
fun printProperties(obj: Any) {
val kc = obj::class
println("Класс: ${kc.qualifiedName}")
println("isData: ${kc.isData}")
println("Свойства:")
for (p in kc.declaredMemberProperties) {
val value = p.getter.call(obj)
println(" ${p.name}: ${p.returnType} = $value "
+ "(nullable=${p.returnType.isMarkedNullable})")
}
}

data class User(val id: Long, val name: String, val email: String?)

fun main() {
printProperties(User(1, "Alice", null))
}
```

Пример 12.2. Простой JSON-сериализатор (фрагмент)

KOTLIN

```
fun Any.toJson(): String {
val kc = this::class
val sb = StringBuilder("{")
val props = kc.declaredMemberProperties.toList()
props.forEachIndexed { i, p ->
if (i > 0) sb.append(", ")
sb.append('"').append(p.name).append("":")
val v = p.getter.call(this)
sb.append(when (v) {
is String -> "\"${v.replace("\"", "\\\"")}\""
is Number, is Boolean -> v.toString()
null -> "null"
else -> "\"${v.toString()}\"" // упрощённо
})
}
sb.append("}")
return sb.toString()
}
```

Задание

В NotesApp реализуйте мини-JSON-сериализатор через рефлексию. Создайте аннотацию @MiniSerializable, функции toJson(obj) и fromJson(json, klass). Примените сериализатор к классам Note и User (User нужно создать). Покройте сериализатор unit-тестами.

Шаг 1. Зависимость kotlin-reflect

Добавьте в build.gradle.kts: implementation("org.jetbrains.kotlin:kotlin-reflect:2.0.21"). Синхронизируйте Gradle.

Шаг 2. Аннотация @MiniSerializable

В файле util/MiniSerializable.kt создайте аннотацию с параметром name: String = "" (для переопределения имени в JSON). Target: CLASS и PROPERTY. Retention: RUNTIME.

Шаг 3. Класс User

В файле domain/User.kt создайте data class:

```kotlin
@MiniSerializable(name = "user")
data class User(
@MiniSerializable(name = "user_id") val id: Long,
@MiniSerializable(name = "user_name") val name: String,
val email: String?,
val age: Int = 0,
)
```

Шаг 4. toJson()

В файле util/MiniJson.kt создайте функцию fun Any.toJson(): String. Алгоритм:

1. Получить this::class.
2. Если класс не помечен @MiniSerializable — бросить IllegalArgumentException.
3. Получить declaredMemberProperties.
4. Для каждого свойства: если есть аннотация @MiniSerializable(name = ...) — использовать это имя, иначе — p.name.
5. Значение: String → в кавычках с экранированием, Number/Boolean → как есть, null → "null", enum → имя, объект → рекурсивный toJson() (если тоже @MiniSerializable).
6. Собрать JSON в StringBuilder.

Шаг 5. fromJson()

Там же создайте функцию fun <T : Any> fromJson(json: String, klass: KClass<T>): T. Алгоритм:

1. Распарсить JSON в Map<String, Any?> (можно простейшим регэкспом, без вложенных объектов — упрощённый парсер).
2. Получить klass.primaryConstructor.
3. Для каждого параметра конструктора: найти соответствие в Map (по имени или по аннотации @MiniSerializable), привести тип.
4. Вызвать ctor.callBy(params).
5. Если обязательный параметр отсутствует — бросить исключение.

> **Ограничение**

> Парсер JSON не обязан поддерживать вложенные объекты и массивы — достаточно простых типов: String, Int, Long, Double, Boolean, null. Это упрощение для практики. В реальном коде используют готовые библиотеки.

Шаг 6. Применение к Note (упрощённо)

Note содержит поле типа NoteType (sealed class) — это усложняет сериализацию. На этой практике ограничимся: сериализуем Note, но поле type представим как строку (через NoteTypeConverter из практики 7 или аналогично). Создайте data class SimpleNote (без NoteType) для демонстрации полного цикла toJson/fromJson.

Шаг 7. Тесты

В src/test/ создайте MiniJsonTest.kt с тестами:

1. test toJson для User с заполненным email — проверка структуры JSON
2. test toJson для User с null email — null должен стать "null" без кавычек
3. test fromJson для User — обратное преобразование, сравнение полей
4. test @MiniSerializable name override — проверка, что "user_id" а не "id" в JSON
5. test класс без @MiniSerializable бросает исключение
6. test round-trip: fromJson(toJson(x)) == x

Контрольные вопросы

Ответьте письменно на следующие вопросы. Подготовьте устные ответы для защиты практической работы — преподаватель может задать любой из них или попросить пояснить код вашего решения.

1. Что такое рефлексия? Какие возможности она даёт?
2. Чем Kotlin Reflection отличается от Java Reflection? Что выбрать для нового проекта?
3. Как получить KClass класса? Три способа.
4. Чем members отличается от declaredMembers у KClass?
5. Как вызвать функцию через KFunction.callBy? Чем этот способ лучше call?
6. Зачем нужна аннотация @Retention(RUNTIME)? Что будет без неё?
7. Какие сценарии использования рефлексии вы знаете? Приведите минимум 4.
8. Какие минусы у рефлексии? Почему её избегают на горячих путях?
9. Что делает isMarkedNullable? Зачем это знать при сериализации?
10. Как Room использует рефлексию? А Hilt? А Moshi?
11. Как R8/ProGuard влияет на рефлексию? Что нужно сделать в release-сборке?
12. Можно ли обойти private через рефлексию? Какие риски это создаёт?

Чек-лист сдачи работы

Перед сдачей работы преподавателю убедитесь, что выполнены все пункты ниже. Отсутствие любого пункта снижает оценку минимум на один балл.

- □ В build.gradle.kts добавлен org.jetbrains.kotlin:kotlin-reflect

- □ В util/MiniSerializable.kt есть annotation class MiniSerializable(name: String = "")
- □ Target = [CLASS, PROPERTY], Retention = RUNTIME
- □ В domain/User.kt есть @MiniSerializable data class User с 4 полями
- □ Поля id и name помечены @MiniSerializable с переопределёнными именами
- □ В util/MiniJson.kt есть fun Any.toJson(): String
- □ toJson() бросает IllegalArgumentException, если класс без @MiniSerializable
- □ toJson() использует declaredMemberProperties и findAnnotation
- □ String-значения экранируются (минимум " → \\)", null → "null" без кавычек
- □ В util/MiniJson.kt есть fun fromJson(json, klass): T
- □ fromJson() использует primaryConstructor.callBy(params)
- □ Создан SimpleNote data class для полной демонстрации цикла
- □ В src/test/MiniJsonTest.kt есть 6+ тестов
- □ Тест round-trip проходит (fromJson(toJson(x)) == x)
- □ Тест на отсутствие @MiniSerializable бросает исключение
- □ Все тесты зелёные (./gradlew test)

Что сдавать

В результате выполнения практической работы студент предоставляет:

1. Git-коммит «Практика 11» с MiniSerializable, MiniJson, User, MiniJsonTest;

2. Скриншот прохождения тестов (./gradlew test --info);

3. Скриншот Logcat с примером toJson/fromJson для SimpleNote;

4. Краткий отчёт: какие альтернативы рефлексии вы знаете (kotlinx.serialization, кодогенерация) и почему они лучше.

Практика 13. Тестирование Android-приложений

Трудоёмкость: 5 академических часов. **Форма работы:** индивидуальная, в IDE Android Studio. **Форма отчёта:** исходный код проекта + устная защита.

Цель и задачи

Цель работы — освоить основы тестирования Android-приложений на примере NotesApp. Студент научится писать unit-тесты для ViewModel и логики (JVM, без устройства), инструментальные тесты для DAO Room (на in-memory базе), UI-тесты для Compose (через createAndroidComposeRule), а также тестировать Flow через Turbine. После этой практики ключевые компоненты NotesApp должны иметь покрытие тестами не менее 60%.

1. различать unit-тесты (JVM) и instrumentation-тесты (требуют устройство);
2. использовать JUnit4, Truth assertions, MockK для моков;
3. тестировать корутины и Flow через runTest и Turbine;
4. тестировать Room DAO на in-memory базе;
5. писать Compose UI-тесты через createAndroidComposeRule;
6. тестировать Hilt-инжекцию через @HiltAndroidTest;
7. измерять покрытие кода тестами.

Теоретическая часть

13.2.1. Пирамида тестов

Пирамида тестов — модель, согласно которой тестов должно быть много в основании (unit, быстрые) и мало на вершине (E2E, медленные). В Android три уровня:

• **Unit-тесты (JVM)** — локально на машине разработчика, без устройства. Тестируют чистую логику: ViewModel, Repository (с mock DAO/Api), UseCase, Mappers. Каталог src/test/. Сотни тестов, миллисекунды каждый.

• **Инструментальные тесты (AndroidX Test)** — на эмуляторе или устройстве. Тестируют Android-зависимый код: Room DAO (нужен SQLite), Compose UI (нужен Activity), Hilt-инъекция. Каталог src/androidTest/. Десятки тестов, секунды каждый.

• **E2E / UI-автоматизация** — проходят через всё приложение, имитируют действия пользователя. Используют Espresso или UI Automator. Единицы тестов, десятки секунд каждый.

13.2.2. JUnit4 — основа

JUnit4 — стандартный фреймворк тестирования в JVM. Главные аннотации:

Аннотация	Назначение
`@Test`	Помечает метод как тест
`@Before`	Вызывается перед каждым тестом (setup)
`@After`	Вызывается после каждого теста (teardown)
`@BeforeClass`	Один раз перед всеми тестами в классе (static)
`@AfterClass`	Один раз после всех тестов
`@Ignore`	Пропустить тест
`@RunWith(X.class)`	Кастомный runner (например, Parameterized)

Утверждения делаются через JUnit assertEquals (старый стиль) или через библиотеку Truth assertThat(x).isEqualTo(y) (более читаемый, рекомендуется Google). Truth уже включена в androidx.test.ext:junit.

13.2.3. MockK — моки для Kotlin

MockK — библиотека для создания mock-объектов, специально разработанная для Kotlin (поддерживает coroutines, extension functions, final classes). Альтернативы — Mockito-Kotlin (legacy), MockInterceptor. Главные операции:

```kotlin
// Зависимость: testImplementation("io.mockk:mockk:1.13.12")

class NotesViewModelTest {
private val repo: NotesRepository = mockk()
private lateinit var viewModel: NotesViewModel

@Before
fun setup() {
viewModel = NotesViewModel(repo)
}

@Test
fun `load notes success`() = runTest {
// Arrange
val notes = listOf(Note(1, "test", "content", 0L, NoteType.Text))
every { repo.observeAll() } returns flowOf(notes)

// Act
viewModel.uiState.test { assertEquals(NotesUiState.Success(notes), awaitItem()) }

// Verify
coVerify(exactly = 1) { repo.observeAll() }
}

@Test
fun `add note calls repository`() = runTest {
coEvery { repo.add(any()) } returns NetworkResult.Success(1L)
viewModel.addNote("title", "content", NoteType.Text)
coVerify { repo.add(any()) }
}
}
```

13.2.4. Turbine — тестирование Flow

Turbine — библиотека для тестирования Flow. Предоставляет suspend-функцию flow.test { }, внутри которой доступны awaitItem(), awaitComplete(), awaitError(), expectNoEvents(). Решает проблему: обычный toList() блокирует навсегда на горячих Flow, а Turbine корректно работает с остановкой и возобновлением.

```kotlin
// Зависимость: testImplementation("app.cash.turbine:turbine:1.1.0")

@Test
fun `uiState emits Loading then Success`() = runTest {
val fakeNotes = listOf(Note(1, "x", "y", 0L, NoteType.Text))
val repo = mockk<NotesRepository>()
every { repo.observeAll() } returns flowOf(emptyList(), fakeNotes)

val vm = NotesViewModel(repo)

vm.uiState.test {
// Первое значение — Loading (initialValue в stateIn)
assertEquals(NotesUiState.Loading, awaitItem())
// Empty (первый эммит репо)
assertEquals(NotesUiState.Empty, awaitItem())
// Success со списком
assertEquals(NotesUiState.Success(fakeNotes), awaitItem())
}
}
```

13.2.5. runTest и TestDispatcher

runTest из kotlinx-coroutines-test виртуализирует время: delay(1000) мгновенно завершается. Внутри runTest создаются TestScope и TestDispatcher. Для тестов ViewModel нужно подменять Dispatcher через DI (чтобы не хардкодить Dispatchers.Main в коде). Паттерн:

```kotlin
// Зависимость: testImplementation("org.jetbrains.kotlinx:kotlinx-coroutines-test:1.8.1")

interface DispatcherProvider {
val main: CoroutineDispatcher
val io: CoroutineDispatcher
val default: CoroutineDispatcher
}

class DefaultDispatcherProvider : DispatcherProvider {
override val main = Dispatchers.Main
override val io = Dispatchers.IO
override val default = Dispatchers.Default
}

// В тестах:
class TestDispatcherProvider(testDispatcher: TestDispatcher) : DispatcherProvider {
override val main = testDispatcher
override val io = testDispatcher
override val default = testDispatcher
}

@Test
fun `test`() = runTest {
val dp = TestDispatcherProvider(StandardTestDispatcher(testScopeScheduler))
val vm = NotesViewModel(repo, dp)
// ...
}
```

13.2.6. Room DAO тесты — in-memory database

Для тестирования DAO используют in-memory Room: база создаётся в RAM и уничтожается после теста. Не нужно устройство (можно в JVM через Robolectric, но обычно запускают как instrumentation test в src/androidTest/). Шаблон:

```kotlin
@RunWith(AndroidJUnit4::class)
class NoteDaoTest {
private lateinit var db: AppDatabase
private lateinit var dao: NoteDao

@Before
fun setup() {
val ctx = ApplicationProvider.getApplicationContext<Context>()
db = Room.inMemoryDatabaseBuilder(ctx, AppDatabase::class.java)
.allowMainThreadQueries() // только в тестах!
.build()
dao = db.noteDao()
}

@After
fun teardown() { db.close() }

@Test
fun insert_and_observe() = runTest {
dao.insert(NoteEntity(title = "test", content = "x", createdAt = 0L, type = NoteType.Text))
dao.observeAll().test {
val list = awaitItem()
assertEquals(1, list.size)
assertEquals("test", list[0].title)
}
}
}
```

13.2.7. Compose UI-тесты

Compose предоставляет собственный фреймворк UI-тестирования. Главные компоненты:

- createAndroidComposeRule<MyActivity>() — правило, запускающее Activity для теста.
- composeTestRule.onNodeWithText("Hello").assertIsDisplayed() — поиск узла и assertion.
- onNodeWithText("Click").performClick() — действие.
- onAllNodesWithText("Item")[0] — поиск нескольких узлов.
- waitForIdle() — дождаться завершения всех recomposition и корутин.

```kotlin
@HiltAndroidTest
class NoteCardTest {

@get:Rule(order = 0)
val hiltRule = HiltAndroidRule(this)

@get:Rule(order = 1)
val composeRule = createAndroidComposeRule<MainActivity>()

@Test
fun click_on_note_card_triggers_callback() {
val note = Note(1, "Test", "Content", 0L, NoteType.Text)
var clicked = false

composeRule.setContent {
NoteCard(note = note, onClick = { clicked = true })
}

composeRule.onNodeWithText("Test").performClick()
assertTrue(clicked)
}
}
```

13.2.8. Hilt-тестирование: @HiltAndroidTest и @UninstallModules

Для тестирования классов, использующих Hilt-инъекцию, нужен специальный runner и правила. Аннотация @HiltAndroidTest помечает тестовый класс. @UninstallModules(RealModule::class) заменяет модуль на тестовый (с фейковыми реализациями). Также нужен кастомный AndroidJUnitRunner:

```kotlin
// android/app/src/androidTest/java/.../CustomTestRunner.kt
class CustomTestRunner : AndroidJUnitRunner() {
override fun newApplication(cl: ClassLoader?, name: String?, context: Context?):
Application {
return super.newApplication(cl, HiltTestApplication::class.java.name, context)
}
}

// B build.gradle.kts:
android {
defaultConfig {
testInstrumentationRunner = "ru.notesapp.CustomTestRunner"
}
}
```

13.2.9. Зависимости для тестов

KOTLIN

```
dependencies {
// Unit-тесты
testImplementation("junit:junit:4.13.2")
testImplementation("com.google.truth:truth:1.4.4")
testImplementation("io.mockk:mockk:1.13.12")
testImplementation("org.jetbrains.kotlinx:kotlinx-coroutines-test:1.8.1")
testImplementation("app.cash.turbine:turbine:1.1.0")

// Instrumentation-тесты
androidTestImplementation("androidx.test.ext:junit:1.2.1")
androidTestImplementation("androidx.test:runner:1.6.1")
androidTestImplementation("com.google.truth:truth:1.4.4")
androidTestImplementation("androidx.room:room-testing:2.6.1")
androidTestImplementation("androidx.compose.ui:ui-test-junit4:1.7.0")
debugImplementation("androidx.compose.ui:ui-test-manifest:1.7.0")

// Hilt
androidTestImplementation("com.google.dagger:hilt-android-testing:2.51.1")
kaptAndroidTest("com.google.dagger:hilt-compiler:2.51.1")
}
```

13.2.10. Параметризованные тесты — JUnitParams и Parameterized

Когда нужно проверить одно и то же поведение с разными входными данными (например, 10 разных способов передать пустую строку), не пишите 10 одинаковых тестов — используйте параметризованные тесты. Два варианта:

- **JUnitParamsRunner** (библиотека pl.pragmatists:JUnitParams) — аннотация @RunWith(JUnitParamsRunner::class) + @Parameters(method = "...") над тестом. Удобнее и читаемее, чем встроенный в JUnit4 Parameterized.
- **JUnit4 Parameterized** — встроенный runner, но требует отдельного тест-класса на каждый параметризованный тест. Менее гибкий.

```kotlin
@RunWith(JUnitParamsRunner::class)
class NoteDaoSearchTest {

private lateinit var dao: NoteDao
private lateinit var db: AppDatabase

@Before fun setup() { /* in-memory Room */ }
@After fun teardown() { db.close() }

@Parameters(method = "searchQueries")
fun searchQueries() = arrayOf(
arrayOf("test", 1), // query, expectedCount
arrayOf("TEST", 1), // case insensitive
arrayOf("Test", 1),
arrayOf("tes", 1), // partial match
arrayOf("xyz", 0), // no match
arrayOf("", 3), // empty query → all
arrayOf("тест", 1), // Cyrillic
arrayOf("a b", 0), // phrase with space
)

@Test
fun `search returns expected count for each query`(
query: String,
expectedCount: Int,
) = runTest {
// Arrange: insert 3 notes with known titles
dao.insert(note(title = "Test note"))
dao.insert(note(title = "Another"))
dao.insert(note(title = "тест"))

// Act
val results = dao.searchByTitle(query).first()

// Assert
assertThat(results).hasSize(expectedCount)
}
}
```

Один параметризованный тест заменяет 8–10 обычных. При падении видно, на каком именно наборе параметров упало — JUnitParams вставляет параметры в имя теста. Это критически важно для 40 тестов на слой: без параметризации вы утонете в копипасте.

13.2.11. MainDispatcherRule — тестирование ViewModel с корутинами

ViewModel запускает корутины в viewModelScope, который по умолчанию использует Dispatchers.Main.immediate. В unit-тестах на JVM Dispatchers.Main недоступен (нет Android-окружения), и тест упадёт с IllegalStateException: Module with the Main dispatcher had failed to initialize. Решение — подменить Main на TestDispatcher через JUnit Rule.

KOTLIN

```
// MainDispatcherRule.kt
class MainDispatcherRule(
private val testDispatcher: TestDispatcher = StandardTestDispatcher(),
) : TestWatcher() {

override fun starting(description: Description) {
Dispatchers.setMain(testDispatcher)
}

override fun finished(description: Description) {
Dispatchers.resetMain()
}
}

// В тесте:
class NotesViewModelTest {
@get:Rule val mainDispatcherRule = MainDispatcherRule()

private val repo = mockk<NotesRepository>()
private lateinit var viewModel: NotesViewModel

@Before fun setup() {
viewModel = NotesViewModel(repo)
}

@Test
fun `initial state is Loading`() = runTest(mainDispatcherRule.testDispatcher) {
// Arrange: mock repo returns empty flow initially
every { repo.observeAll() } returns emptyFlow()

// Act + Assert
viewModel.uiState.test {
assertEquals(NotesUiState.Loading, awaitItem())
cancelAndIgnoreRemainingEvents()
}
}
}
```

StandardTestDispatcher — корутины не запускаются автоматически, нужно явно advanceUntilIdle() или runCurrent(). UnconfinedTestDispatcher — запускает сразу, удобнее для простых тестов. Для debounce/delay тестов нужен StandardTestDispatcher + advanceTimeBy(300).

13.2.12. Структура тест-класса — AAA pattern

Каждый тест-метод должен следовать паттерну **AAA**: **A**rrange (подготовка данных), **A**ct (вызов), **A**ssert (проверка). Иногда добавляют четвёртый шаг — **C**leanup, но в Kotlin с @Before/@After это не нужно. Главные правила:

• **Один тест — одно поведение**. Не проверяйте 5 разных вещей в одном тесте. Если тест упал, должно быть сразу видно, что именно сломалось.

• **Имя теста описывает поведение**. Не test1, а refresh_success_emits_Success_with_notes. Backtick-имена в Kotlin позволяют писать читаемые имена.

• **Arrange должен быть минимальным**. Если для теста нужна база с 5 заметками, не создавайте их вручную — вынесите helper-функцию notesWith(vararg titles: String).

• **Act — одна строка**. Вызов тестируемого метода. Если нужно несколько действий — это, скорее всего, два теста.

• **Assert — конкретный**. Не assertNotNull(result), а assertThat(result).isEqualTo(expected). Truth и MockK дают хорошие сообщения об ошибках.

Примеры кода

Пример 13.1. NoteDaoTest — структура класса и helpers

KOTLIN

```
@RunWith(AndroidJUnit4::class)
class NoteDaoTest {

private lateinit var db: AppDatabase
private lateinit var dao: NoteDao

@Before
fun setup() {
val ctx = ApplicationProvider.getApplicationContext<Context>()
db = Room.inMemoryDatabaseBuilder(ctx, AppDatabase::class.java)
.allowMainThreadQueries()
.build()
dao = db.noteDao()
}

@After
fun teardown() { db.close() }

// Helper — заметка с дефолтами для быстрых тестов
private fun note(
id: Long = 0L,
title: String = "title",
content: String = "content",
createdAt: Long = System.currentTimeMillis(),
type: NoteType = NoteType.Text,
) = NoteEntity(id, title, content, createdAt, 0L, type, null)

@Test
fun observeAll_empty_db_emits_empty_list() = runTest {
dao.observeAll().test {
assertThat(awaitItem()).isEmpty()
cancelAndIgnoreRemainingEvents()
}
}

@Test
fun observeAll_sorted_by_created_at_desc() = runTest {
dao.insert(note(title = "old", createdAt = 1000L))
dao.insert(note(title = "new", createdAt = 5000L))
dao.insert(note(title = "mid", createdAt = 3000L))
val list = dao.observeAll().first()
assertThat(list.map { it.title })
.containsExactly("new", "mid", "old").inOrder()
}
}
```

Пример 13.2. NotesViewModelTest — структура с MainDispatcherRule

KOTLIN

```
class NotesViewModelTest {

@get:Rule
val mainDispatcherRule = MainDispatcherRule()

private val repo = mockk<NotesRepository>(relaxed = true)
private lateinit var viewModel: NotesViewModel

@Before
fun setup() {
every { repo.observeAll() } returns emptyFlow()
viewModel = NotesViewModel(repo)
}

@Test
fun `initial uiState is Loading`() = runTest {
viewModel.uiState.test {
assertEquals(NotesUiState.Loading, awaitItem())
cancelAndIgnoreRemainingEvents()
}
}

@Test
fun `observeAll with empty list emits Empty`() = runTest {
every { repo.observeAll() } returns flowOf(emptyList())
val vm = NotesViewModel(repo)
vm.uiState.test {
assertEquals(NotesUiState.Empty, awaitItem())
cancelAndIgnoreRemainingEvents()
}
}

@Test
fun `refresh NetworkError does not change uiState to Error`() = runTest {
coEvery { repo.refresh() } returns NetworkResult.NetworkError(IOException("offline"))
val initialState = viewModel.uiState.value
viewModel.refresh()
advanceUntilIdle()
assertEquals(initialState, viewModel.uiState.value)
}
}
```

Пример 13.3. NoteCardTest — Compose UI с asserters

KOTLIN

```
@HiltAndroidTest
class NoteCardTest {

@get:Rule(order = 0)
val hiltRule = HiltAndroidRule(this)

@get:Rule(order = 1)
val composeRule = createAndroidComposeRule<MainActivity>()

@Test
fun note_card_displays_short_title_fully() {
val note = Note(id = 1, title = "Короткий заголовок", content = "контент",
createdAt = 0L, type = NoteType.Text)
composeRule.setContent {
NotesAppTheme { NoteCard(note = note, onClick = {}) }
}
composeRule.onNodeWithText("Короткий заголовок").assertIsDisplayed()
}

@Test
fun note_card_click_triggers_onClick_callback() {
var clicked = false
val note = Note(id = 42, title = "test", content = "x",
createdAt = 0L, type = NoteType.Text)
composeRule.setContent {
NotesAppTheme { NoteCard(note = note, onClick = { clicked = true }) }
}
composeRule.onNodeWithText("test").performClick()
assertTrue(clicked)
}

@Test
fun note_card_Text_type_shows_blue_circle() {
val note = Note(id = 1, title = "x", content = "y",
createdAt = 0L, type = NoteType.Text)
composeRule.setContent {
NotesAppTheme { NoteCard(note = note, onClick = {}) }
}
composeRule.onNodeWithContentDescription("Type indicator")
.assertIsDisplayed()
.assertColorEquals(Color(0xFF3592C0))
}

// ... ещё 37 UI-тестов
}
```

Задание

Напишите тесты для NotesApp. Цель — не менее **40 тестов на каждый слой**: NoteDaoTest (data layer), NotesViewModelTest (presentation layer), NoteCardTest (UI layer), MiniJsonTest (utility layer). Итого ≥ 160 тестов. Покрытие кода ключевых классов — ≥ 80%. Тесты должны быть зелёными и на CI, и локально.

> **Почему так много тестов**

> 40 тестов на слой — это не «для галочки». Это реалистичный минимум для production-кода: около 8–12 happy-path тестов, 15–20 edge-case-ов (пустые значения, ошибки, граничные условия), 5–8 параметризованных тестов, 5–7 regression-тестов под конкретные баги. Меньше — значит, вы тестируете только "как должно работать в идеальном мире", а идеального мира не бывает.

Шаг 1. Зависимости

Добавьте все тестовые зависимости в build.gradle.kts (app), как описано в 13.2.9. Также добавьте плагин JaCoCo для покрытия (опционально): id("org.jacoco"). Дополнительно подключите JUnitParamsRunner для параметризованных тестов — это сильно упростит написание 40 тестов на слой через единый шаблон.

KOTLIN

```
dependencies {
// ... стандартные зависимости из 13.2.9 ...

// Параметризованные тесты (JUnitParams)
testImplementation("pl.pragmatists:JUnitParams:1.1.1")
androidTestImplementation("pl.pragmatists:JUnitParams:1.1.1")

// Robolectric — для запуска instrumentation-тестов на JVM
testImplementation("org.robolectric:robolectric:4.13")
}
```

Шаг 2. CustomTestRunner

Создайте CustomTestRunner в каталоге androidTest/ (как в 13.2.8). Зарегистрируйте его в testInstrumentationRunner в defaultConfig.

Шаг 3. NoteDaoTest (40 тестов, data layer)

В src/androidTest/.../NoteDaoTest.kt напишите минимум 40 тестов. Используйте in-memory Room (как в 13.2.6), runTest и Turbine для Flow. Группируйте тесты по методам DAO — каждая группа по 5-8 тестов: happy path, edge cases, ошибки, параметры. Все 40 тестов должны быть зелёными.

Шаг 3.1. observeAll() — 8 тестов

1. observeAll_empty_db_emits_empty_list
2. observeAll_single_insert_emits_list_with_one_item
3. observeAll_two_inserts_emits_two_items
4. observeAll_sorted_by_created_at_desc
5. observeAll_with_same_created_at_preserves_insertion_order
6. observeAll_after_delete_emits_updated_list
7. observeAll_after_update_emits_updated_fields
8. observeAll_concurrent_inserts_no_data_loss

Шаг 3.2. getById() — 6 тестов

9. getById_existing_returns_note
10. getById_nonexistent_returns_null
11. getById_zero_id_returns_null
12. getById_negative_id_returns_null
13. getById_max_long_id_returns_null

14. getById_after_update_returns_updated_note

Шаг 3.3. insert() — 8 тестов

15. insert_returns_new_id

16. insert_increments_id_sequence

17. insert_with_explicit_id_uses_it

18. insert_replaces_on_conflict_when_strategy_replace

19. insert_throws_on_conflict_when_strategy_abort

20. insert_null_title_throws

21. insert_null_content_throws

22. insert_negative_created_at_succeeds

Шаг 3.4. update() — 6 тестов

23. update_changes_title

24. update_changes_content

25. update_changes_type

26. update_changes_image_url

27. update_nonexistent_id_no_effect

28. update_preserves_id_and_created_at

Шаг 3.5. deleteById() — 5 тестов

29. deleteById_existing_removes_from_db

30. deleteById_nonexistent_returns_zero

31. deleteById_zero_id_no_effect

32. deleteById_after_delete_getById_returns_null

33. deleteById_does_not_affect_other_notes

Шаг 3.6. searchByTitle() — 7 тестов

34. searchByTitle_exact_match_returns_note

35. searchByTitle_partial_match_returns_note

36. searchByTitle_case_insensitive

37. searchByTitle_empty_query_returns_all

38. searchByTitle_no_matches_returns_empty

39. searchByTitle_special_characters_in_query

40. searchByTitle_unicode_cyrillic_query

Шаг 4. NotesViewModelTest (40 тестов, presentation layer)

В src/test/.../NotesViewModelTest.kt напишите минимум 40 unit-тестов с MockK, Turbine и runTest. Подменяйте NotesRepository через mockk, контролируйте Flow через flowOf(...) и MutableSharedFlow. Жизненный цикл ViewModel имитируется через MainDispatcherRule с StandardTestDispatcher.

Шаг 4.1. Initial state — 5 тестов

1. initial_uiState_is_Loading

2. initial_query_is_empty_string

3. initial_isRefreshing_is_false
4. initial_notes_is_empty_list
5. init_subscribes_to_repo_observeAll

Шаг 4.2. observeAll → UiState mapping — 6 тестов

6. observeAll_empty_list_emits_Empty
7. observeAll_single_note_emits_Success
8. observeAll_multiple_notes_emits_Success_with_list
9. observeAll_error_emits_Error_with_message
10. observeAll_throws_Exception_emits_Error
11. observeAll_throws_IOException_emits_Error

Шаг 4.3. addNote() — 7 тестов

12. addNote_calls_repo_add_once
13. addNote_with_empty_title_does_not_call_repo
14. addNote_with_blank_title_does_not_call_repo
15. addNote_with_empty_content_calls_repo
16. addNote_passes_correct_title_to_repo
17. addNote_passes_NoteType_Text_by_default
18. addNote_passes_NoteType_Image_when_imageUrl_provided

Шаг 4.4. deleteNote() — 5 тестов

19. deleteNote_calls_repo_delete_with_id
20. deleteNote_with_invalid_id_still_calls_repo
21. deleteNote_emits_updated_list_after_delete
22. deleteNote_repo_failure_emits_Error
23. deleteNote_repo_success_no_error_emitted

Шаг 4.5. refresh() — 7 тестов

24. refresh_sets_isRefreshing_true_then_false
25. refresh_success_calls_repo_refresh
26. refresh_success_emits_Success_with_notes
27. refresh_NetworkError_emits_no_state_change
28. refresh_HttpError_emits_no_state_change
29. refresh_simultaneous_calls_no_double_state
30. refresh_during_already_refreshing_is_ignored

Шаг 4.6. Search & query — 5 тестов

31. onQueryChange_updates_query_state
32. search_with_min_length_2_triggers_search_use_case
33. search_with_length_1_does_not_trigger_search
34. search_with_empty_query_switches_to_observeAll
35. search_debounce_300ms_filters_intermediate_emissions

Шаг 4.7. Edge cases & cancellation — 5 тестов

36. viewModel_cleared_cancels_observeAll_collector

37. viewModel_cleared_cancels_in_progress_refresh

38. repeated_onQueryChange_same_value_does_not_trigger_search

39. clearState_after_Error_then_refresh_recovers

40. stateIn_WhileSubscribed_unsubscribes_when_no_collectors

Шаг 5. NoteCardTest (40 тестов, UI layer)

В src/androidTest/.../NoteCardTest.kt напишите минимум 40 Compose UI-тестов через createAndroidComposeRule. Используйте onNodeWithText, onNodeWithContentDescription, performClick, assertIsDisplayed, assertTextContains. Каждое поведение UI описывается отдельным тестом — это регрессионная защита от случайных поломок дизайна.

Шаг 5.1. Title rendering — 7 тестов

1. note_card_displays_short_title_fully

2. note_card_displays_long_title_truncated_with_ellipsis

3. note_card_title_uses_titleMedium_style

4. note_card_title_max_lines_is_1

5. note_card_title_overflow_is_ellipsis

6. note_card_title_color_is_onSurface

7. note_card_empty_title_shows_nothing

Шаг 5.2. Summary rendering — 6 тестов

8. note_card_displays_short_summary_fully

9. note_card_displays_long_summary_truncated

10. note_card_summary_uses_bodyMedium_style

11. note_card_summary_max_lines_is_2

12. note_card_summary_includes_type_prefix_T

13. note_card_summary_includes_type_prefix_I

Шаг 5.3. Type indicator color — 5 тестов

14. note_card_Text_type_shows_blue_circle

15. note_card_Image_type_shows_green_circle

16. note_card_Audio_type_shows_orange_circle

17. note_card_circle_size_is_12dp

18. note_card_circle_is_circular_clipped

Шаг 5.4. Click handling — 6 тестов

19. note_card_click_triggers_onClick_callback

20. note_card_click_passes_correct_note_to_callback

21. note_card_double_click_triggers_callback_twice

22. note_card_long_press_does_not_trigger_onClick

23. note_card_click_when_disabled_does_not_trigger

24. note_card_click_ripple_animation_visible

Шаг 5.5. Image notes — 5 тестов

25. note_card_image_note_shows_glide_image

26. note_card_image_note_height_is_120dp

27. note_card_image_note_corners_rounded

28. note_card_image_note_placeholder_during_load

29. note_card_image_note_error_fallback

Шаг 5.6. Modifier & layout — 5 тестов

30. note_card_fills_max_width

31. note_card_padding_is_8dp

32. note_card_card_elevation_default

33. note_card_card_shape_is_material_default

34. note_card_modifier_passed_to_card

Шаг 5.7. NoteList & EmptyState — 6 тестов

35. note_list_empty_shows_EmptyState

36. note_list_empty_does_not_show_any_card

37. note_list_one_item_shows_one_card

38. note_list_three_items_shows_three_cards

39. note_list_click_on_second_card_triggers_callback_with_correct_note

40. note_list_scroll_reveals_hidden_cards

Шаг 6. MiniJsonTest (40 тестов, utility layer)

В src/test/.../MiniJsonTest.kt напишите минимум 40 unit-тестов для сериализатора из практики 12. Группируйте по: базовые типы, null/edge cases, аннотации, round-trip, escape-последовательности, ошибки.

Шаг 6.1. toJson — базовые типы — 8 тестов

1. toJson_string_field_wrapped_in_quotes

2. toJson_int_field_no_quotes

3. toJson_long_field_no_quotes

4. toJson_double_field_no_quotes

5. toJson_float_field_no_quotes

6. toJson_boolean_true_field_lowercase

7. toJson_boolean_false_field_lowercase

8. toJson_null_field_emits_null_without_quotes

Шаг 6.2. toJson — структуры — 6 тестов

9. toJson_single_field_object

10. toJson_three_fields_object

11. toJson_field_separator_is_comma

12. toJson_key_value_separator_is_colon

13. toJson_object_starts_with_brace

14. toJson_object_ends_with_brace

Шаг 6.3. toJson — аннотации — 5 тестов

15. toJson_uses_annotated_name_when_present

16. toJson_uses_property_name_when_annotation_absent

17. toJson_uses_class_annotation_for_root

18. toJson_annotation_empty_string_falls_back_to_property_name

19. toJson_mixed_annotated_and_non_annotated_fields

Шаг 6.4. toJson — escape — 5 тестов

20. toJson_string_with_quote_escaped

21. toJson_string_with_backslash_escaped

22. toJson_string_with_newline_escaped

23. toJson_string_with_tab_escaped

24. toJson_string_with_unicode_preserved

Шаг 6.5. toJson — ошибки — 3 теста

25. toJson_class_without_annotation_throws_IllegalArgumentException

26. toJson_class_without_annotation_error_message_includes_class_name

27. toJson_non_data_class_with_annotation_throws_or_succeeds

Шаг 6.6. fromJson — базовые типы — 5 тестов

28. fromJson_string_field_parsed_correctly

29. fromJson_int_field_parsed_correctly

30. fromJson_long_field_parsed_correctly

31. fromJson_boolean_field_parsed_correctly

32. fromJson_null_field_becomes_null_property

Шаг 6.7. fromJson — структуры и аннотации — 4 теста

33. fromJson_single_field_object

34. fromJson_multiple_fields_object

35. fromJson_uses_annotated_name_to_map_to_property

36. fromJson_missing_optional_field_uses_default

Шаг 6.8. Round-trip & edge cases — 4 теста

37. round_trip_user_with_email_preserves_all_fields

38. round_trip_user_with_null_email_preserves_null

39. round_trip_simple_note_preserves_all_fields

40. round_trip_unicode_strings_preserved

Шаг 7. Запуск и покрытие

Запустите все тесты: ./gradlew test (unit, MiniJson + ViewModel) + ./gradlew connectedAndroidTest (instrumentation, NoteDao + NoteCard). Проверьте, что все 160+ тестов зелёные. Если добавили JaCoCo — ./gradlew jacocoTestReport и откройте отчёт в build/reports/jacoco/. Цель — ≥ 80% покрытия на классах ViewModel, Repository, MiniJson, NoteDao. Менее 60% на любом из этих классов — работа не зачтётся.

Контрольные вопросы

Ответьте письменно на следующие вопросы. Подготовьте устные ответы для защиты практической работы — преподаватель может задать любой из них или попросить пояснить код вашего решения.

1. Что такое пирамида тестов? Какие уровни есть в Android?

2. Чем unit-тесты (src/test) отличаются от instrumentation (src/androidTest)?

3. Зачем нужен JUnit4? Какие аннотации вы знаете?

4. Чем Truth assertion лучше JUnit assertEquals?

5. Зачем нужен MockK? Чем отличается от Mockito-Kotlin?

6. Что делает coVerify в MockK? Чем отличается от verify?

7. Что такое Turbine? Зачем она нужна для Flow?

8. Чем awaitItem() в Turbine отличается от toList()?

9. Зачем нужен runTest? Что виртуализируется?

10. Зачем нужен DispatcherProvider? Почему нельзя напрямую использовать Dispatchers.Main в ViewModel?

11. Как тестировать Room DAO без устройства? Что такое in-memory database?

12. Как написать Compose UI-тест? Что такое createAndroidComposeRule?

13. Зачем нужен @HiltAndroidTest? Что делает @UninstallModules?

14. Как измерить покрытие кода тестами? Что такое JaCoCo?

Чек-лист сдачи работы

Перед сдачей работы преподавателю убедитесь, что выполнены все пункты ниже. Отсутствие любого пункта снижает оценку минимум на один балл.

- □ В build.gradle.kts добавлены все тестовые зависимости
- □ Подключён JUnitParams для параметризованных тестов
- □ Создан CustomTestRunner : AndroidJUnitRunner
- □ testInstrumentationRunner указывает на CustomTestRunner
- □ В src/androidTest/.../NoteDaoTest.kt есть 40+ тестов
- □ NoteDaoTest использует Room.inMemoryDatabaseBuilder
- □ @Before создаёт БД, @After закрывает
- □ Тесты observeAll/getById/insert/update/deleteById/searchByTitle — все 6 групп
- □ В src/test/.../NotesViewModelTest.kt есть 40+ тестов
- □ NotesViewModelTest использует mockk(), every/coVerify, runTest, Turbine
- □ Есть MainDispatcherRule с StandardTestDispatcher
- □ Тесты initial state / observeAll mapping / addNote / deleteNote / refresh / search / edge cases — все 7 групп
- □ В src/androidTest/.../NoteCardTest.kt есть 40+ тестов
- □ NoteCardTest использует @HiltAndroidTest и createAndroidComposeRule
- □ Тесты title / summary / type indicator / click / image / modifier / NoteList — все 7 групп
- □ В src/test/.../MiniJsonTest.kt есть 40+ тестов

- □ MiniJsonTest покрывает toJson и fromJson по всем базовым типам
- □ Есть escape-тесты для quote, backslash, newline, tab, unicode
- □ Есть round-trip тесты для User с email и с null email
- □ Есть тест на IllegalArgumentException для класса без @MiniSerializable
- □ Все 160+ тестов проходят (./gradlew test + ./gradlew connectedAndroidTest)
- □ Покрытие кода ≥ 80% на ViewModel, Repository, MiniJson, NoteDao
- □ Менее 60% на любом из ключевых классов — работа не зачтётся

Что сдавать

В результате выполнения практической работы студент предоставляет:

1. Git-коммит «Практика 13» со всеми тестовыми классами и CustomTestRunner;

2. Скриншот прохождения ./gradlew test — видно 40+ MiniJson + 40+ ViewModel тестов зелёные;

3. Скриншот прохождения ./gradlew connectedAndroidTest — видно 40+ NoteDao + 40+ NoteCard тестов зелёные;

4. Отчёт JaCoCo (HTML) с покрытием ≥ 80% на ключевых классах;

5. Таблица с количеством тестов по слоям: NoteDao (40), ViewModel (40), NoteCard (40), MiniJson (40), итого 160+.

Практика 14. Финальный проект: сборка приложения

Трудоёмкость: 5 академических часов. **Форма работы:** индивидуальная, в IDE Android Studio. **Форма отчёта:** исходный код проекта + устная защита.

Цель и задачи

Цель работы — интегрировать все знания, полученные за предыдущие 13 практик, в законченное Android-приложение NotesApp. Это финальный проект практикума: студент должен продемонстрировать владение архитектурой MVVM + Repository, Jetpack Compose + Material 3, Room + Retrofit + Hilt + Coil, корутинами и Flow, тестами. По итогам выполняется устная защита с презентацией проекта и демонстрацией работы.

1. провести ревью архитектуры NotesApp на соответствие clean layer pattern;
2. разбить проект на Gradle-модули (app, core, data, domain, ui);
3. настроить ProGuard/R8 для release-сборки;
4. собрать signed APK / App Bundle;
5. подготовить презентацию 5–7 минут и демо приложения;
6. ответить на расширенный набор контрольных вопросов (15 шт).

Теоретическая часть

14.2.1. Архитектура MVVM + Repository + Clean Layers

Современное Android-приложение обычно строится по рекомендациям Guide to App Architecture от Google. Главные слои:

• **UI layer (presentation)** — Composable-функции, ViewModel, UiState. Отвечает за отображение состояния и проброс пользовательских действий в ViewModel.

• **Domain layer** (опционально) — UseCase-классы, инкапсулирующие бизнес-логику. В NotesApp это SearchUseCase, SyncAllUseCase.

• **Data layer** — Repository, DataSource (Room, Retrofit), модели Entity/DTO, mappers. Реализует SSOT и offline-first.

Принципы: 1) Unidirectional Data Flow (события вверх, состояние вниз); 2) Single Source of Truth (Repository — единственный источник данных для UI); 3) separation of concerns (UI не знает про БД, БД не знает про UI); 4) dependency rule (зависимости идут от внешних слоёв к внутренним: UI → domain → data).

14.2.2. Gradle multi-module

Когда приложение растёт, его разбивают на модули. Стандартная структура (от Google):

Модуль	Назначение	Зависимости
`:app`	Точка входа, Application, MainActivity, навигация	`:core, :data, :domain, :ui`
`:core`	Базовые утилиты, model-классы, константы	—
`:data`	Repository, Room Entity/Dao, Retrofit Api, DTO	`:core, :domain`
`:domain`	UseCase, доменные модели (Note, User), интерфейсы репозиториев	`:core`
`:ui`	Compose-экраны, ViewModel, тема	`:core, :domain`
`:feature-*`	(опционально) отдельные feature-модули	`:core, :ui`

Преимущества multi-module: 1) ускорение сборки (incremental build перестраивает только изменённые модули); 2) чёткие границы (нельзя случайно залезть из UI в Room); 3) возможность переиспользования модулей в других приложениях; 4) динамическая поставка (Dynamic Feature Modules). На финальном проекте достаточно базовой структуры: app + core + data + domain + ui.

14.2.3. ProGuard / R8 — обфускация и оптимизация

R8 — компилятор от Google, заменил ProGuard. Делает три операции: 1) shrinking — удаляет неиспользуемый код; 2) optimization — инлайнинг, удаление мёртвого кода; 3) obfuscation — переименование классов/методов в короткие имена (a, b, c). Уменьшает размер APK в 2–5 раз. Включается в build.gradle.kts:

```kotlin
android {
buildTypes {
release {
isMinifyEnabled = true
isShrinkResources = true
proguardFiles(
getDefaultProguardFile("proguard-android-optimize.txt"),
"proguard-rules.pro",
)
}
}
}
```

Важные keep-правила для NotesApp (в файле proguard-rules.pro):

```text
# Room entities and DAOs — не удалять
-keep class ru.notesapp.data.local.** { *; }

# Retrofit / Moshi — keep generic signatures
-keepattributes Signature
-keepattributes *Annotation*
-keep class ru.notesapp.data.remote.** { *; }

# Kotlin Metadata (для рефлексии, если используете)
-keep class kotlin.Metadata { *; }

# Hilt — уже встроены правила в Hilt-gradle-plugin
```

14.2.4. Подпись APK / App Bundle

Release-сборка должна быть подписана. Для генерации keystore:

```kotlin
keytool -genkeypair -v \
-keystore notesapp.jks \
-keyalg RSA -keysize 2048 -validity 10000 \
-alias notesapp

# B build.gradle.kts:
android {
signingConfigs {
create("release") {
storeFile = file("notesapp.jks")
storePassword = System.getenv("NOTESAPP_STORE_PASSWORD")
keyAlias = "notesapp"
keyPassword = System.getenv("NOTESAPP_KEY_PASSWORD")
}
}
buildTypes {
release {
signingConfig = signingConfigs.getByName("release")
}
}
}
```

Хранить пароли в репозитории нельзя — используйте переменные окружения или локальный local.properties. Для публикации в Google Play нужно собрать App Bundle (AAB): ./gradlew :app:bundleRelease. Google Play сам подписывает финальный APK для каждого устройства.

14.2.5. Тестирование release-сборки

Release-сборка может вести себя иначе, чем debug: минификация может удалить нужный код, обфускация ломает рефлексию. Перед публикацией обязательно:

1. Запустить ./gradlew assembleRelease без ошибок.

2. Установить release-APK на устройство (через adb install app-release.apk).

3. Пройти ключевые сценарии: создание/удаление/поиск заметок, синхронизация с сетью, офлайн-режим.

4. Проверить Logcat на наличие ClassNotFoundError, NoSuchMethodException — типичные симптомы агрессивной минификации.

5. Запустить instrumentation-тесты на release-сборке: ./gradlew connectedAndroidTestRelease (если тесты покрывают критичный путь).

14.2.6. Презентация финального проекта

Защита состоит из устной презентации (5–7 минут) и демонстрации работающего приложения. Структура презентации:

1. **Слайд 1: Титульный.** Название проекта, ФИО студента, группа.
2. **Слайд 2: Постановка задачи.** Какую проблему решает приложение, целевая аудитория.
3. **Слайд 3: Архитектура.** Схема слоёв, dependency rule, ключевые паттерны (MVVM, SSOT, UDF).
4. **Слайд 4: Стек технологий.** Список библиотек с версиями.
5. **Слайд 5: Структура проекта.** Package layout, модули, количество классов.

6. **Слайд 6: Ключевые фичи.** Скриншоты 3-4 экранов.
7. **Слайд 7: Тесты и покрытие.** Количество тестов, % покрытия.
8. **Слайд 8: Что было сложным.** 2-3 нетривиальные задачи, с которыми столкнулись, и их решение.
9. **Демо.** Запуск приложения на устройстве/эмуляторе, прохождение 2-3 сценариев.

14.2.7. Ревью архитектуры NotesApp

Перед финальной сборкой проверьте, что проект соответствует следующим критериям:

Критерий	Требование
`Пакетная структура`	ui / domain / data / di / util — разделены
`Архитектура`	MVVM: ViewModel → UseCase → Repository → DAO/Api
`DI`	Hilt: @HiltAndroidApp, @AndroidEntryPoint, @HiltViewModel
`Network`	Retrofit + Moshi, NetworkResult sealed, OkHttp с интерсепторами
`Persistence`	Room с миграциями, TypeConverter для NoteType
`UI`	Jetpack Compose + Material 3, кастомная тема
`State`	StateFlow + collectAsStateWithLifecycle, sealed UiState
`Search`	Flow с debounce + distinctUntilChanged + flatMapLatest
`Images`	Coil AsyncImage с placeholder/error
`Navigation`	Navigation-Compose с маршрутами и BottomBar
`Tests`	Unit + instrumentation, покрытие ≥ 60%
`Build`	Release APK с подписью, ProGuard/R8 включён

14.2.8. Package layout — рекомендуемая структура

```text
ru.notesapp/
├── NotesApp.kt # @HiltAndroidApp
├── MainActivity.kt # @AndroidEntryPoint
├── di/
│ ├── NetworkModule.kt
│ ├── DatabaseModule.kt
│ ├── RepositoryModule.kt
│ └── ImageLoaderModule.kt
├── data/
│ ├── local/
│ │ ├── AppDatabase.kt
│ │ ├── NoteDao.kt
│ │ ├── NoteEntity.kt
│ │ ├── NoteTypeConverter.kt
│ │ └── Mappers.kt
│ ├── remote/
│ │ ├── NoteApi.kt
│ │ ├── NoteDto.kt
│ │ ├── NetworkResult.kt
│ │ └── NetworkModule.kt
│ ├── NotesRepository.kt # interface
│ └── SyncedNotesRepository.kt # implementation
├── domain/
│ ├── Note.kt
│ ├── NoteType.kt
│ ├── NoteExt.kt
│ ├── User.kt
│ ├── SearchUseCase.kt
│ └── SyncAllUseCase.kt
├── ui/
│ ├── theme/ (Color.kt, Type.kt, Theme.kt, Shape.kt)
│ ├── navigation/ (Routes.kt)
│ ├── NoteCard.kt
│ ├── NoteList.kt
│ ├── NoteImage.kt
│ ├── EmptyState.kt
│ ├── NotesListScreen.kt
│ ├── NoteDetailScreen.kt
│ ├── SettingsScreen.kt
│ ├── PhotoGalleryScreen.kt
│ ├── BannerAdView.kt
│ └── RootScreen.kt
├── viewmodel/
│ ├── NotesUiState.kt
│ └── NotesViewModel.kt
└── util/
├── MiniSerializable.kt
└── MiniJson.kt
```

14.2.9. App Bundle (.aab) vs APK

С августа 2021 года Google Play требует публиковать приложения в формате **Android App Bundle** (.aab), а не .apk. App Bundle — это «исходник» для Google Play: содержит весь код и ресурсы, но не подписан и не оптимизирован под конкретное устройство. Google Play на лету генерирует оптимизированные APK под каждое устройство (по плотности экрана, ABI, языку), что уменьшает размер установки на 30–50%.

Свойство	APK	AAB
Расширение	.apk	.aab
Подпись	Обязательно (keystore)	Подписывается upload key, Google пересоздаёт своим ключом
Размер установки	Один APK на все устройства	Оптимизированный APK под каждое устройство
Dynamic delivery	Нет	Да (можно доставлять фичи по запросу)
Локальная установка	Да (adb install)	Нет (нужен bundletool)
Публикация в Google Play	С 2021 запрещена	Обязательно
Сборка	./gradlew assembleRelease	./gradlew bundleRelease

Для локальной установки .aab используйте bundletool: bundletool build-apks --bundle=app.aab --output=app.apks --mode=universal — создаст универсальный APK. Для проверки release-сборки на устройстве этого достаточно. В финальном проекте соберите и .apk (для демо), и .aab (для теоретической публикации).

14.2.10. Crashlytics и мониторинг в production

Production-приложение без мониторинга — чёрный ящик. Если пользователь столкнулся с крашом, вы об этом не узнаете, пока не придёт плохой отзыв. Стандартный stack для мониторинга Android-приложений:

Сервис	Назначение	Альтернативы
Firebase Crashlytics	Автоматический сбор краш-репортов с stack trace	Bugsnag, Sentry, Crashloop
Firebase Performance Monitoring	Slow frames, network latency, cold start	Sentry Performance, Datadog RUM
Firebase Analytics	События в приложении, retention, funnels	Amplitude, Mixpanel, Yandex Metrica
Firebase Cloud Messaging	Push-уведомления	OneSignal, Yandex Push
LeakCanary (debug)	Поиск утечек памяти в debug-сборке	Square Detective
Timber + Bugfender	Remote logging из release-сборки	Logcat в production недоступен

В финальном проекте обязательно: подключить Crashlytics (минимум — для учебной публикации) и LeakCanary в debug. Analytics — опционально, но полезно для демо: можно показать преподавателю live-график событий в Firebase консоли.

KOTLIN

```
// build.gradle.kts (app)
dependencies {
// Crashlytics
implementation(platform("com.google.firebase:firebase-bom:33.1.0"))
implementation("com.google.firebase:firebase-crashlytics-ktx")
implementation("com.google.firebase:firebase-analytics-ktx")

// LeakCanary — только debug
debugImplementation("com.squareup.leakcanary:leakcanary-android:2.14")
}

// В Application классе:
@HiltAndroidApp
class NotesApp : Application() {
override fun onCreate() {
super.onCreate()
if (BuildConfig.DEBUG) {
Timber.plant(Timber.DebugTree())
} else {
Timber.plant(CrashlyticsTree())
}
}
}

class CrashlyticsTree : Timber.Tree() {
override fun log(priority: Int, tag: String?, message: String, t: Throwable?) {
if (priority >= Log.WARN) {
FirebaseCrashlytics.getInstance().log("$tag: $message")
t?.let { FirebaseCrashlytics.getInstance().recordException(it) }
}
}
}
```

14.2.11. CI/CD — GitHub Actions для Android

CI (Continuous Integration) — автоматический запуск тестов на каждый pull request. CD (Continuous Delivery) — автоматическая сборка release-APK на каждый коммит в main. GitHub Actions — самый доступный CI/CD для Android-проектов (бесплатный для public-репозиториев, 2000 минут/месяц для private).

```yaml
# .github/workflows/android.yml
name: Android CI

on:
push:
branches: [main, develop]
pull_request:
branches: [main]

jobs:
unit-tests:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- name: Setup JDK 17
uses: actions/setup-java@v4
with:
java-version: '17'
distribution: 'temurin'
cache: gradle
- name: Setup Android SDK
uses: android-actions/setup-android@v3
- name: Run unit tests
run: ./gradlew testDebugUnitTest
- name: Upload test report
if: always()
uses: actions/upload-artifact@v4
with:
name: unit-test-report
path: app/build/reports/tests/testDebugUnitTest/
```

Для instrumentation-тестов на CI нужен эмулятор — используйте reactivecircus/android-emulator-runner. Для release-сборки — отдельная job с матрицей по API levels:

```yaml
instrumentation-tests:
runs-on: ubuntu-latest
strategy:
matrix:
api-level: [29, 34]
steps:
- uses: actions/checkout@v4
- uses: reactivecircus/android-emulator-runner@v2
with:
api-level: ${{ matrix.api-level }}
arch: x86_64
script: ./gradlew connectedAndroidTest

build-release:
needs: [unit-tests]
runs-on: ubuntu-latest
if: github.ref == 'refs/heads/main'
steps:
- uses: actions/checkout@v4
- name: Build release AAB
run: ./gradlew bundleRelease
env:
NOTESAPP_STORE_PASSWORD: ${{ secrets.STORE_PASSWORD }}
NOTESAPP_KEY_PASSWORD: ${{ secrets.KEY_PASSWORD }}
- uses: actions/upload-artifact@v4
with:
name: release-aab
path: app/build/outputs/bundle/release/app-release.aab
```

14.2.12. Code style — ktlint и detekt

Единый стиль кода — половина читаемости команды. Два инструмента: **ktlint** (авто-форматирование по официальным Kotlin guidelines) и **detekt** (статический анализ на code smells, цикломатическую сложность, потенциальные баги). Подключаются как Gradle-плагины и могут падать в CI при нарушениях.

```kotlin
# build.gradle.kts (project)
plugins {
id("org.jlleitschuh.gradle.ktlint") version "12.1.0"
id("io.gitlab.arturbosch.detekt") version "1.23.6"
}

# build.gradle.kts (app)
plugins {
id("org.jlleitschuh.gradle.ktlint")
id("io.gitlab.arturbosch.detekt")
}

detekt {
buildUponDefaultConfig = true
config.setFrom("$projectDir/config/detekt.yml")
}

# Команды:
# ./gradlew ktlintCheck — проверить
# ./gradlew ktlintFormat — авто-формат
# ./gradlew detekt — статический анализ
```

Задание

Завершите NotesApp как финальный проект практикума. Все компоненты из практик 1–13 должны быть интегрированы и работать вместе. Соберите release-APK, подготовьте презентацию и продемонстрируйте приложение на защите.

Шаг 1. Ревью кода

Пройдитесь по чек-листу 14.2.7. Если что-то не соответствует — поправьте. Особое внимание: пакетная структура (нет ли классов в корне), разделение слоёв (UI не должен импортировать Room), DI (все зависимости через Hilt, никаких ручных созданий).

Шаг 2. Multi-module (опционально, но рекомендуется)

Разбейте проект на модули: :app, :core, :data, :domain, :ui. Базовый шаблон смотрите в 14.2.2. Если не успеваете — достаточно хорошо организованной пакетной структуры в одном модуле.

Шаг 3. ProGuard / R8

В build.gradle.kts (app) включите isMinifyEnabled = true и isShrinkResources = true для release. Создайте proguard-rules.pro с keep-правилами из 14.2.3.

Шаг 4. Подпись release

Сгенерируйте keystore (см. 14.2.4), настройте signingConfig. Пароли — через переменные окружения, НЕ коммитьте в репозиторий. Соберите: ./gradlew :app:assembleRelease. APK появится в app/build/outputs/apk/release/.

Шаг 5. Тестирование release-сборки

Установите release-APK на устройство через adb install app-release.apk. Пройдите сценарии:

1. Создание заметки (текстовой, Image с URL)
2. Редактирование заметки
3. Удаление заметки (свайпом или через меню)
4. Поиск с debounce
5. Синхронизация с JSONPlaceholder
6. Офлайн-режим (отключить интернет, перезапустить)
7. Смена темы (Pastel ↔ Forest) и тёмной темы
8. Просмотр деталей заметки
9. Просмотр галереи изображений

Шаг 6. Презентация

Подготовьте презентацию по структуре из 14.2.6. Используйте Pptx или аналогичный инструмент. Длительность — 5–7 минут. Обязательно: схема архитектуры (нарисуйте сами), скриншоты экранов, цифры покрытия тестами.

Шаг 7. Демонстрация

Подготовьте устройство или эмулятор с установленным release-APK. Демонстрация — 2-3 минуты. Покажите: создание заметки, поиск, смену темы. Если есть время — офлайн-режим.

Шаг 8. Защита

Преподаватель задаёт 3-5 вопросов из расширенного списка контрольных (ниже). Оценка за финальный проект формируется из оценок за презентацию, демо, ответы на вопросы и качество кода (по чек-листу 14.2.7).

Шаг 9. Crashlytics и LeakCanary (опционально, но рекомендуется)

Подключите Firebase Crashlytics и LeakCanary по образцу 14.2.10. Crashlytics нужен для учебной публикации в Google Play Console (даже если не публикуете реально — настройте, покажите, что отчёт об искусственном crash доходит до консоли). LeakCanary в debug-сборке — на каждом выходе из Activity проверяет, что нет утечек. Если LeakCanary найдёт утечку — исправьте её.

Шаг 10. CI/CD pipeline (опционально, но рекомендуется)

Настройте GitHub Actions workflow по образцу 14.2.11. На каждый pull request должны автоматически запускаться unit-тесты. На каждый merge в main — собираться release-AAB и выкладываться как artifact. Если есть аккаунт Firebase Distribution — можно автоматически распределять AAB тестировщикам.

Шаг 11. Code style (опционально)

Подключите ktlint и detekt (14.2.12). Все файлы проекта должны проходить ./gradlew ktlintCheck без ошибок. ./gradlew detekt — без ошибок уровня error (warning-и допустимы, но постарайтесь их минимизировать).

> **Минимальные требования для допуска к защите**
>
> **1.** Все 13 предыдущих практик сданы и зачтены.
> **2.** Release-APK собирается без ошибок и запускается.
> **3.** Покрытие тестами ≥ 50%.
> **4.** Презентация готова (минимум 6 слайдов из 8).
> **5.** Демо на устройстве/эмуляторе работает без crash.

Контрольные вопросы (расширенные)

Для финальной защиты — 15 расширенных вопросов, охватывающих все темы практикума. Преподаватель выбирает 3-5 из списка.

1. Опишите архитектуру NotesApp. Какие слои вы выделили и почему?

2. Какие паттерны проектирования вы применили? (MVVM, Repository, UseCase, Singleton, Observer — минимум 3).

3. Почему для DI выбран Hilt, а не Dagger или Koin? Сравните подходы.

4. Как в вашем проекте реализован offline-first? Опишите поток данных при создании новой заметки без интернета.

5. Что такое Single Source of Truth? Где он в вашем коде?

6. Опишите lifecycle ViewModel. Что происходит при повороте экрана? Что переживает поворот, а что — нет?

7. Чем StateFlow отличается от SharedFlow? Где вы использовали каждый?

8. Как работает debounce в поиске? Что произойдёт, если убрать flatMapLatest?

9. Опишите миграцию Room 1→2→3 в вашем проекте. Что было бы, если бы вы не написали миграции?

10. Как тестировать ViewModel с корутинами? Что нужно подменять и почему?

11. Опишите процесс сборки release-APK. Какие этапы проходят в R8?

12. Что такое keep-правила в ProGuard? Какие вы написали и почему?

13. Какие метрики вы бы добавили в NotesApp для мониторинга в продакшене? (Crashlytics, Performance, Analytics)

14. Что бы вы улучшили в проекте при наличии ещё 2 недель времени? Назовите минимум 3 улучшения.

15. Сравните Compose и View. Когда вы бы выбрали View для нового проекта?

Критерии оценки финального проекта

Итоговая оценка за практикум формируется как взвешенная сумма оценок за все 14 практик. Финальный проект имеет наибольший вес (30% итоговой оценки). Распределение веса:

Компонент	Вес	Что оценивается
Практики 1-13	50%	Зачёт/незачёт каждой практики
Финальный проект (код)	20%	Архитектура, тесты, чек-лист 14.2.7
Презентация	10%	Полнота, структура, наглядность
Демонстрация	10%	Работоспособность, smooth UX
Защита (вопросы)	10%	Глубина понимания, ответы на 3-5 вопросов

Шкала оценок по каждому компоненту:

Оценка	Критерий
Отлично (5)	Все пункты чек-листа выполнены, тесты ≥ 60% покрытия, защита без замечаний
Хорошо (4)	1-2 пункта чек-листа не выполнены, тесты 40-60%, защита с мелкими замечаниями
Удовлетворительно (3)	3-5 пунктов не выполнены, тесты < 40%, защита с замечаниями
Неудовлетворительно (2)	Более 5 пунктов не выполнены, release-APK не собирается, нет презентации

Чек-лист сдачи работы

Перед сдачей работы преподавателю убедитесь, что выполнены все пункты ниже. Отсутствие любого пункта снижает оценку минимум на один балл.

- □ Все 13 предыдущих практик сданы и зачтены
- □ NotesApp компилируется в debug-режиме без warnings
- □ NotesApp компилируется в release-режиме (isMinifyEnabled = true)

- □ Подписан release-APK через keystore
- □ ProGuard rules корректны (release-APK запускается без crash)
- □ Покрытие тестами ≥ 60% (JaCoCo отчёт приложен)
- □ Пакетная структура соответствует 14.2.8
- □ Все зависимости предоставляются через Hilt
- □ Repository pattern: интерфейс + реализация
- □ Offline-first: список работает без интернета
- □ Поиск с debounce 300 мс
- □ Material 3 тема: Pastel и Forest, light/dark
- □ Navigation-Compose с 3 экранами и BottomBar
- □ Coil для изображений с placeholder и error
- □ Презентация содержит 6+ слайдов
- □ Демо на устройстве/эмуляторе работает без crash
- □ Готовы ответы на 15 контрольных вопросов

Что сдавать

В результате выполнения практической работы студент предоставляет:

1. Git-репозиторий с тегом «v1.0-final» и всеми 14 практиками;
2. Release-APK (app-release.apk) в /download или прикреплённый к коммиту;
3. Отчёт JaCoCo (HTML, в /build/reports/jacoco/);
4. Презентация (.pptx или .pdf) с 6+ слайдами;
5. Финальный текстовый отчёт (2-3 страницы): архитектура, технологии, метрики, выводы;
6. Подготовленное демо на устройстве или эмуляторе.